

BỘ XÂY DỰNG
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



BÁO CÁO THỰC TẬP TỐT NGHIỆP

ĐỀ TÀI:

**XÂY DỰNG HỆ THỐNG QUẢN LÝ CHẤM CÔNG THÔNG MINH TÍCH
HỢP NHẬN ĐIỆN KHUÔN MẶT VÀ TRỢ LÝ ẢO AI TRÊN NỀN TẢNG
DI ĐỘNG**

Ngành : Công nghệ thông tin

Chuyên ngành : Công nghệ thông tin

Giảng viên hướng dẫn: Nguyễn Văn Huy

Sinh viên thực hiện : Lê Quốc Trí

MSSV : 068205011936

Lớp : CN2302D

TP. Hồ Chí Minh, ngày 25 tháng 04 năm 2026

LỜI NÓI ĐẦU

Trong bối cảnh Cách mạng Công nghiệp 4.0, quá trình số hóa nguồn nhân lực đang diễn ra mạnh mẽ ở hầu hết các tổ chức, từ trường học đến doanh nghiệp sản xuất, dịch vụ. Các nghiên cứu cho thấy việc áp dụng công nghệ số, đặc biệt là hệ thống chấm công và giám sát thời gian làm việc, góp phần quan trọng vào nâng cao năng suất, kỷ luật lao động và tính minh bạch trong quản trị nhân sự.

Các phương pháp chấm công truyền thống như ký tên giấy, quét thẻ từ, máy chấm công vân tay hay mã số đang bộc lộ nhiều hạn chế: tốn thời gian, dễ sai sót, phát sinh gian lận “chấm công hộ”, khó tích hợp dữ liệu và khó đáp ứng nhu cầu làm việc linh hoạt, từ xa hoặc luân chuyển vị trí. Điều này dẫn tới gia tăng khối lượng xử lý thủ công cho bộ phận nhân sự, giảm độ chính xác số liệu và làm chậm quá trình ra quyết định quản lý.

Song song với đó, sự phát triển nhanh của thiết bị di động, GPS, IoT và các thuật toán nhận diện khuôn mặt đã mở ra hướng tiếp cận mới cho bài toán chấm công. Nhiều công trình nghiên cứu, hệ thống thử nghiệm và sản phẩm thực tế đã chứng minh hiệu quả của giải pháp chấm công di động kết hợp nhận diện khuôn mặt và định vị địa lý (GPS/geofencing) trong việc tăng độ chính xác, hạn chế gian lận và tự động hóa quy trình.

Các hệ thống như WorkMate – nền tảng quản lý lực lượng lao động đa tổ chức trên cloud – tích hợp chấm công bằng GPS/QR, tính lương tự động, chatbot AI trả lời câu hỏi HR, cùng phân tích lực lượng lao động theo thời gian thực, đã cho thấy sự cải thiện rõ rệt về hiệu quả hành chính, độ chính xác dữ liệu và sự hài lòng của nhân viên. Ở môi trường giáo dục, nhiều nghiên cứu đề xuất ứng dụng chấm công sinh viên trên nền tảng Android, kết hợp geo-fencing vùng lớp học với nhận diện khuôn mặt: hệ thống tự động ghi nhận chuyên cần khi sinh viên ở trong vùng địa lý lớp và được xác thực khuôn mặt trong phần lớn thời lượng buổi học, giúp nâng cao độ tin cậy dữ liệu chuyên cần.

Trong lĩnh vực doanh nghiệp, các giải pháp “smart attendance” sử dụng deep learning cho nhận diện khuôn mặt, kết hợp GPS và geofencing, đã đạt độ chính xác nhận diện trên 95%, đồng thời ràng buộc vị trí chấm công trong khu vực cho phép để đảm bảo nhân viên thực sự có mặt tại nơi làm việc. Một số hệ thống còn tích hợp cơ chế liveness detection nhằm chống giả mạo bằng ảnh hoặc video, hạn chế tối đa hành vi gian lận chấm công. Các nghiên cứu đánh giá cho thấy những hệ thống này giúp giảm đáng kể tình trạng “buddy punching”, tăng tính trách nhiệm, và hỗ trợ phân tích xu hướng đi trễ/vắng mặt để phân bổ nhân sự hiệu quả hơn.

Bối cảnh trên đặt ra nhu cầu cấp thiết phải xây dựng những ứng dụng chấm công đơn giản, linh hoạt nhưng đủ tin cậy, có thể triển khai trên điện thoại thông minh – thiết bị mà hầu như mọi người lao động đều sở hữu. Ứng dụng chấm công di động không chỉ giải quyết bài toán điểm danh tại văn phòng, mà còn phù hợp với mô hình làm việc ngoài hiện trường, tại công trình, hoặc làm việc từ xa, vốn ngày càng phổ biến trong thời kỳ hậu COVID-19.

Xuất phát từ thực tiễn đó, đề án lựa chọn đề tài xây dựng ứng dụng chấm công trên di động (app chấm công) với mục tiêu chính là:

Thay thế hoặc hỗ trợ hệ thống chấm công truyền thống bằng giải pháp linh hoạt, thân thiện, dễ sử dụng;

Ứng dụng các kỹ thuật định vị GPS/geofencing và nhận diện khuôn mặt/ảnh selfie kết hợp vị trí để nâng cao độ chính xác và hạn chế gian lận.

Tự động hóa tối đa quy trình ghi nhận, lưu trữ và tổng hợp dữ liệu chấm công, tạo tiền đề tích hợp với hệ thống lương thưởng, đánh giá hiệu suất trong tương lai .

Đề án hướng tới thiết kế kiến trúc hệ thống gồm: ứng dụng di động cho nhân viên chấm công; dịch vụ backend và cơ sở dữ liệu lưu trữ thông tin người dùng, ca làm việc và bản ghi chấm công; cùng giao diện web cơ bản cho quản trị viên xem báo cáo, tra cứu và xuất dữ liệu . Trên cơ sở đó, sinh viên có cơ hội vận dụng tổng hợp kiến thức về lập trình di động, thiết kế cơ sở dữ liệu, lập trình web dịch vụ, bảo mật và, nếu có điều kiện, thử nghiệm một số thuật toán xử lý ảnh/nhận diện khuôn mặt đơn giản.

Về mặt ý nghĩa thực tiễn, ứng dụng chấm công di động mà đề án xây dựng có thể được tùy biến áp dụng cho: doanh nghiệp vừa và nhỏ chưa có điều kiện đầu tư hệ thống chấm công phức tạp; các đơn vị sử dụng nhân sự làm việc ngoài hiện trường; hoặc môi trường giáo dục/trung tâm đào tạo cần quản lý chuyên cần linh hoạt 6789+3 MORE. Đồng thời, kết quả đề án cũng góp phần minh họa xu hướng chuyển đổi số trong quản trị nhân sự – một trong những chủ đề trọng tâm của quản trị nguồn nhân lực trong thời đại 4.0.

Với các lý do trên, đề tài “Ứng dụng chấm công trên di động ...” được thực hiện với kỳ vọng xây dựng một giải pháp chấm công hiện đại, dễ triển khai, phù hợp với điều kiện thực tế, đồng thời tạo nền tảng để phát triển các tính năng nâng cao trong tương lai như phân tích dữ liệu chấm công, nhắc nhở thông minh, hay tích hợp thêm các mô-đun quản lý nhân sự khác.

Danh sách các bảng

Bảng 2.1: Đặc tả Use Case chức năng Chấm công bằng FaceID.....	13
Bảng 2.2: Tóm tắt các chức năng chính của hệ thống.....	18
Bảng 3.1: Danh sách các mô hình dữ liệu (Models) trong hệ thống.....	27
Bảng 3.2: Danh sách các ViewModels và trách nhiệm xử lý.....	29
Bảng 3.3: Từ điển dữ liệu - Bảng departments (Phòng ban).....	30
Bảng 3.4: Từ điển dữ liệu - Bảng employees (Nhân viên).....	30
Bảng 3.5: Từ điển dữ liệu - Bảng attendance (Chấm công).....	31
Bảng 3.6: Từ điển dữ liệu - Bảng approvals (Đơn từ / Phê duyệt).	32
Bảng 3.7: Từ điển dữ liệu - Bảng meetings (Lịch họp).	32
Bảng 3.8: Từ điển dữ liệu - Bảng face_embeddings (Dữ liệu khuôn mặt).....	33
Bảng 3.9: Từ điển dữ liệu - Bảng chat_messages (Tin nhắn chat).....	33
Bảng 3.10: Từ điển dữ liệu - Bảng company_config (Cấu hình công ty).....	34
Bảng 4.1 Hệ thống định nghĩa 4 sự kiện Socket.IO chính.....	51
Bảng 5.1: Danh sách các Middleware trong kiến trúc Backend.	56
Bảng 5.2: Các biện pháp bảo mật hệ thống.....	62
Bảng 7.1: Chi tiết môi trường phân cứng kiểm thử hệ thống.....	91
Bảng 7.2: Tổng hợp các công nghệ và môi trường phần mềm sử dụng.....	91
Bảng 7.3: Ma trận kiểm thử chức năng (Functional Testing).	93
Bảng 7.4: Kết quả kiểm thử hiệu năng FaceID theo điều kiện ánh sáng.	94
Bảng 7.5: Kết quả kiểm thử tính năng chống giả mạo khuôn mặt (Anti-Spoofing).....	95
Bảng 7.6: Kết quả đánh giá thời gian xử lý thuật toán AI (Inference Time).....	95
Bảng 7.7: Kết quả kiểm thử độ trễ phản hồi của các REST API.	96
Bảng 7.8: Kết quả kiểm thử độ trễ giao tiếp thời gian thực với Socket.IO.....	97
Bảng 7.9: Kết quả kiểm thử mức độ tiêu thụ bộ nhớ (Memory Usage).....	97
Bảng 7.10: Kết quả đánh giá giao diện người dùng theo tiêu chí Nielsen Heuristics.	97

Danh sách các hình

Hình 2.1: Sơ đồ Use Case tổng quát hệ thống WorkMate.....	8
Hình 2.2a: Sơ đồ trạng thái – Giai đoạn khởi tạo và thu thập khuôn mặt chính diện.....	10
Hình 2.2b: Sơ đồ trạng thái – Quy trình thu thập ảnh đa hướng (Trái, Phải, Trên, Dưới).....	11
Hình 2.2c: Sơ đồ trạng thái – Giai đoạn đồng bộ dữ liệu lên Server và hoàn tất	12
Hình 2.3a: Use Case quy trình đăng ký dữ liệu khuôn mặt mẫu.....	15
Hình 2.3b: Use Case luồng xử lý trích xuất và đối sánh sinh trắc học.	16
Hình 2.3c: Use Case các ràng buộc kiểm tra kỹ thuật và chống giả mạo.	16
Hình 3.1a: Kiến trúc phân tầng và các thành phần xử lý AI trên thiết bị di động.	21
Hình 3.1b: Kiến trúc hạ tầng Backend, Cơ sở dữ liệu và các dịch vụ đám mây.....	22
Hình 3.1c: Sơ đồ tích hợp hệ thống tổng thể và quy trình triển khai tự động.....	22
Hình 3.2a: Cấu trúc thư mục gốc và các gói thành phần hỗ trợ.....	24
Hình 3.2b: Cấu trúc gói Data (Xử lý dữ liệu và Repository).	24
Hình 3.2c: Cấu trúc gói Presentation (Giao diện và ViewModel).	25
Hình 3.3a: Sơ đồ ERD – Quản lý cấu trúc tổ chức và thông tin nhân sự.....	35
Hình 3.3b: Sơ đồ ERD – Quản lý dữ liệu chấm công và định danh sinh trắc học.....	36
Hình 3.3c: Sơ đồ ERD – Quản lý quy trình phê duyệt và tương tác nội bộ.....	37
Hình 4.1a: Sơ đồ hoạt động – Quy trình kiểm tra vị trí và điều kiện kết nối mạng.....	39
Hình 4.1b: Sơ đồ hoạt động – Quy trình kiểm tra chất lượng hình ảnh và nhận diện người thật (Liveness).	40
Hình 4.1c: Sơ đồ hoạt động – Quy trình đối sánh AI và ghi nhận dữ liệu chấm công.	41
Hình 5.1a: Sơ đồ triển khai hạ tầng tại các thiết bị đầu cuối (Client-side).	54
Hình 5.1b: Sơ đồ triển khai hệ thống máy chủ và cơ sở dữ liệu.	55
Hình 5.1c: Sơ đồ luồng kết nối mạng và tích hợp các dịch vụ đám mây.....	55
Hình 5.2a: Sơ đồ tuần tự – Quy trình xác thực vị trí và điểm truy cập mạng.....	59
Hình 5.2b: Sơ đồ tuần tự – Quy trình nhận diện khuôn mặt và trích xuất đặc trưng sinh trắc học.....	60
Hình 5.2c: Sơ đồ tuần tự – Quy trình gọi API đối sánh và lưu bản ghi chấm công.....	61
Hình 5.3a: Quy trình đẩy mã nguồn và kích hoạt Webhook tự động.....	63
Hình 5.3b: Quy trình thiết lập môi trường và thực thi kịch bản biên dịch app iOS.....	64
Hình 5.3c: Quy trình đóng gói sản phẩm và phân phối tới nhóm kiểm thử.....	65
Hình 6.1: Màn hình Splash & Onboarding.....	67
Hình 6.2: Màn hình Đăng nhập & Quên mật khẩu	68

Hình 6.3: Màn hình chính (Dashboard)..... 69

Hình 6.4: Quy trình kiểm tra vị trí & WiFi 70

Hình 6.5: Giao diện quét khuôn mặt FaceID..... 71

Hình 6.6: Thông báo Chấm công thành công..... 72

Hình 6.7: Đăng ký nghỉ phép (Leave Request)..... 73

Hình 6.8: Lịch sử và Thống kê (Statistics)..... 74

Hình 6.9: Giao diện quản lý tài khoản ngân hàng 75

Hình 6.10: Giao diện theo dõi thâm niên và điểm công hiện..... 76

Hình 6.11: Giao diện chi tiết ngày công..... 77

Hình 6.12: Giao diện tạo đơn làm thêm giờ (OT Request) 78

Hình 6.13: Giao diện tạo đơn bổ sung công (Supplement Request) 79

Hình 6.14: Giao diện danh sách lịch họp (Meeting list) 80

Hình 6.15: Chat với Trợ lý ảo AI..... 81

Hình 6.16: Chat với Quản trị viên (Admin Chat)..... 82

Hình 6.17: Chat với Đồng nghiệp và Nhóm phòng ban..... 83

Hình 6.18: Dashboard tổng quan của Admin 84

Hình 6.19: Quản lý danh sách Nhân viên..... 84

Hình 6.20: Phê duyệt đơn từ (Approval Center) 85

Hình 6.21: Giao diện xem lịch sử chấm công chi tiết của nhân viên..... 85

Hình 6.22: Giao diện cấu hình ca làm việc (Shift Settings)..... 86

Hình 6.23: Giao diện web admin trong chế độ Dark Mode 87

Hình 6.24: Giao diện app trong chế độ Dark Mode 88

Hình 6.25: Minh họa thông báo đẩy (Push Notification) trên thiết bị di động 89

Hình 6.26: Giao diện thông báo thời gian thực (Real-time Popup) trên Web Admin..... 90

Danh mục các từ viết tắt

Viết tắt	Tiếng Anh đầy đủ	Ý nghĩa Tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
BSSID	Basic Service Set Identifier	Mã định danh duy nhất của điểm truy cập WiFi
CI/CD	Continuous Integration / Deployment	Tích hợp và Triển khai liên tục
CSDL	Database	Cơ sở dữ liệu
ERD	Entity Relationship Diagram	Sơ đồ mối quan hệ thực thể
FCM	Firebase Cloud Messaging	Dịch vụ gửi thông báo đẩy của Google
GPS	Global Positioning System	Hệ thống định vị toàn cầu
HTTP/S	Hypertext Transfer Protocol (Secure)	Giao thức truyền tải siêu văn bản (bảo mật)
OT	Overtime	Làm thêm giờ
REST	Representational State Transfer	Một kiểu kiến trúc thiết kế API phổ biến
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
SSID	Service Set Identifier	Tên mạng WiFi
UI/UX	User Interface / User Experience	Giao diện và Trải nghiệm người dùng

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ THỊ TRƯỜNG	1
1.1 Tính cấp thiết và bối cảnh xã hội.....	1
1.2 Phân tích các giải pháp hiện hữu (SWOT Analysis)	1
1.2.1 Máy chấm công vân tay.....	1
1.2.2 Ứng dụng chấm công GPS thông thường	2
1.2.3 Các hệ thống chấm công Face + Geofencing hiện đại.....	3
1.3 Mục tiêu và nhiệm vụ nghiên cứu.....	3
1.3.1 Nghiên cứu mô hình CNN tối ưu cho thiết bị di động	3
1.3.2 Xây dựng quy trình xác thực đa lớp: GPS Geofencing, WiFi Fingerprinting, Face Recognition	4
1.3.3 Triển khai kiến trúc micro-backend hỗ trợ giao tiếp thời gian thực (Real-time)	5
1.3.4 Trợ lý ảo thông minh dựa trên ngữ cảnh và tập luật (Context-Aware Rule-Based Assistant) làm trợ lý ảo hỗ trợ thông tin nhân sự tự động	5
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG CHI TIẾT	7
2.1 Yêu cầu Chức năng (Functional Requirements).....	8
2.1.1 Phân hệ dành cho Nhân viên (Employee App).....	9
2.1.2 Phân hệ dành cho Quản trị viên (Web Admin Dashboard)	17
2.2 Yêu cầu Phi chức năng (Non-functional Requirements)	18
2.2.1 Tính bảo mật (Security & Privacy by Design)	18
2.2.2 Tính khả dụng (Usability & UX)	19
2.2.3 Tính mở rộng (Scalability & Performance).....	20
2.2.4 Các yêu cầu chất lượng khác.....	20
CHƯƠNG 3: THIẾT KẾ KIẾN TRÚC VÀ CƠ SỞ DỮ LIỆU CHUYÊN SÂU	21

3.1 Tổng quan về Clean Architecture	21
3.2 Chi tiết từng tầng kiến trúc	26
3.2.1 Tầng Core (Lõi hệ thống)	26
3.2.2 Tầng Data (Dữ liệu)	27
3.2.3 Tầng Services (Dịch vụ chuyên biệt)	28
3.2.4 Tầng Presentation (Giao diện)	29
3.3 Thiết kế Cơ sở dữ liệu (PostgreSQL)	30
3.3.1 Lý do chọn PostgreSQL	30
3.3.2 Từ điển dữ liệu chi tiết (Data Dictionary)	30
3.3.3 Mối quan hệ giữa các bảng	34
CHƯƠNG 4: GIẢI PHÁP KỸ THUẬT VÀ THUẬT TOÁN AI CHI TIẾT ...	39
4.1 Thuật toán xác thực vị trí địa lý (Geofencing – Haversine)	42
4.1.1 Bài toán và thách thức	42
4.1.2 Công thức Haversine chi tiết	42
4.1.3 Triển khai trong Backend	42
4.1.4 Luồng xử lý tại Server	43
4.1.5 Kiểm tra WiFi (Bước song song với GPS)	43
4.2 Pipeline Nhận diện Khuôn mặt (FaceID – On-Device AI)	44
4.2.1 Kiến trúc tổng thể	44
4.2.2 Bước 1 – Khởi tạo Model (Initialization)	44
4.2.3 Bước 2 – Phát hiện khuôn mặt và Liveness Detection	45
4.2.3 Bước 3 – Cắt và Tiền xử lý ảnh (Preprocessing)	46
4.2.4 Bước 4 – Chuẩn hóa pixel (Normalization)	46
4.2.6 Bước 5 – Trích xuất Vector đặc trưng (Feature Extraction)	47
4.2.7 Bước 6 – So khớp khuôn mặt (Face Matching)	48
4.3 Hệ thống trợ lý ảo WorkMate Assistant (Hybrid Rule-based)	49

4.3.1 Kiến trúc Context-Aware Chatbot.....	49
4.3.2 Luồng xử lý chi tiết (File: hybrid_chatbot.js)	50
4.4 Hệ thống Chat Realtime (Socket.IO)	51
4.4.1 Sơ đồ sự kiện.....	51
4.4.2 Cơ chế Routing tin nhắn:.....	52
4.4.3 Thuật toán tính điểm thâm niên và cống hiến	52
4.5 Thuật toán tính điểm thâm niên và cống hiến (Seniority Points)	52
4.5.1 Cơ chế và Dữ liệu nền tảng	52
4.5.2 Quy trình vận hành hệ thống.....	52
CHƯƠNG 5: TRIỂN KHAI PHẦN MỀM VÀ DEVOPS CHI TIẾT	54
5.1 Tổng thể kiến trúc Backend (Node.js + Express)	54
5.1.1 Cấu trúc server khởi động	56
5.1.2 Danh sách đầy đủ các Middleware	56
5.1.3: Cơ chế tự động khởi tạo và cập nhật CSDL (Auto-migration)	56
5.2 Đặc tả REST API đầy đủ (API Documentation)	57
5.2.1 Nhóm API Xác thực (Auth)	57
5.2.2 Nhóm API Chấm công FaceID.....	57
5.2.3 Nhóm API Đơn từ (Approvals)	61
5.2.4 Nhóm API Chat & AI.....	62
5.3 Bảo mật hệ thống	62
5.3.1: Bảo mật truy vấn và phòng chống SQL Injection	63
5.4 Quy trình CI/CD với Codemagic.....	63
5.4.1 Cấu hình build tự động.....	65
5.4.2 Các bước thực thi trên Codemagic	65
5.4.3 Lý do chọn Codemagic.....	65
CHƯƠNG 6: MINH HỌA GIAO DIỆN HỆ THỐNG.....	67

6.1 Giao diện dành cho Nhân viên (Mobile App)	67
6.1.1 Nhóm màn hình Khởi đầu & Đăng nhập	67
6.1.2 Nhóm màn hình Chấm công (Tính năng cốt lõi)	69
6.1.3 Nhóm màn hình Quản lý hành chính	73
6.1.4 Nhóm màn hình Tương tác & AI	81
6.2 Giao diện dành cho Quản trị viên (Web Admin)	84
6.3 Giao diện Chế độ tối (Dark Mode - Bonus)	87
6.4 TÍNH NĂNG ĐẶC BIỆT (Technical Highlights)	89
CHƯƠNG 7: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ CHI TIẾT	91
7.1 Môi trường thực nghiệm	91
7.1.1 Môi trường phần cứng	91
7.1.2 Môi trường phần mềm	91
7.2 Kiểm thử Chức năng (Functional Testing)	93
7.2.1 Ma trận kiểm thử Use Case	93
7.3 Kiểm thử và Đánh giá Hiệu năng AI (FaceID)	94
7.3.1 Phương pháp đánh giá	94
7.3.2 Kết quả kiểm thử theo điều kiện ánh sáng	94
7.3.3 Kiểm thử chống giả mạo (Anti-Spoofing)	95
7.3.4 Đánh giá Thời gian xử lý (Inference Time)	95
7.4 Kiểm thử Hiệu năng Hệ thống (Performance Testing)	96
7.4.1 Độ trễ API	96
7.4.2 Kiểm thử Realtime (Socket.IO)	97
7.4.3 Kiểm thử Bộ nhớ (Memory Usage)	97
7.5 Đánh giá Giao diện Người dùng (UI/UX Evaluation)	97
7.5.1 Tiêu chí đánh giá theo Nielsen Heuristics	97
7.6 Đánh giá tổng hợp về hiệu quả của trợ lý ảo WorkMate	98

CHƯƠNG 8: KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN	99
8.1 Kết luận tổng quan.....	99
8.1.1 Các mục tiêu đã đạt được.....	99
8.1.2 Những điểm sáng tạo nổi bật.....	99
8.1.3 Hạn chế và thách thức	100
8.2 Hướng phát triển tương lai	100
8.2.1 Nâng cấp hệ thống nhận diện khuôn mặt (FaceID 2.0)	100
8.2.2 Mô-đun Quản lý Lương tự động (Payroll Module)	101
8.2.3 Hệ thống phân tích và dự báo thông minh (AI Analytics)	101
8.2.4 Mở rộng sang nền tảng Web Progressive App (PWA)	101
8.2.5 Tích hợp hệ sinh thái doanh nghiệp (ERP Integration).....	101
8.3 Tổng kết.....	102
<i>TÀI LIỆU THAM KHẢO</i>	103

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ THỊ TRƯỜNG

1.1 Tính cấp thiết và bối cảnh xã hội

Cách mạng Công nghiệp 4.0 và làn sóng số hóa đã làm thay đổi căn bản cách doanh nghiệp quản lý nguồn nhân lực. Sau đại dịch COVID-19, các hình thức làm việc linh hoạt, làm từ xa và hybrid trở nên phổ biến, buộc doanh nghiệp phải chuyển từ mô hình quản lý “tập trung – chấm công tại cổng” sang mô hình quản trị dựa trên dữ liệu thời gian thực và không phụ thuộc hoàn toàn vào không gian vật lý .

Nghiên cứu về HR 4.0 cho thấy các xu hướng chính gồm: số hóa quy trình nhân sự, tăng tự động hóa, dùng IoT, AI, phân tích dữ liệu lớn để giám sát, đánh giá, trả lương, và đảm bảo kỷ luật lao động trong bối cảnh linh hoạt . Trong bối cảnh đó, chấm công không chỉ là ghi nhận giờ vào/ra, mà còn là đầu vào quan trọng cho hệ thống lương thưởng, đánh giá hiệu suất, quản lý lực lượng lao động phân tán (nhân viên đi thị trường, công trình, patrol, làm việc từ xa).

Các nghiên cứu nhân mạnh chuyển đổi từ phương pháp thủ công, máy vân tay, thẻ từ... sang hệ thống chấm công số, tích hợp sinh trắc học + geolocation + phân tích dữ liệu giúp tăng độ chính xác, giảm gian lận “chấm hộ”, giảm gánh nặng hành chính và nâng cao trách nhiệm của người lao động. Điều này phù hợp với bối cảnh hậu COVID-19, nơi doanh nghiệp vừa phải đảm bảo kỷ luật, minh bạch, vừa tạo điều kiện linh hoạt cho nhân sự .

1.2 Phân tích các giải pháp hiện hữu (SWOT Analysis)

1.2.1 Máy chấm công vân tay

Nhiều tổ chức, trường học, nhà máy vẫn dùng máy chấm vân tay / RFID / ký tay do chi phí ban đầu thấp, dễ triển khai . Tuy nhiên, các hệ thống này bộc lộ nhiều nhược điểm:

Tiếp xúc vật lý: nguy cơ lây nhiễm, đặc biệt trong bối cảnh dịch bệnh; các nghiên cứu về hệ thống chấm công số nhân mạnh nhu cầu chuyển sang giải pháp không tiếp xúc như nhận diện khuôn mặt, app di động .

Hạn chế về không gian: chỉ hoạt động tại một điểm cố định; không phù hợp cho nhân viên làm việc ngoài văn phòng, đi công trình, patrol, làm việc hybrid.

Bảo trì và độ bền: đầu đọc vân tay dễ hỏng, giảm chính xác khi bề mặt bẩn, trầy xước; việc này gây gián đoạn và tốn chi phí vận hành .

Nguy cơ gian lận: một số hình thức gian lận (dùng thẻ, mã hộ) vẫn tồn tại trong các hệ thống không có xác thực sinh trắc học đủ mạnh .

=> Nhìn chung, máy chấm công vân tay phù hợp bối cảnh nhân sự làm việc tập trung, ít linh hoạt, nhưng thiếu đáp ứng cho mô hình hybrid/di động hiện nay.

1.2.2 Ứng dụng chấm công GPS thông thường

Nhiều nghiên cứu và triển khai thực tế đã dùng app chấm công dựa trên GPS/geolocation để tăng tính linh hoạt, cho phép nhân viên check-in từ nhiều địa điểm, kể cả ngoài văn phòng .

Ưu điểm:

Người lao động có thể dùng smartphone để chấm công tại hiện trường, nhà, điểm khách hàng...; dữ liệu vị trí và thời gian được ghi lại tự động, hỗ trợ tốt cho môi trường hybrid .

Hệ thống GIS/GPS cho phép thiết lập vùng địa lý (geofence) quanh văn phòng, công trình, khu patrol, giúp tự động hóa quá trình xác thực vị trí.

Nhược điểm :

Một số nghiên cứu chỉ ra GPS có thể bị giả mạo (spoof/fake GPS) nếu không có cơ chế phát hiện, dẫn đến chấm công từ xa mà vẫn được tính hợp lệ .

GPS đơn thuần không định danh được người cầm máy – chỉ biết vị trí thiết bị, không biết có đúng nhân viên đó hay không, nên vẫn tồn tại rủi ro mượn điện thoại chấm hộ.

Độ chính xác GPS giảm trong môi trường đô thị dày đặc, trong nhà, hoặc khi tín hiệu kém, tạo khoảng trống cần bù đắp bằng các kỹ thuật khác như WiFi/Bluetooth, tối ưu thuật toán geofencing.

=> Các hệ thống chỉ dùng GPS là chưa đủ an toàn, cần kết hợp thêm sinh trắc học (Face Recognition) và cơ chế chống giả mạo vị trí.

1.2.3 Các hệ thống chấm công Face + Geofencing hiện đại

Nhiều công trình gần đây đề xuất kết hợp nhận diện khuôn mặt và geofencing để chống gian lận:

Hệ thống dùng face recognition + geofence cho sinh viên/nhân viên, chỉ đánh dấu hiện diện khi vừa nhận diện đúng khuôn mặt đã đăng ký, vừa nằm trong vùng địa lý lớp học/công ty, đạt độ chính xác 92–99% và giảm rõ rệt tình trạng “điểm danh hộ”.

Một số nghiên cứu thêm liveness detection / anti-spoofing (dùng MobileNetV2, mô hình nhẹ) để chống trường hợp dùng ảnh/video để lừa hệ thống .

Hệ thống face + GPS trong bối cảnh hybrid cho thấy độ chính xác nhận diện cao (~95–97%), thời gian xử lý 1–2 giây và GPS geofence đạt >90% ở môi trường mở, giúp đảm bảo cả định danh và hiện diện thực tế.

Những kết quả này củng cố lựa chọn giải pháp của đề tài: app chấm công đa lớp Face + Geofencing + các lớp xác thực vị trí khác.

1.3 Mục tiêu và nhiệm vụ nghiên cứu

Dựa trên bối cảnh và hạn chế của giải pháp hiện hữu, đề tài hướng tới xây dựng một hệ thống chấm công thông minh “WorkMate” với các mục tiêu/nhiệm vụ sau:

1.3.1 Nghiên cứu mô hình CNN tối ưu cho thiết bị di động

Các hệ thống hiện đại thường dùng CNN nhẹ (MobileNetV2, các biến thể tối ưu) để chạy trên thiết bị di động, vẫn bảo đảm độ chính xác cao trong nhận diện khuôn mặt và liveness detection . Nhiệm vụ của đề tài:

Khảo sát các kiến trúc CNN nhẹ dùng cho nhận diện khuôn mặt on-device và chống giả mạo (MobileNetV2, MobileFaceNet, ResNet rút gọn, LBPH kết hợp AI, v.v.) .

Lựa chọn, tinh chỉnh mô hình phù hợp với tài nguyên smartphone (RAM, CPU/GPU di động), cân bằng giữa độ chính xác, tốc độ suy luận và dung lượng model.

Thiết kế quy trình huấn luyện/inference: đăng ký khuôn mặt, trích xuất embedding, so khớp theo ngưỡng, xử lý trường hợp ánh sáng yếu/ngoài trời .

1.3.2 Xây dựng quy trình xác thực đa lớp: GPS Geofencing, WiFi Fingerprinting, Face Recognition

Nhiều nghiên cứu nhân mạnh xác thực đa yếu tố / đa lớp là cách hữu hiệu để giảm rủi ro giả mạo vị trí và danh tính. Đề tài đặt mục tiêu:

Lớp 1 – Geofencing GPS:

- Thiết lập vùng địa lý được phép chấm công (văn phòng, công trình, khu patrol).
- Chỉ cho phép gửi yêu cầu chấm công khi thiết bị nằm trong polygon geofence, hoặc ở khoảng cách $<$ ngưỡng từ tâm vùng.
- Kết hợp cơ chế phát hiện vị trí giả (mock/fake GPS) tương tự các hệ thống đã triển khai.

Lớp 2 – WiFi Fingerprinting (tùy bối cảnh):

- Với môi trường indoor nhiều tầng, dùng thông tin WiFi SSID/BSSID/độ mạnh sóng làm “dấu vân tay vị trí” để nâng cao độ tin cậy khi GPS kém chính xác.
- WiFi fingerprinting có thể bổ sung thêm một lớp xác thực vị trí trong geofence, hỗ trợ môi trường đô thị dày đặc nơi GPS bị nhiễu .

Lớp 3 – Face Recognition:

- Yêu cầu người dùng chụp khuôn mặt (hoặc video ngắn) tại thời điểm chấm công, so với mẫu đã đăng ký.
- Kết hợp liveness detection / anti-spoofing để ngăn việc dùng ảnh/video người khác .
- Chỉ ghi nhận chấm công khi đồng thời thỏa cả 3 lớp: đúng vị trí (GPS/WiFi), đúng khuôn mặt, thời gian hợp lệ.
- Quy trình này tương đồng với các hệ thống đã chứng minh hiệu quả trong môi trường sinh viên, nhân viên, lực lượng tuần tra... giúp nâng cao độ tin cậy và khả năng mở rộng.

1.3.3 Triển khai kiến trúc micro-backend hỗ trợ giao tiếp thời gian thực (Real-time)

Các hệ thống chăm công hiện đại có xu hướng dùng kiến trúc dịch vụ nhỏ, realtime (cloud, Firebase, WebSocket) để đảm bảo dữ liệu vào/ra tức thời, dễ mở rộng cho nhiều tổ chức và địa điểm. Nhiệm vụ của đề tài:

Thiết kế micro-backend chia nhỏ các chức năng:

- Dịch vụ xác thực & quản lý người dùng.
- Dịch vụ chăm công (lưu log thời gian, vị trí, kết quả face).
- Dịch vụ báo cáo & phân tích (tổng hợp công, trễ/sớm, vắng mặt).

Sử dụng cơ chế đồng bộ thời gian thực (websocket, push, chịu ảnh hưởng từ các hệ thống dùng Firebase/Google Cloud trong nghiên cứu WorkIQ và các app chăm công) để:

- Cập nhật trạng thái nhân viên theo thời gian thực cho phía quản lý.
- Hạn chế trễ dữ liệu khi tính công, hỗ trợ giám sát ca làm, patrol ngoài hiện trường.
- Đảm bảo các yếu tố an toàn dữ liệu, phân quyền theo tổ chức, có khả năng mở rộng multi-tenant như các hệ thống HR cloud hiện đại .

1.3.4 Trợ lý ảo thông minh dựa trên ngữ cảnh và tập luật (Context-Aware Rule-Based Assistant) làm trợ lý ảo hỗ trợ thông tin nhân sự tự động

Trong bối cảnh HR 4.0, các nghiên cứu cho thấy việc xây dựng Trợ lý ảo thông minh hỗ trợ thông tin nhân sự tự động vào hệ thống nhân sự giúp tự động hóa trả lời câu hỏi, giảm tải cho bộ phận HR và nâng trải nghiệm nhân viên . Nhiệm vụ của đề tài:

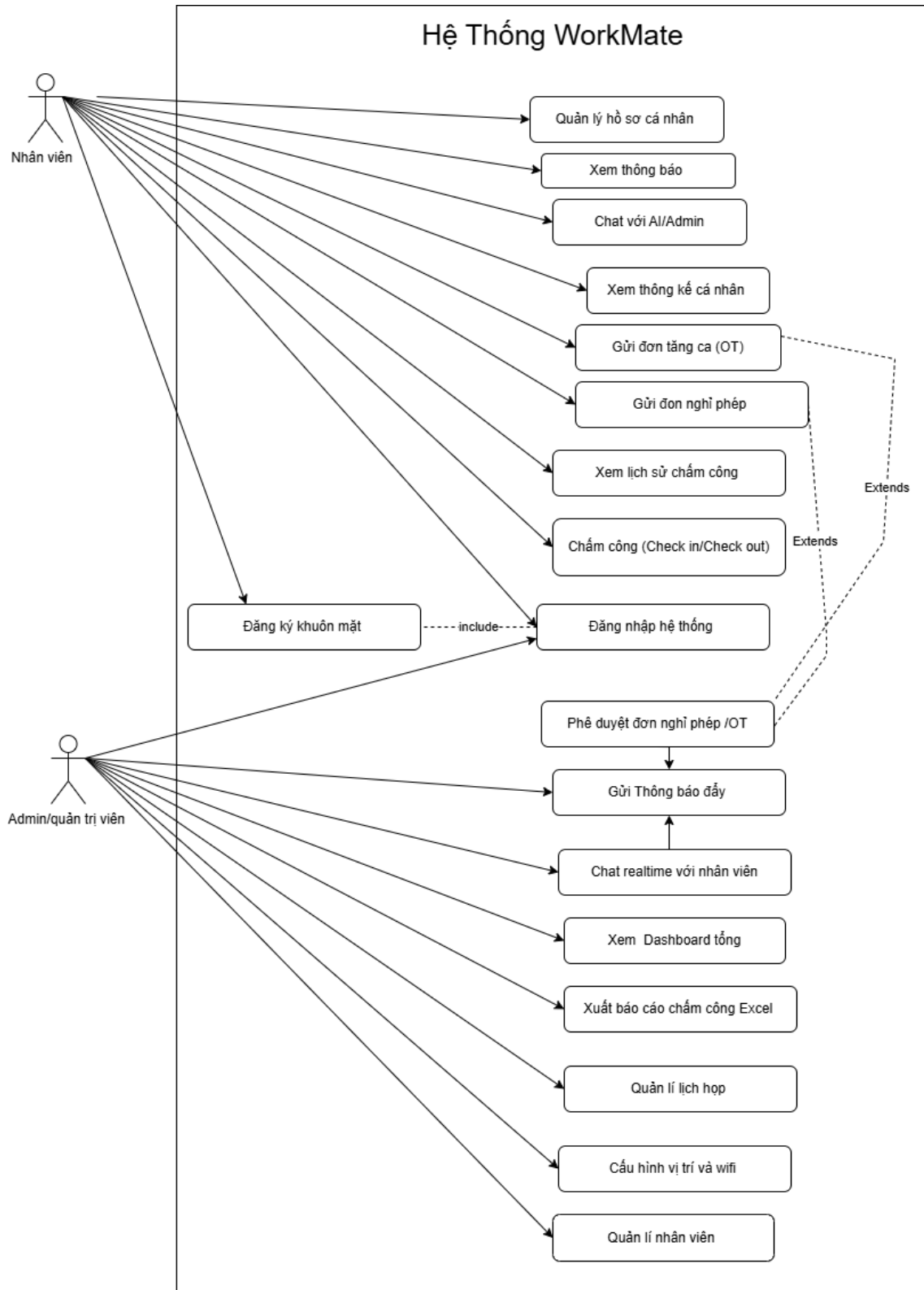
- Thiết kế trợ lý ảo tích hợp với dữ liệu chăm công, ca làm, chính sách công ty để:
- Trả lời tự động các câu hỏi như: “Tháng này em đi làm bao nhiêu ngày?”, “Hôm nay em đã check-in chưa?”, “Quy định đi trễ mấy phút thì bị trừ lương?”, v.v.

- Hỗ trợ HR giải đáp các thắc mắc lặp đi lặp lại về chấm công, phép, công tác, tránh phải xử lý tay.
- Nghiên cứu cách giới hạn phạm vi của Generative AI trong domain nhân sự để đảm bảo an toàn, tránh trả lời sai lệch về chính sách.
- Tận dụng xu hướng phát triển công cụ số HR tập trung vào trải nghiệm nhân viên, gắn kết, hỗ trợ tự phục vụ (self-service) như các concept app HR hiện đại.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG CHI TIẾT

Yêu cầu hệ thống cho app chăm công thông minh: Tổng hợp từ nghiên cứu và đề tài của bạn . Các nghiên cứu về hệ thống chăm công thông minh đều nhấn mạnh: phải mô tả rõ phân hệ mobile + web, quy trình nhận diện khuôn mặt kèm liveness, geofencing, lưu trữ đám mây và khả năng mở rộng, rất phù hợp với đề tài của bạn.

2.1 Yêu cầu Chức năng (Functional Requirements)



Hình 2.1: Sơ đồ Use Case tổng quát hệ thống WorkMate

2.1.1 Phân hệ dành cho Nhân viên (Employee App)

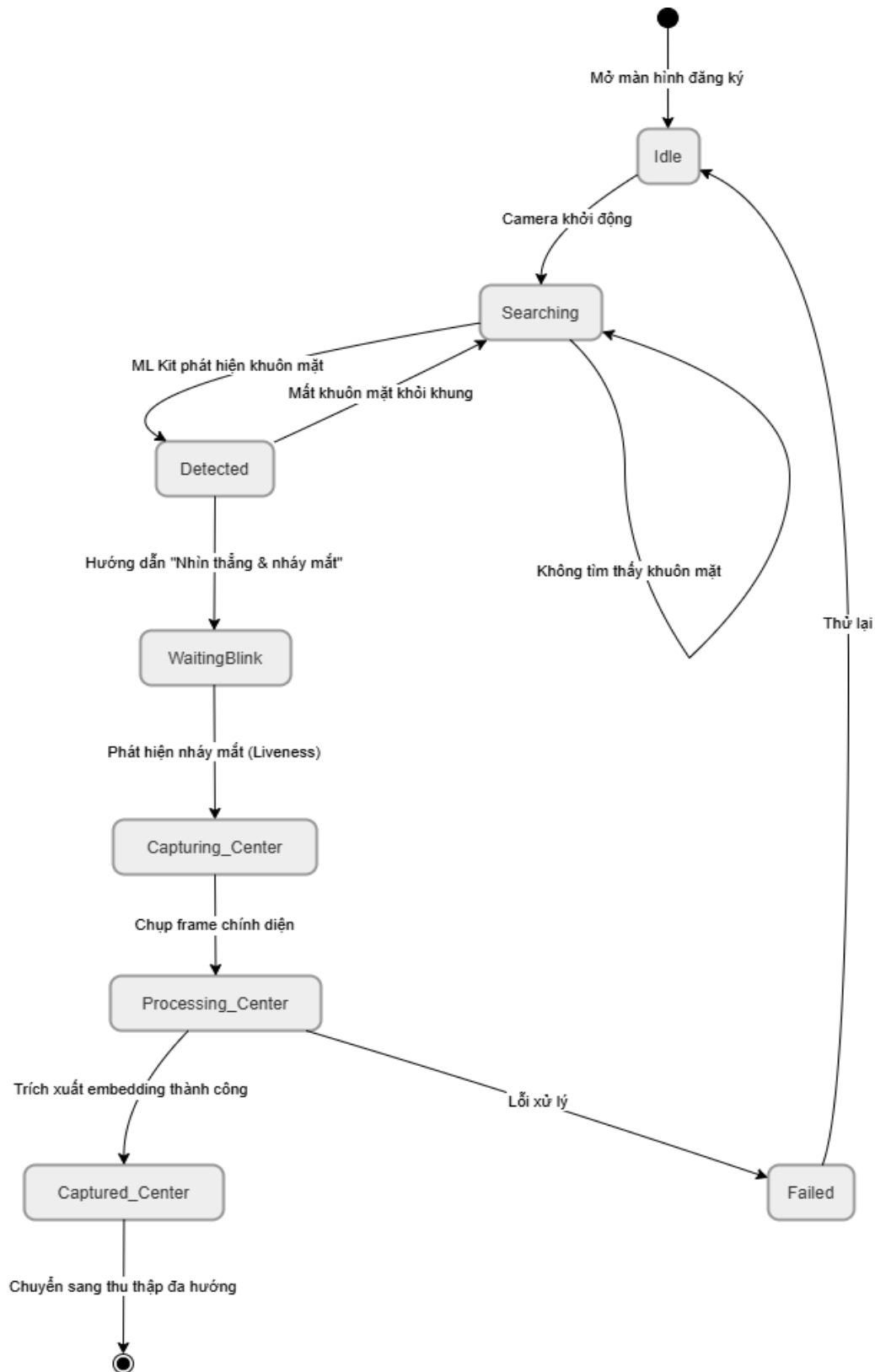
a) Đăng ký danh tính sinh trắc học

App yêu cầu nhân viên chụp ít nhất 5 góc mặt (chính diện, hơi nghiêng trái, hơi nghiêng phải, ngược lên , ngược xuống) để tạo bộ vector đặc trưng (embedding) lưu trên cloud, tương tự các hệ thống dùng FaceNet/MobileNetV2, DeepFace hoặc ResNet trên Azure/Firebase.

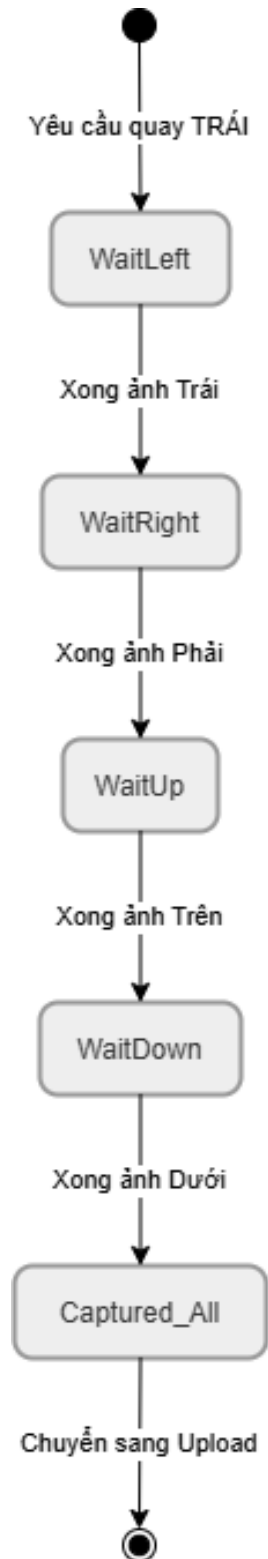
Trong bước đăng ký, tích hợp Passive Liveness Detection: mô hình MobileNetV2/nhẹ on-device phân biệt mặt thật với ảnh/video, không yêu cầu người dùng làm động tác phức tạp, giống cách các hệ thống anti-spoofing dùng MobileNetV2 để chặn fake attendance.

Quy trình:

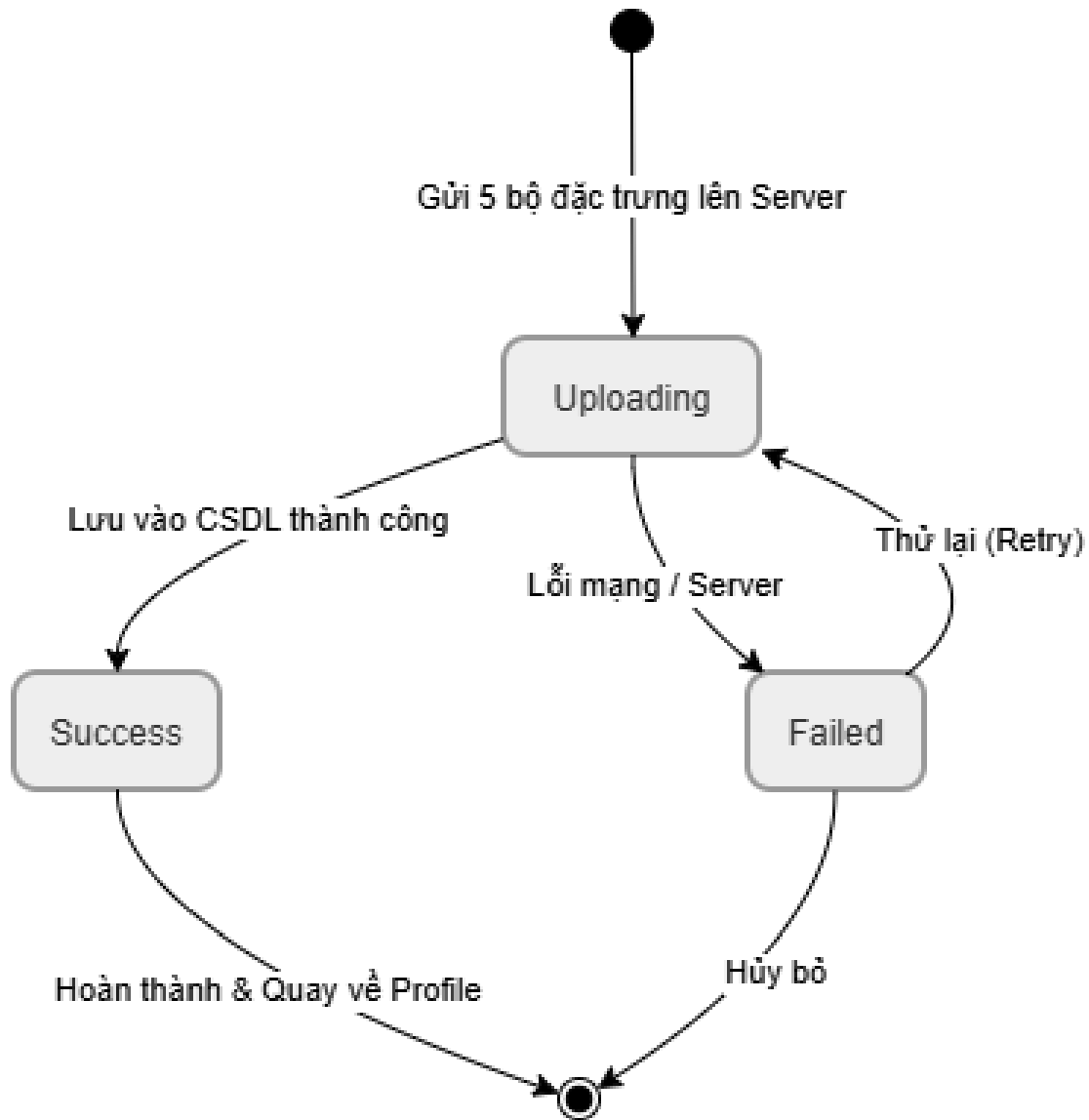
nhập thông tin cá nhân → chụp nhiều ảnh → kiểm tra liveness → trích xuất vector → mã hóa và gửi lên backend/cloud.



Hình 2.2a: Sơ đồ trạng thái – Giai đoạn khởi tạo và thu thập khuôn mặt chính diện



Hình 2.2b: Sơ đồ trạng thái – Quy trình thu thập ảnh đa hướng (Trái, Phải, Trên, Dưới)



Hình 2.2c: Sơ đồ trạng thái – Giai đoạn đồng bộ dữ liệu lên Server và hoàn tất

* Quy trình đăng ký khuôn mặt sinh trắc học được mô tả chi tiết qua sơ đồ trạng thái tại Hình 2.2, bao gồm ba giai đoạn chính: khởi tạo/lấy mẫu chính diện (Hình 2.2a), thu thập dữ liệu đa hướng (Hình 2.2b) và cuối cùng là bước đồng bộ dữ liệu với hệ thống Backend (Hình 2.2c).

b) Chấm công thông minh kích hoạt bởi Geofence

Geofencing dựa trên GPS/Geolocation API: app liên tục (ở mức tối ưu) kiểm tra vị trí; khi thiết bị đi vào vùng geofence chi nhánh thì tự kích hoạt quy trình chấm công .

Hệ thống kết hợp:

- Kiểm tra vị trí trong geofence (có thể dùng thuật toán polygon/winding number như trong hệ thống sinh viên geo-fence + face) .
- Ghi nhận thời gian check-in/out chính xác; các nghiên cứu thường log timestamp millisecond kèm toạ độ, ID khuôn mặt/nhân viên.
- Kiểm tra liveness + đối sánh khuôn mặt trước khi ghi log, tránh chấm hộ.

Bảng 2.1: Đặc tả Use Case chức năng Chấm công bằng FaceID

Mục	Nội dung
Tên Use Case	Chấm công (Check-in / Check-out)
Mã Use Case	UC-002
Tác nhân chính	Nhân viên
Mục tiêu	Nhân viên thực hiện chấm công bằng cách xác thực vị trí GPS, mạng WiFi công ty và nhận diện khuôn mặt
Điều kiện tiên quyết	Nhân viên đã đăng nhập; đã đăng ký khuôn mặt
Điều kiện kết thúc (thành công)	Bản ghi chấm công được lưu vào CSDL với timestamp, toạ độ và trạng thái success
Điều kiện kết thúc (thất bại)	Hiển thị thông báo lỗi, không tạo bản ghi chấm công

Luồng sự kiện chính:

Bước	Nhân viên	Hệ thống
1	Nhấn nút "Chấm công" trên màn hình chính	—
2	—	Kiểm tra quyền truy cập GPS và Camera
3	—	Lấy toạ độ GPS hiện tại
4	—	So sánh với toạ độ văn phòng, kiểm tra bán kính cho phép ($\leq 100m$)
5	—	Đọc tên mạng WiFi (SSID) đang kết nối

6	—	So sánh với SSID đã cấu hình trong hệ thống
7	Đưa khuôn mặt vào khung quét	Camera mở, ML Kit phát hiện khuôn mặt
8	—	TFLite MobileFaceNet trích xuất vector embedding (192 chiều)
9	—	Tính Cosine Similarity giữa vector hiện tại và các mẫu đã lưu
10	—	Nếu similarity ≥ 0.75 : Xác nhận thành công
11	—	Lưu bản ghi vào bảng attendance(PostgreSQL)
12	—	Cập nhật UI, hiển thị màn hình thành công với timestamp

Luồng thay thế:

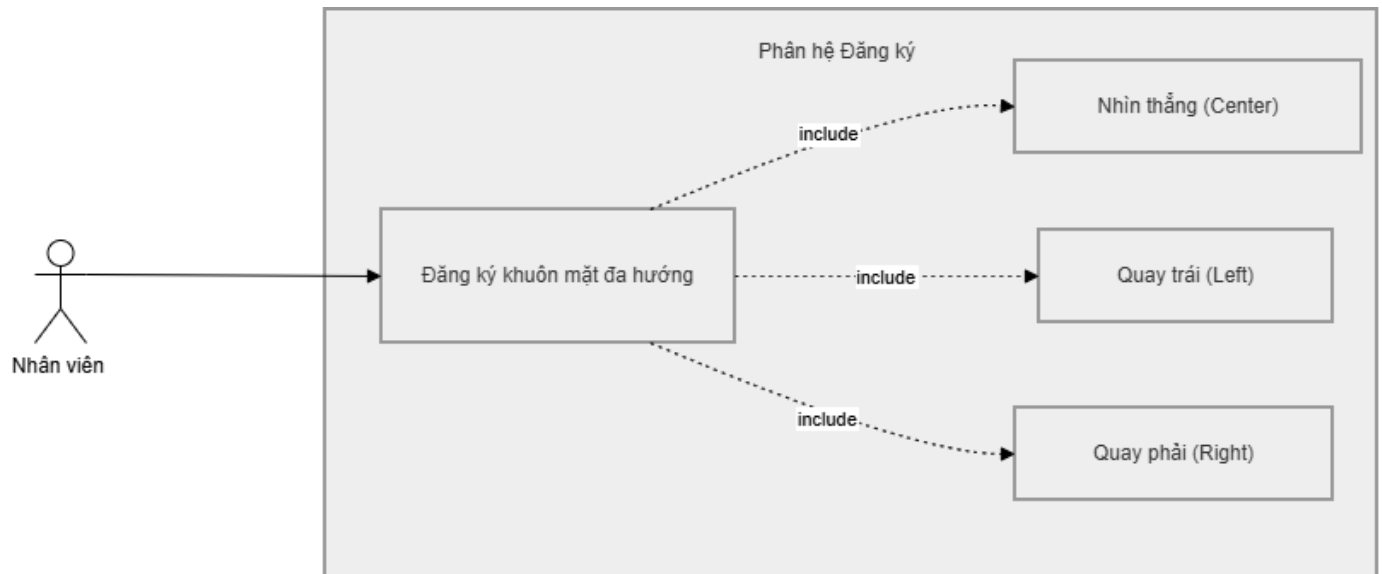
Điều kiện	Hành động hệ thống
GPS ngoài phạm vi	Hiển thị "Bạn đang ở ngoài khu vực văn phòng" và dừng
WiFi sai / không kết nối	Hiển thị "Vui lòng kết nối WiFi công ty" và dừng
Độ tương đồng khuôn mặt < 0.75	Hiển thị "Không nhận ra khuôn mặt, thử lại" (tối đa 3 lần)
Thiếu sáng	Hiển thị "Môi trường thiếu ánh sáng, vui lòng ra chỗ sáng hơn"
Khuôn mặt quá gần / xa	Hiển thị hướng dẫn điều chỉnh khoảng cách

c) Quản lý hành chính cá nhân

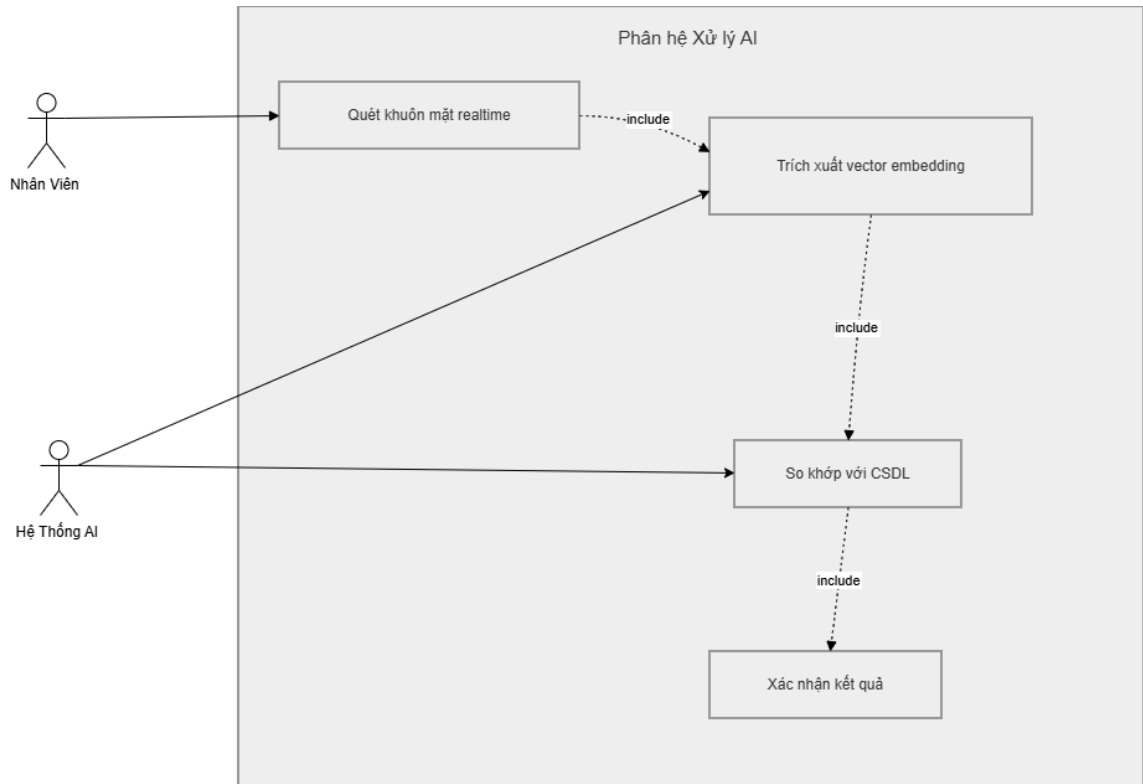
Giao diện Calendar View cho phép xem lịch sử chấm công theo ngày/tuần/tháng, tương tự các hệ thống cloud lưu log thời gian và cho phép xuất báo cáo .

Chức năng tạo và gửi đơn nghỉ phép / đi trễ / làm thêm:

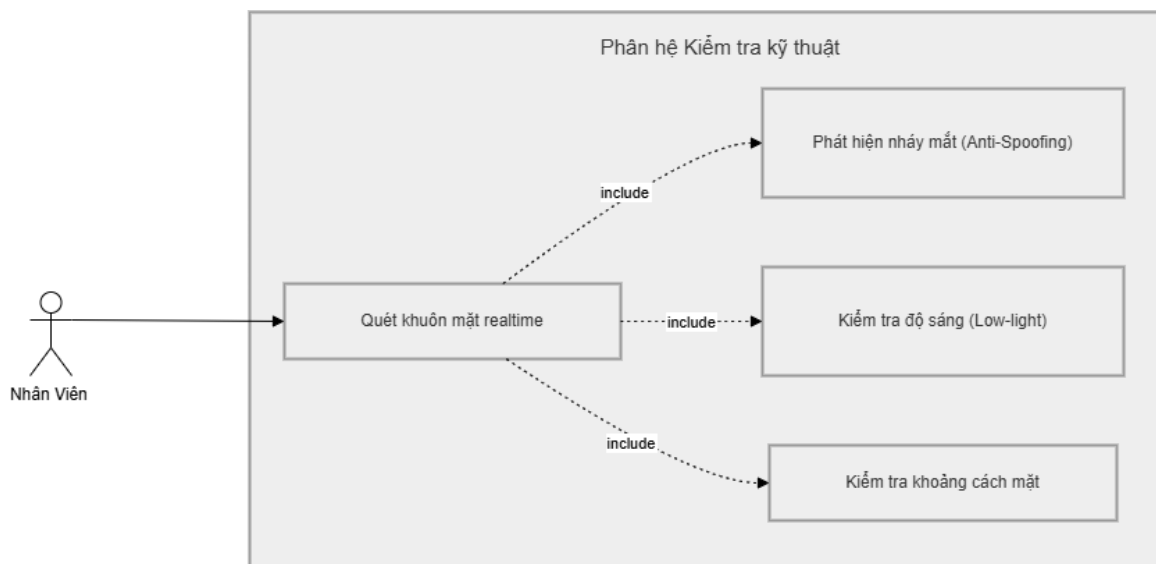
- Chọn ngày, ca, loại đơn, lý do.
- Đính kèm ảnh minh chứng (giấy khám bệnh, công tác lệnh...), tải lên cloud storage; nhiều hệ thống chấm công dùng Firebase/Azure để lưu file và metadata .
- Người dùng theo dõi được trạng thái đơn (đang chờ duyệt, đã duyệt, từ chối).



Hình 2.3a: Use Case quy trình đăng ký dữ liệu khuôn mặt mẫu.



Hình 2.3b: Use Case luồng xử lý trích xuất và đối sánh sinh trắc học.



Hình 2.3c: Use Case các ràng buộc kiểm tra kỹ thuật và chống giả mạo.

* Phân hệ nhân diện khuôn mặt FaceID được thiết kế với các tương tác chuyên sâu nhằm đảm bảo tính chính xác và bảo mật. Chi tiết các Use Case được chia thành ba nhóm chính: Quy trình đăng ký mẫu (Hình 2.3a), Luồng xử lý đối sánh của hệ thống AI (Hình 2.3b) và các cơ chế kiểm tra kỹ thuật tự động (Hình 2.3c).

d) Trợ lý ảo AI (Generative AI HR Assistant)

Chatbot tích hợp trong app, tham khảo các xu hướng HR 4.0, công cụ HR số và trợ lý AI hỗ trợ tự phục vụ nhân viên.

Các kịch bản:

- Tra cứu nhanh chính sách công ty: quy định đi trễ, tăng ca, nghỉ phép, làm từ xa, v.v..
- Tính lương dự kiến dựa trên dữ liệu chấm công, phép, tăng ca được lưu trên cloud .
- Nhắc lịch họp, nhắc check-in/check-out, cảnh báo nguy cơ quên chấm công dựa trên hành vi lịch sử (mức cao cấp có thể dùng phân tích dữ liệu).

2.1.2 Phân hệ dành cho Quản trị viên (Web Admin Dashboard)

a) Dashboard tổng quan realtime

Màn hình chính hiển thị:

- Tổng số nhân viên, số đã check-in, đang làm việc, đi muộn, nghỉ phép trong ngày.
- Biểu đồ cột/đường/thời gian thực cập nhật từ database đám mây, giống các hệ thống sử dụng Firebase Realtime DB / cloud database để sync attendance.
- Cho phép lọc theo chi nhánh, phòng ban, ca làm, ngày.

b) Quản lý địa điểm (Geofencing)

Giao diện bản đồ số (Google Maps/OpenStreetMap):

- Admin vẽ geofence cho từng chi nhánh/công trình (hình tròn với toạ độ + bán kính, hoặc polygon), tương tự các hệ thống kết hợp geofencing cho sinh viên/nhân viên.

- Cấu hình khung giờ cho phép chấm công theo ca (ví dụ 8:00–8:30 check-in).
- Lưu thông tin geofence lên cloud; các request chấm công từ mobile sẽ được backend kiểm tra trong vùng này trước khi chấp nhận .

c) Phê duyệt đơn từ (nghỉ phép, công tác, làm thêm)

- Thiết lập workflow 1 cấp hoặc đa cấp (trưởng nhóm → trưởng phòng → HR...), phù hợp định hướng quản trị số HR 4.0.
- Khi có đơn mới, hệ thống gửi thông báo đẩy đến người duyệt (qua web/notification/email).
- Admin/manager có thể xem chi tiết đơn, ảnh minh chứng, lịch sử chấm công liên quan rồi phê duyệt/từ chối; kết quả đồng bộ về app nhân viên.

d) Quản trị dữ liệu nhân sự và báo cáo

- CRUD thông tin nhân sự: thêm/sửa/xoá nhân viên, phân phòng ban, gán ca làm, gán chi nhánh (geofence).
- Quản lý mẫu sinh trắc (vector khuôn mặt), trạng thái kích hoạt tài khoản.
- Xuất báo cáo tổng hợp ra Excel/PDF: bảng công, đi trễ, tăng ca, nghỉ phép, phục vụ kế toán và HR, tương tự các hệ thống xuất báo cáo tự động từ cloud DB .

Bảng 2.2: Tóm tắt các chức năng chính của hệ thống

Nhóm chức năng	Mô tả ngắn
Đăng ký khuôn mặt + liveness	Nhiều ảnh, anti-spoof, lưu embedding
Chấm công geo-fence + face	Vào vùng → chụp mặt → log thời gian + GPS
App nhân viên	Lịch công, đơn phép, xem trạng thái
Web admin	Dashboard realtime, geofence, báo cáo

2.2 Yêu cầu Phi chức năng (Non-functional Requirements)

2.2.1 Tính bảo mật (Security & Privacy by Design)

Mã hóa đường truyền:

- Toàn bộ giao tiếp giữa ứng dụng di động, trang quản trị web, máy chủ backend và cơ sở dữ liệu đám mây được định hướng sử dụng giao thức HTTPS với chứng chỉ bảo mật SSL/TLS.

- Trong giai đoạn thực nghiệm và demo, hệ thống sử dụng giao thức HTTP qua địa chỉ IP nội bộ để thuận tiện cho việc kiểm thử và kết nối nhanh giữa các thiết bị. Tuy nhiên, kiến trúc hệ thống đã được thiết kế sẵn sàng để triển khai chứng chỉ SSL/TLS (HTTPS) khi đưa lên môi trường Production với tên miền chính thức, đảm bảo an toàn dữ liệu trên đường truyền.

Lưu trữ khuôn mặt dạng vector số:

- Thay vì lưu trữ ảnh gốc của nhân viên, hệ thống chỉ lưu trữ các đoạn mã embedding (vector đặc trưng 192 chiều) được trích xuất từ mô hình AI.
- Giải pháp này giúp giảm thiểu rủi ro lộ lọt thông tin cá nhân và tuân thủ nguyên tắc Privacy by Design trong các hệ thống nhận diện sinh trắc học hiện đại.

Phân quyền & nhật ký truy cập:

- Hệ thống phân quyền chặt chẽ: Chỉ cấp quản trị viên (Admin) hoặc nhân sự (HR) có thẩm quyền mới được phép truy cập và xuất báo cáo dữ liệu; hoàn toàn không cho phép truy cập dữ liệu vector khuôn mặt ở dạng thô.
- Mọi thao tác quan trọng như cấu hình vùng an toàn (Geofence), điều chỉnh dữ liệu công hoặc phê duyệt đơn từ đều được hệ thống ghi log (nhật ký) chi tiết để phục vụ công tác truy vết và đối soát khi cần thiết.

2.2.2 Tính khả dụng (Usability & UX)

Material Design 3, Dark Mode, đa ngôn ngữ:

Giao diện mobile/web cần tuân theo nguyên tắc UX hiện đại; các hệ thống prototype thành công đều nhấn mạnh yếu tố dễ dùng, một màn hình rõ ràng cho check-in, lịch sử, và thông báo.

- Hỗ trợ tiếng Việt – tiếng Anh, chuyển đổi nhanh trong Setting.
- Dark Mode giúp sử dụng ban đêm, giảm mỏi mắt.

Thao tác tối giản:

Quy trình chấm công cần dưới vài bước: mở app → tự kích hoạt khi vào geofence → chụp mặt → kết quả; nhiều nghiên cứu nhấn mạnh tầm quan trọng của thời gian xử lý nhanh và ít thao tác để người dùng chấp nhận hệ thống.

2.2.3 Tính mở rộng (Scalability & Performance)

Kiến trúc cloud-based, micro-backend:

- Các hệ thống dùng Azure, Firebase, cloud database cho thấy khả năng mở rộng người dùng và tổ chức tốt, với dữ liệu đồng bộ realtime.
- Backend xây dựng trên Node.js hoặc Python Flask/FastAPI, kết hợp kiến trúc non-blocking / async I/O để phục vụ hàng ngàn request đồng thời (check-in, liveness, đặt câu hỏi cho chatbot, truy vấn dashboard).

Tối ưu mô hình AI trên mobile & server:

- Mô hình nhận diện/on-device dùng MobileNetV2/MobileFaceNet, SSD cải tiến để đạt >95% độ chính xác mà vẫn đảm bảo FPS cao trên thiết bị cấu hình vừa phải .
- Các hệ thống thực chứng đã đạt ~25 FPS và độ chính xác >95% trên dataset nhỏ, cho thấy khả năng triển khai thực tế trong bối cảnh chấm công .

Độ tin cậy và phục hồi lỗi:

Nếu mất kết nối, app lưu log chấm công tạm thời và đồng bộ lên cloud khi online, tương tự cách các hệ thống attendance cloud xử lý để đảm bảo tính liên tục.

2.2.4 Các yêu cầu chất lượng khác

Tính chính xác & toàn vẹn dữ liệu:

- Kết hợp multi-factor (geofence + liveness + face) làm tăng độ tin cậy, đã được chứng minh làm giảm proxy attendance trong nhiều bối cảnh.
- Attendance record luôn gắn với: ID nhân viên, timestamp, geofence, kết quả nhận diện, giúp đối chiếu và kiểm toán.

Khả năng tích hợp:

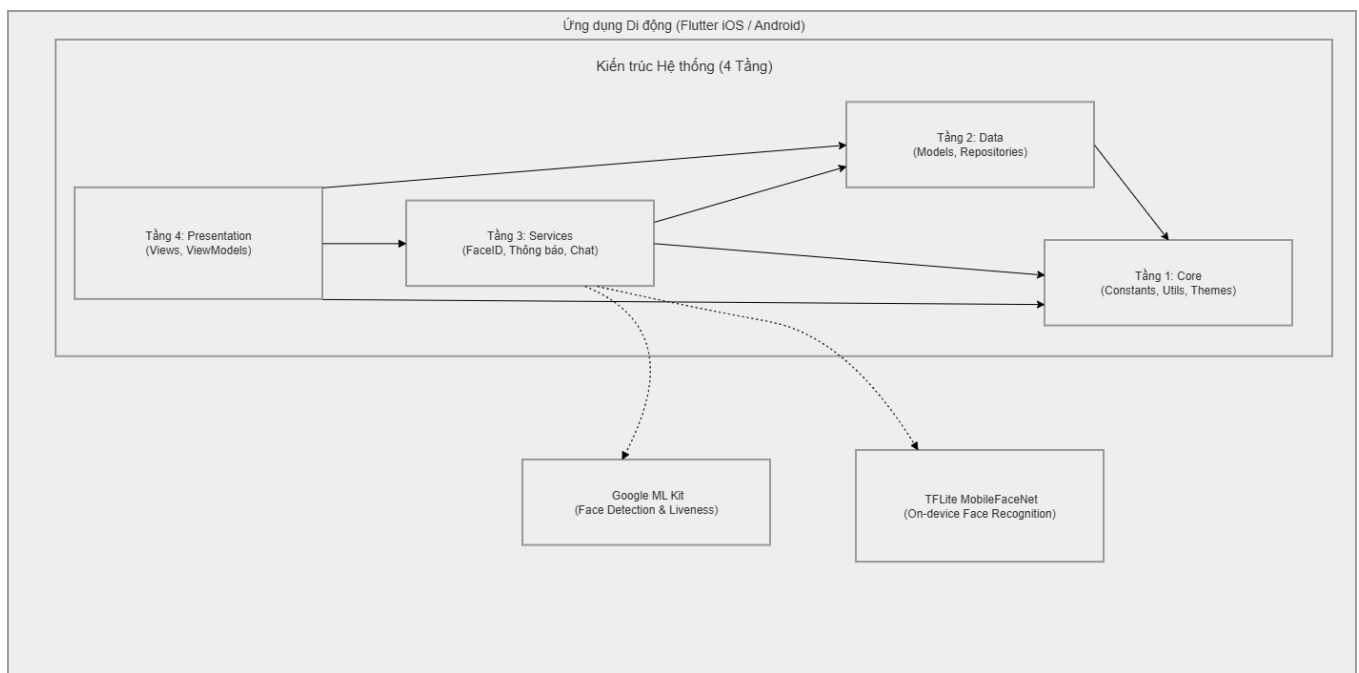
Do dùng cloud và API REST/Realtime DB, hệ thống có thể kết nối với các module HRM, Payroll, hoặc các công cụ HR 4.0 khác trong tương lai.

CHƯƠNG 3: THIẾT KẾ KIẾN TRÚC VÀ CƠ SỞ DỮ LIỆU CHUYÊN SÂU

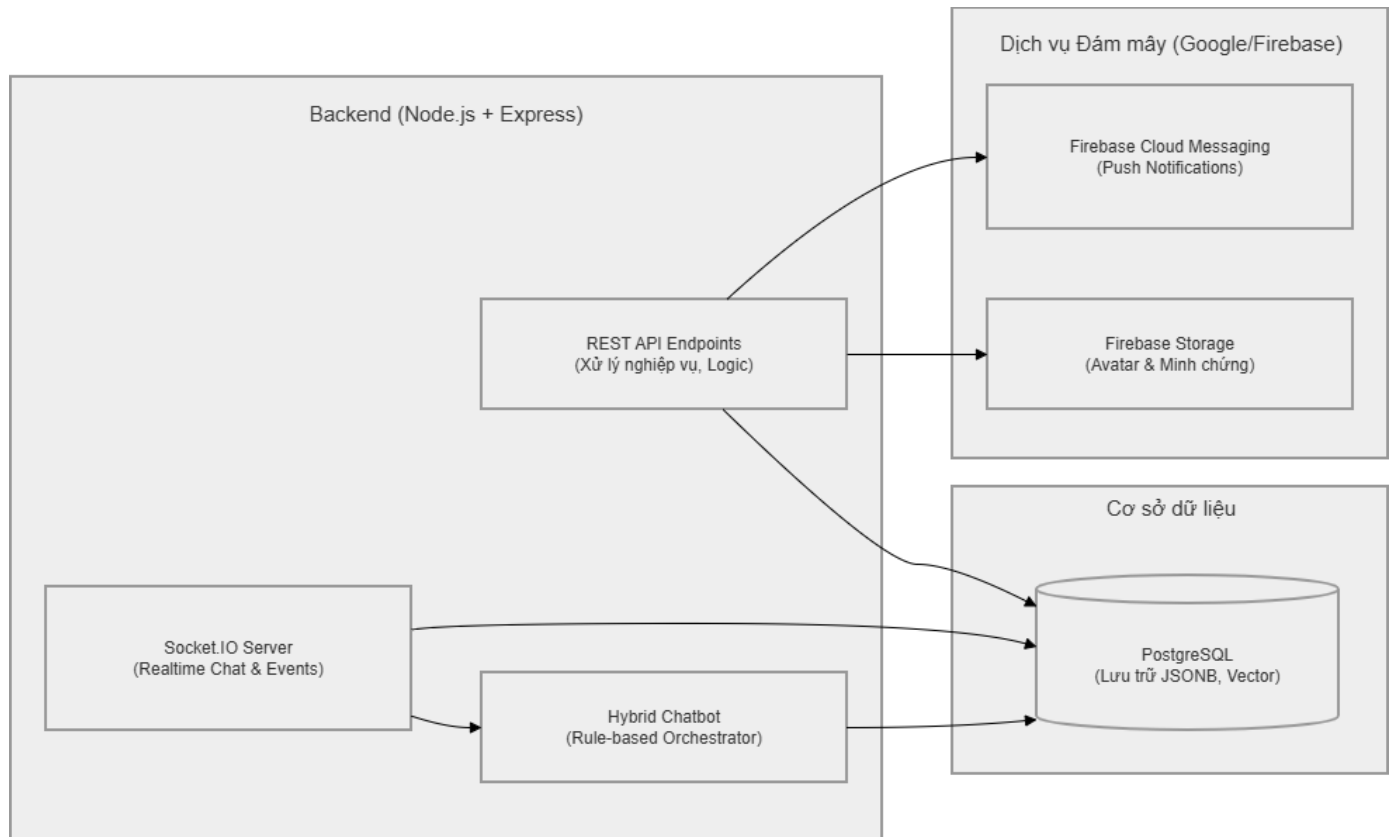
3.1 Tổng quan về Clean Architecture

Clean Architecture là mô hình kiến trúc phần mềm được đề xuất bởi Robert C. Martin (Uncle Bob), với nguyên tắc cốt lõi là "Dependency Rule" – Quy tắc phụ thuộc hướng vào trong". Theo nguyên tắc này, các lớp bên ngoài (UI, Framework) được phép phụ thuộc vào các lớp bên trong (Logic nghiệp vụ), nhưng không bao giờ được phép ngược lại. Điều này giúp:

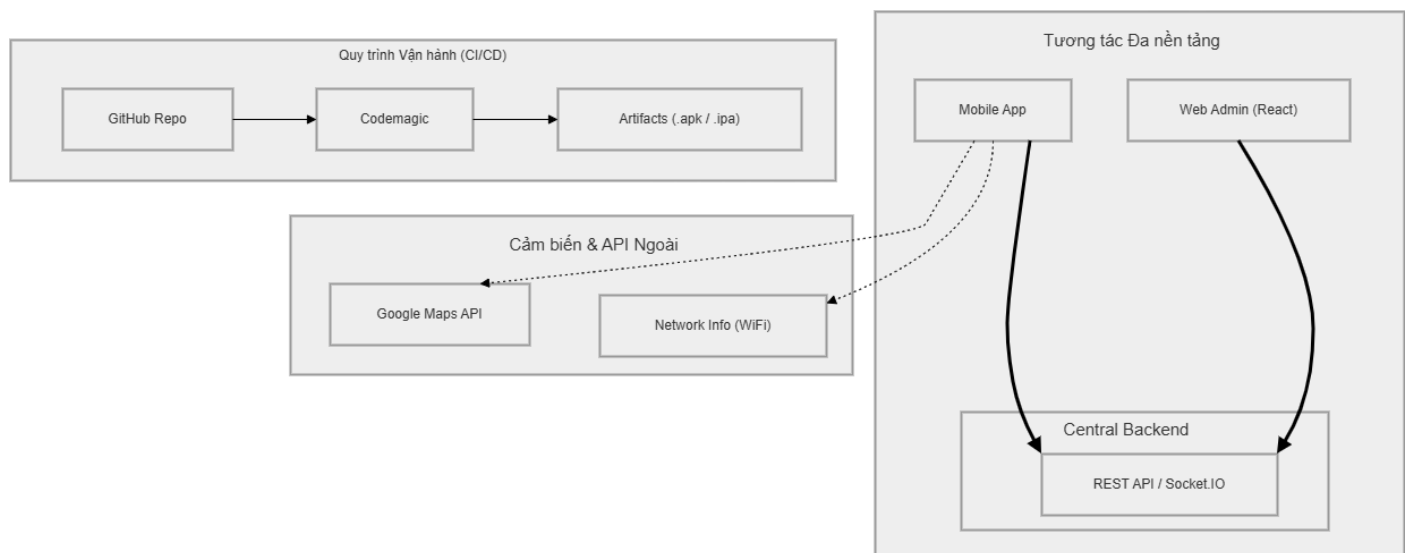
- Dễ bảo trì: Thay đổi giao diện không ảnh hưởng đến logic xử lý.
- Dễ kiểm thử: Có thể viết Unit Test cho logic nghiệp vụ mà không cần khởi chạy giao diện.
- Dễ mở rộng: Thêm tính năng mới chỉ cần tạo thêm module, không cần sửa đổi module cũ.



Hình 3.1a: Kiến trúc phân tầng và các thành phần xử lý AI trên thiết bị di động.



Hình 3.1b: Kiến trúc hạ tầng Backend, Cơ sở dữ liệu và các dịch vụ đám mây.

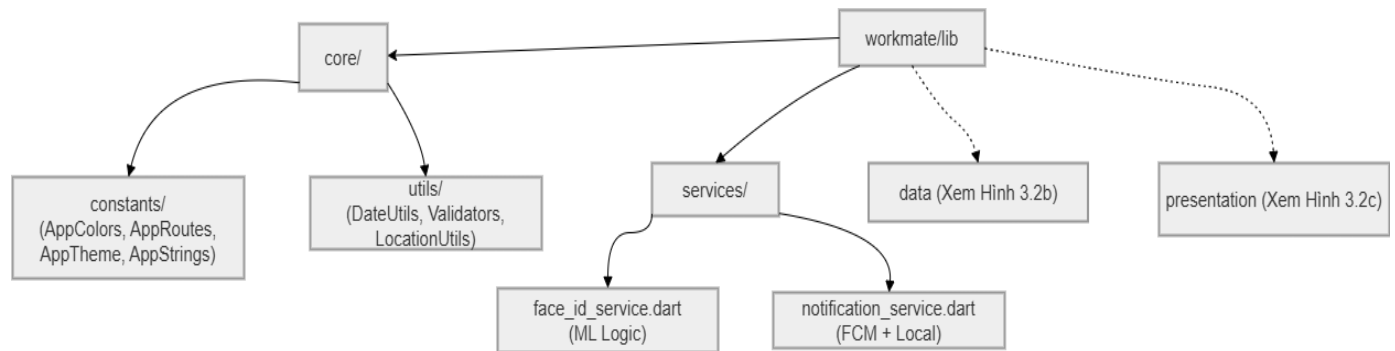


Hình 3.1c: Sơ đồ tích hợp hệ thống tổng thể và quy trình triển khai tự động.

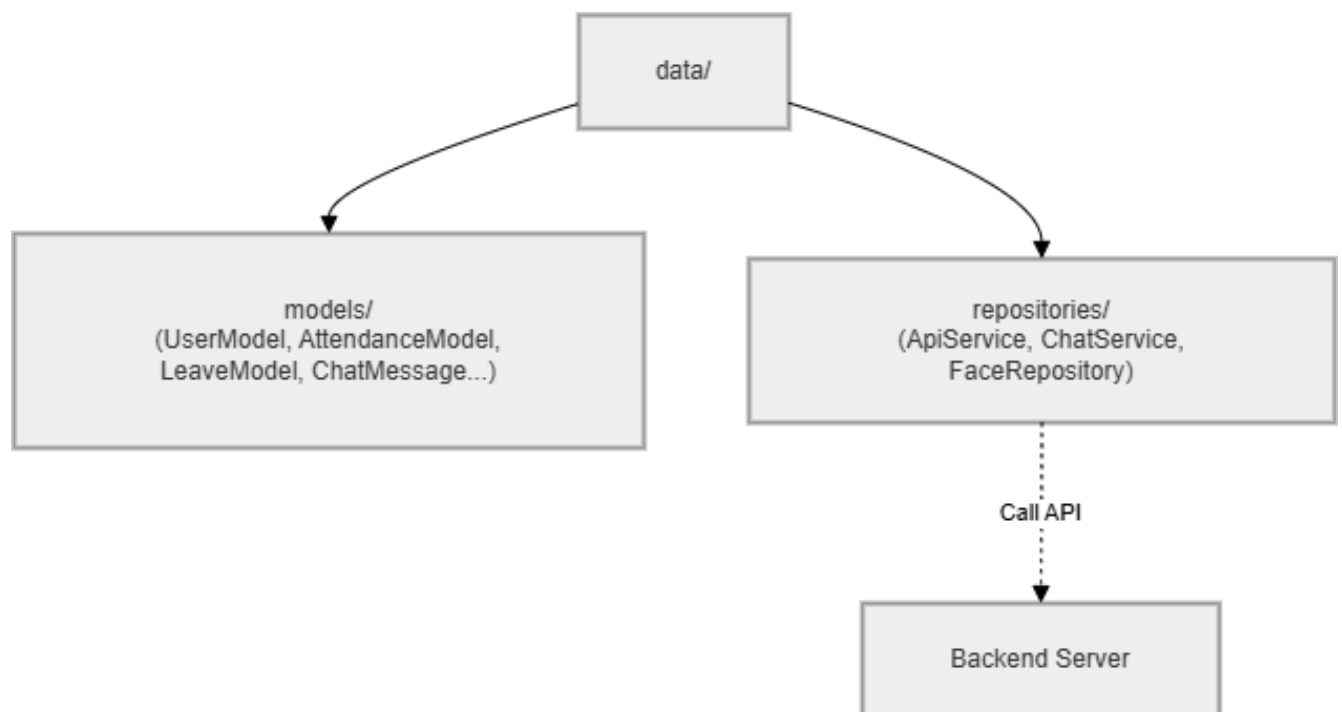
**Hệ thống WorkMate được xây dựng dựa trên kiến trúc phân tán hiện đại, đảm bảo tính mở rộng và hiệu năng cao. Sơ đồ kiến trúc tổng quát (Hình 3.1) được chi tiết hóa qua ba góc nhìn: cấu trúc nội tại của ứng dụng di động (Hình 3.1a), hệ sinh thái xử lý dữ liệu tại Backend (Hình 3.1b) và luồng tích hợp vận hành toàn hệ thống (Hình 3.1c).*

Hệ thống WorkMate được tổ chức thành 4 tầng chính trong thư mục lib/:

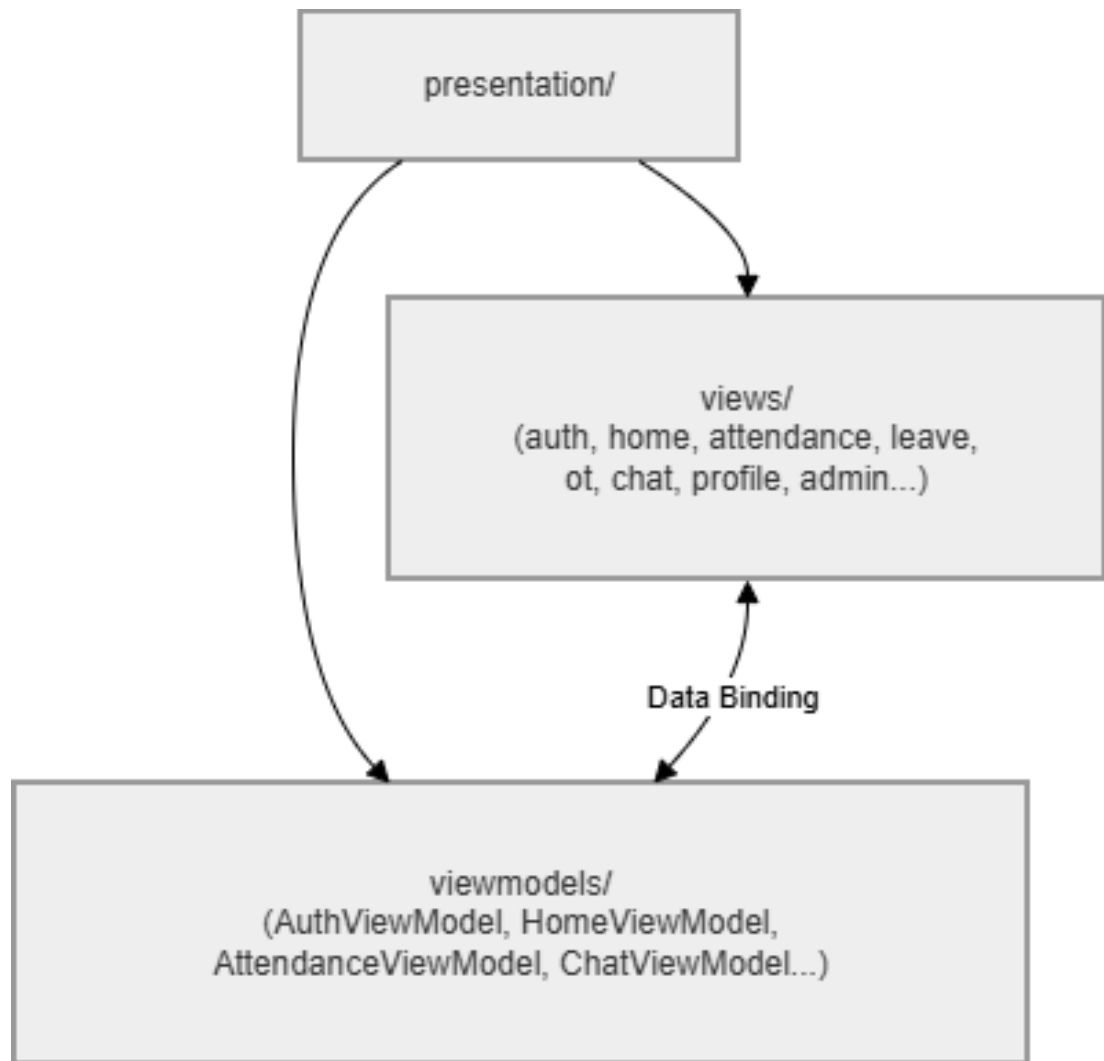
```
workmate/lib/
├── core/           ← Tầng 1: Lõi hệ thống
│   ├── constants/ (AppColors, AppRoutes, AppTextStyles)
│   ├── utils/     (DateUtils, Validators)
│   └── i18n/      (Đa ngôn ngữ Việt-Anh)
├── data/          ← Tầng 2: Dữ liệu
│   ├── models/    (UserModel, AttendanceModel, LeaveModel...)
│   └── repositories/ (ApiService, ChatService, FirebaseService)
├── services/      ← Tầng 3: Dịch vụ chuyên biệt
│   ├── face_id_service.dart (Xử lý AI nhận diện khuôn mặt)
│   └── notification_service.dart (Quản lý thông báo đẩy FCM)
├── presentation/  ← Tầng 4: Giao diện người dùng
│   ├── viewmodels/ (AuthViewModel, AttendanceViewModel...)
│   ├── views/      (14 nhóm màn hình)
│   └── widgets/     (Các widget tái sử dụng)
└── main.dart      ← Điểm khởi chạy ứng dụng
```



Hình 3.2a: Cấu trúc thư mục gốc và các gói thành phần hỗ trợ.



Hình 3.2b: Cấu trúc gói Data (Xử lý dữ liệu và Repository).



Hình 3.2c: Cấu trúc gói Presentation (Giao diện và ViewModel).

* Cấu trúc thư mục của dự án WorkMate được tổ chức chặt chẽ theo nguyên lý Clean Architecture nhằm tách biệt các trách nhiệm xử lý. Sơ đồ phân gói (Hình 3.2) minh họa cách sắp xếp các thành phần: từ các gói tiện ích cốt lõi và dịch vụ hệ thống (Hình 3.2a), đến cách quản lý dữ liệu tại tầng Data (Hình 3.2b), và cuối cùng là tầng Presentation quản lý logic hiển thị MVVM (Hình 3.2c).

3.2 Chi tiết từng tầng kiến trúc

3.2.1 Tầng Core (Lõi hệ thống)

Tầng Core chứa các thành phần không phụ thuộc vào bất kỳ tầng nào khác. Đây là nền tảng được sử dụng chung trong toàn bộ ứng dụng.

a) Hằng số màu sắc (AppColors)

File `app_colors.dart` định nghĩa bảng màu chủ đạo của ứng dụng theo hệ thống Material Design 3. Việc tập trung màu sắc vào một file duy nhất giúp đảm bảo tính nhất quán về giao diện và dễ dàng thay đổi theme khi cần.

Ví dụ:

màu `primary = #1C6185` (Xanh Navy) được sử dụng xuyên suốt cho AppBar, nút bấm và các thành phần nổi bật.

b) Định tuyến màn hình (AppRoutes)

File `app_routes.dart` chứa tất cả các đường dẫn (routes) của ứng dụng dưới dạng hằng số static `const String`. Hệ thống có tổng cộng 25 route được phân chia theo chức năng:

- Auth (4 route)
- Attendance (5 route)
- Leave (3 route)
- OT (3 route)
- Meeting (2 route)
- Profile (5 route)
- Chat (2 route)
- Admin (1 route)

⇒ Cách tổ chức này giúp tránh lỗi gõ sai tên route và dễ dàng quản lý khi ứng dụng mở rộng.

c) Kiểu chữ (AppTextStyles)

File `app_text_styles.dart` quy chuẩn toàn bộ kiểu chữ sử dụng trong ứng dụng (heading, body, caption...) dựa trên font Nunito, đảm bảo typography nhất quán trên mọi màn hình.

3.2.2 Tầng Data (Dữ liệu)

Tầng Data chịu trách nhiệm giao tiếp với các nguồn dữ liệu bên ngoài (API Server, Firebase, Local Storage). Tầng này gồm 2 thành phần chính:

a) Models (Mô hình dữ liệu) File `models.dart` chứa 9 lớp (class) mô hình dữ liệu, mỗi lớp đại diện cho một thực thể trong hệ thống:

Bảng 3.1: Danh sách các mô hình dữ liệu (Models) trong hệ thống.

STT	Tên Model	Số thuộc tính	Mô tả chức năng
1	UserModel	20	Thông tin nhân viên: mã NV, tên, email, phòng ban, chức vụ, lương cơ bản, điểm thâm niên
2	AttendanceModel	11	Bản ghi chấm công: thời gian vào/ra, phương thức, trạng thái (đúng giờ/muộn)
3	LeaveModel	13	Đơn nghỉ phép: loại nghỉ, ngày bắt đầu/kết thúc, tệp đính kèm, trạng thái duyệt
4	OvertimeModel	11	Đơn tăng ca: ngày, số giờ dự kiến, nội dung công việc
5	MeetingModel	12	Lịch họp: tiêu đề, thời gian, địa điểm, link online, danh sách phòng ban tham gia
6	NotificationModel	7	Thông báo đẩy: tiêu đề, nội dung, loại, trạng thái đã đọc
7	CompanyConfigModel	5	Cấu hình công ty: tọa độ GPS an

			toàn, tên WiFi an toàn
8	ChatMessage	7	Tin nhắn chat: người gửi/nhận, nội dung, cờ AI, trạng thái đã đọc
9	BankAccount	5	Tài khoản ngân hàng: tên ngân hàng, số tài khoản, chủ tài khoản

=> Đặc biệt, file `api_service.dart` có cơ chế tự động phát hiện môi trường để chọn đúng địa chỉ IP server: nếu chạy trên Android Emulator sẽ dùng 10.0.2.2, nếu chạy trên iPhone thật sẽ dùng IP máy tính qua mạng cục bộ.

3.2.3 Tầng Services (Dịch vụ chuyên biệt)

Tầng Services chứa các logic phức tạp, chuyên sâu không nên đặt trong ViewModel hay Repository:

a) FaceIdService (7.7KB) Đây là dịch vụ quan trọng nhất của hệ thống, triển khai toàn bộ pipeline nhận diện khuôn mặt:

- Khởi tạo TFLite Interpreter từ file model `mobilefacenet.tflite`.
- Tiền xử lý ảnh: crop, resize (112x112), normalize pixel values từ [0,255] sang [-1,1].
- Chạy inference để trích xuất vector embedding 192 chiều.
- Tính Cosine Similarity để so khớp danh tính.
- Quản lý vòng đời (lifecycle) của model AI trên bộ nhớ thiết bị.

b) NotificationService (4KB) Quản lý toàn bộ hệ thống thông báo:

- Khởi tạo Firebase Cloud Messaging (FCM) một cách an toàn (null-safe) để tránh crash trên môi trường CI/CD.
- Cấu hình Local Notifications cho thông báo nội bộ.
- Xử lý thông báo khi ứng dụng ở foreground, background và terminated.

3.2.4 Tầng Presentation (Giao diện)

Tầng Presentation được chia thành 3 thành phần:

a) ViewModels (Quản lý trạng thái)

Sử dụng thư viện Provider (mô hình ChangeNotifier) để quản lý trạng thái reactive. Hệ thống có 9 ViewModel được đăng ký tại main.dart:

Bảng 3.2: Danh sách các ViewModels và trách nhiệm xử lý.

ViewModel	Trách nhiệm chính
AuthViewModel	Xử lý đăng nhập, đăng xuất, quản lý phiên làm việc
ProfileViewModel	Quản lý thông tin cá nhân, avatar, tài khoản ngân hàng
AttendanceViewModel	Xử lý logic chấm công, kiểm tra GPS/WiFi/FaceID
HomeViewModel	Tải dữ liệu tổng quan cho màn hình chính
LeaveViewModel	Gửi đơn nghỉ phép, lấy lịch sử đơn từ
OvertimeViewModel	Gửi đơn tăng ca, theo dõi trạng thái duyệt
StatisticsViewModel	Tính toán và hiển thị biểu đồ thống kê
NotificationViewModel	Quản lý danh sách thông báo, đánh dấu đã đọc
MeetingViewModel	Lấy danh sách lịch họp theo phòng ban

b) Views (14 nhóm màn hình)

Tổng cộng 14 thư mục con, mỗi thư mục chứa các màn hình liên quan:

- auth/ (Đăng nhập, Quên MK)
- home/ (Trang chính, Navigation)
- attendance/ (Quét mặt, Vị trí)
- leave/ (Gửi đơn, Lịch sử)
- overtime/ (Gửi OT, Lịch sử OT)
- chat/ (Chat AI & Admin)
- profile/ (Hồ sơ, QR, Ngân hàng)
- meeting/ (Lịch họp)
- statistics/ (Biểu đồ)

- notification/ (Thông báo)
- splash/
- onboarding/
- schedule/
- admin/.

c) Widgets (Widget tái sử dụng)

Chứa các component UI được thiết kế để sử dụng lại nhiều lần: StatCard, LoadingShimmer, EmptyState, CustomAppBar...

3.3 Thiết kế Cơ sở dữ liệu (PostgreSQL)

3.3.1 Lý do chọn PostgreSQL

- Hỗ trợ JSONB: Cho phép lưu trữ dữ liệu bán cấu trúc (như mảng embedding 192 chiều) với hiệu suất truy vấn cao.
- Tính toàn vẹn dữ liệu: Hỗ trợ đầy đủ Foreign Key, Constraint, Transaction ACID.
- Mở rộng linh hoạt: Hỗ trợ kiểu dữ liệu POINT cho tọa độ GPS, ARRAY cho danh sách.
- Miễn phí và mã nguồn mở: Phù hợp với quy mô doanh nghiệp vừa và nhỏ.

3.3.2 Từ điển dữ liệu chi tiết (Data Dictionary)

Bảng 3.3: Từ điển dữ liệu - Bảng departments (Phòng ban).

Trường	Kiểu dữ liệu	Ràng buộc
id	SERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
code	VARCHAR(50)	UNIQUE, NOT NULL
positions	JSONB	DEFAULT '[]'
created_at	TIMESTAMP	DEFAULT NOW()

Bảng 3.4: Từ điển dữ liệu - Bảng employees (Nhân viên).

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	SERIAL	PRIMARY KEY	Mã định danh nội bộ

employee_code	VARCHAR(50)	UNIQUE, NOT NULL	Mã nhân viên (VD: NV001)
name	VARCHAR(255)	NOT NULL	Họ và tên đầy đủ
email	VARCHAR(255)	UNIQUE, NOT NULL	Email đăng nhập
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã mã hóa BCrypt
phone	VARCHAR(20)		Số điện thoại liên hệ
department_id	INTEGER	FK → departments(id)	Liên kết phòng ban
department_name	VARCHAR(255)		Tên phòng ban (denormalized)
position	VARCHAR(255)		Chức vụ (Nhân viên, Trưởng phòng...)
role	VARCHAR(50)	DEFAULT 'user'	Vai trò hệ thống: admin / user
avatar_url	TEXT		Đường dẫn ảnh đại diện trên Firebase Storage
base_salary	DECIMAL(15,2)	DEFAULT 0	Lương cơ bản (VNĐ)
join_date	DATE		Ngày bắt đầu làm việc
birthday	DATE		Ngày sinh
fcm_token	TEXT		Token thiết bị để gửi Push Notification
seniority_points	INTEGER	DEFAULT 0	Điểm thâm niên tích lũy
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm tạo tài khoản

Bảng 3.5: Từ điển dữ liệu - Bảng attendance (Chấm công).

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	SERIAL	PRIMARY KEY	Mã bản ghi chấm công
employee_id	INTEGER	FK → employees(id)	Nhân viên thực hiện chấm công
employee_name	VARCHAR(255)		Tên nhân viên (denormalized)
check_in_time	TIMESTAMP	DEFAULT NOW()	Thời điểm check-in chính xác
check_out_time	TIMESTAMP	NULLABLE	Thời điểm check-out
check_in_method	VARCHAR(50)		Phương thức: FaceID / WiFi / QR
check_in_lat	DOUBLE PRECISION		Vĩ độ GPS khi check-in

check_in_lng	DOUBLE PRECISION		Kinh độ GPS khi check-in
wifi_ssid	VARCHAR(100)		Tên mạng WiFi khi chấm công
status	VARCHAR(20)		Trạng thái: on time / late
shift_type	VARCHAR(50)		Ca làm việc: morning / afternoon / full
note	TEXT		Ghi chú bổ sung
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm tạo bản ghi

Bảng 3.6: Từ điển dữ liệu - Bảng approvals (Đơn từ / Phê duyệt).

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	SERIAL	PRIMARY KEY	Mã đơn từ
employee_id	INTEGER	FK → employees(id)	Nhân viên gửi đơn
name	VARCHAR(255)		Tên nhân viên
type	VARCHAR(100)		Loại đơn: Nghỉ phép / Tăng ca / Bổ sung công
reason	TEXT		Lý do xin nghỉ/tăng ca
status	VARCHAR(50)	DEFAULT 'pending'	Trạng thái: pending / approved / rejected
from_date	DATE		Ngày bắt đầu nghỉ
to_date	DATE		Ngày kết thúc nghỉ
attachment_urls	JSONB		Danh sách URL ảnh minh chứng
reviewed_by	VARCHAR(255)		Tên người phê duyệt
reviewed_at	TIMESTAMP		Thời điểm phê duyệt
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm gửi đơn

Bảng 3.7: Từ điển dữ liệu - Bảng meetings (Lịch họp).

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	SERIAL	PRIMARY KEY	Mã cuộc họp
title	VARCHAR(255)		Tiêu đề cuộc họp
content	TEXT		Nội dung chi tiết

location	VARCHAR(255)		Địa điểm (phòng họp / online)
start time	TIMESTAMP		Thời gian bắt đầu
end time	TIMESTAMP		Thời gian kết thúc
is_online	BOOLEAN	DEFAULT FALSE	Cuộc họp online hay offline
meet_link	TEXT		Link Google Meet / Zoom
department_ids	JSONB	DEFAULT '[]'	Danh sách mã phòng ban tham gia
created_by	INTEGER	FK employees(id) →	Người tạo cuộc họp
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm tạo

Bảng 3.8: Từ điển dữ liệu - Bảng face_embeddings (Dữ liệu khuôn mặt).

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	SERIAL	PRIMARY KEY	Mã bản ghi
employee_id	INTEGER	FK employees(id) →	Nhân viên sở hữu
embedding	DOUBLE PRECISION[]	NOT NULL	Mảng 192 giá trị số thực đại diện cho vector khuôn mặt
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm đăng ký khuôn mặt

Lưu ý bảo mật: Hệ thống không lưu ảnh gốc khuôn mặt của nhân viên. Chỉ lưu trữ vector số học 192 chiều. Từ vector này không thể tái tạo ngược lại hình ảnh khuôn mặt ban đầu, đảm bảo tuân thủ nguyên tắc Privacy by Design.

Bảng 3.9: Từ điển dữ liệu - Bảng chat_messages (Tin nhắn chat).

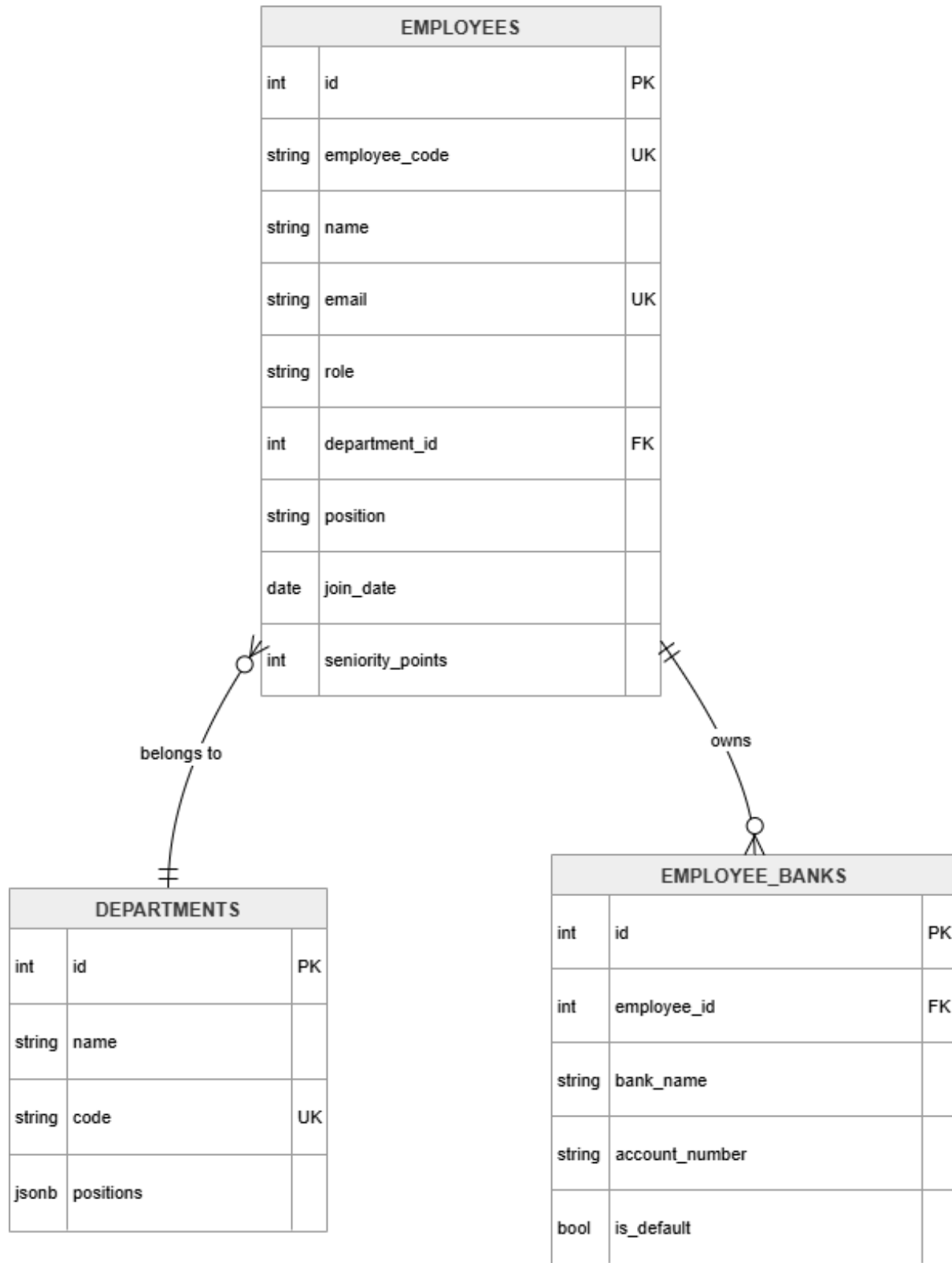
Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	SERIAL	PRIMARY KEY	Mã tin nhắn
sender_id	INTEGER	FK employees(id) →	Người gửi
receiver_id	INTEGER	FK employees(id), NULLABLE →	Người nhận (NULL = gửi cho Admin)

message	TEXT	NOT NULL	Nội dung tin nhắn
is_ai	BOOLEAN	DEFAULT FALSE	TRUE nếu là tin nhắn từ AI Bot
is_read	BOOLEAN	DEFAULT FALSE	Trạng thái đã đọc
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm gửi

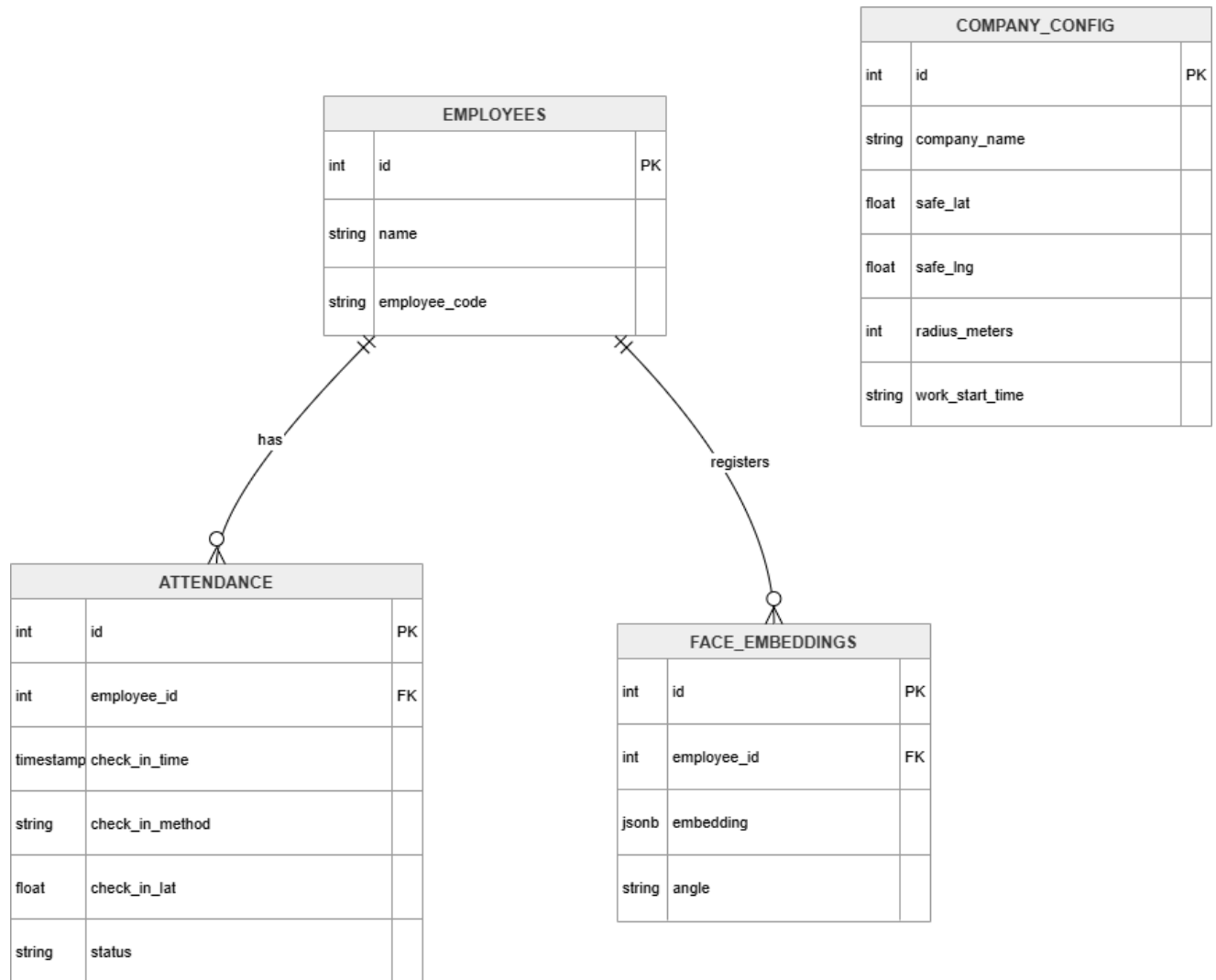
Bảng 3.10: Từ điển dữ liệu - Bảng company_config (Cấu hình công ty)

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	SERIAL	PRIMARY KEY	Mã cấu hình
company_name	VARCHAR(255)		Tên công ty
safe_lat	DOUBLE PRECISION		Vĩ độ trung tâm văn phòng
safe_lng	DOUBLE PRECISION		Kinh độ trung tâm văn phòng
safe_wifi_ssid	VARCHAR(100)		Tên mạng WiFi công ty
allowed_radius	INTEGER	DEFAULT 100	Bán kính cho phép châm công (mét)

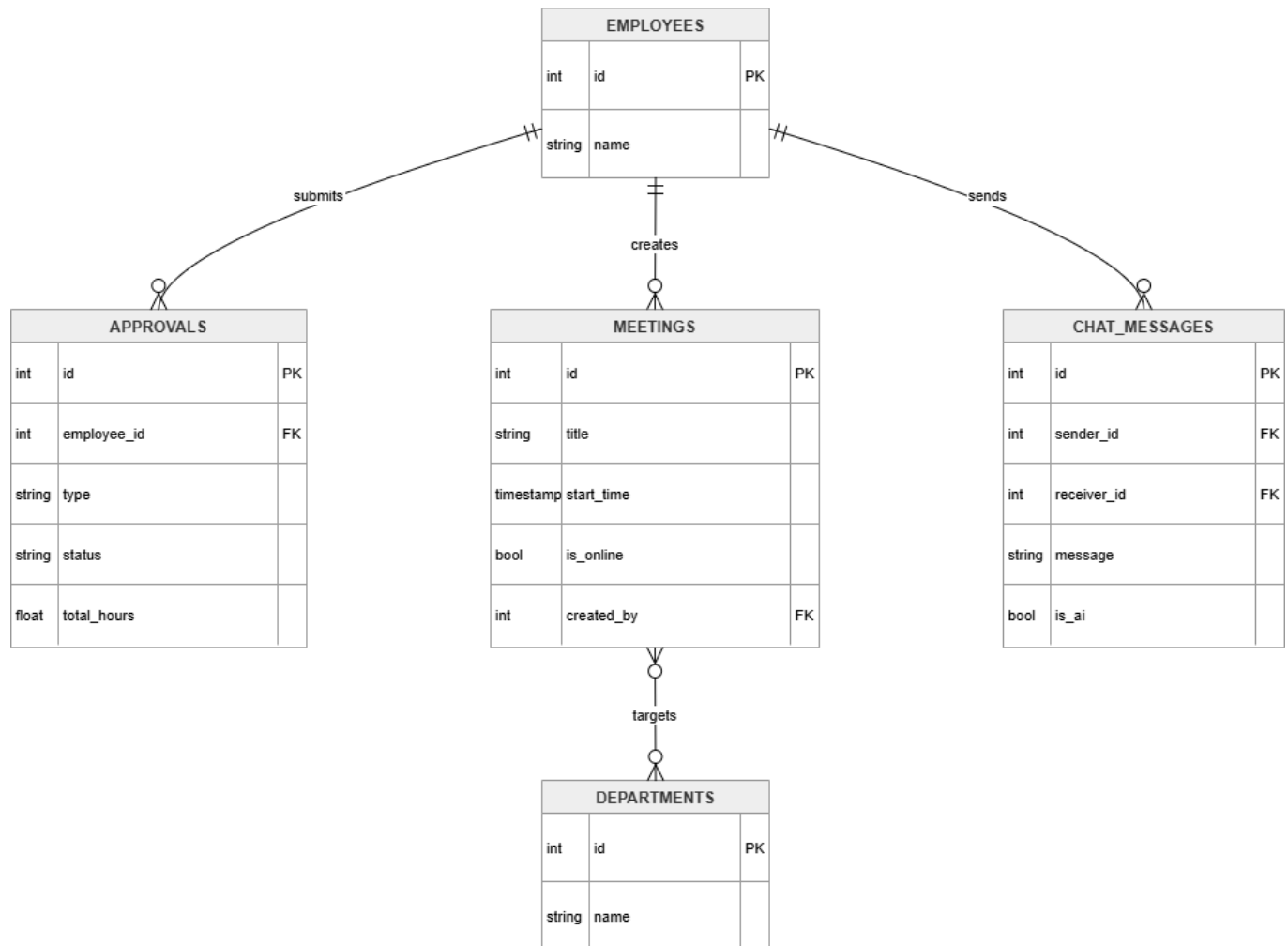
3.3.3 Môi quan hệ giữa các bảng



Hình 3.3a: Sơ đồ ERD – Quản lý cấu trúc tổ chức và thông tin nhân sự.



Hình 3.3b: Sơ đồ ERD – Quản lý dữ liệu chấm công và định danh sinh trắc học.

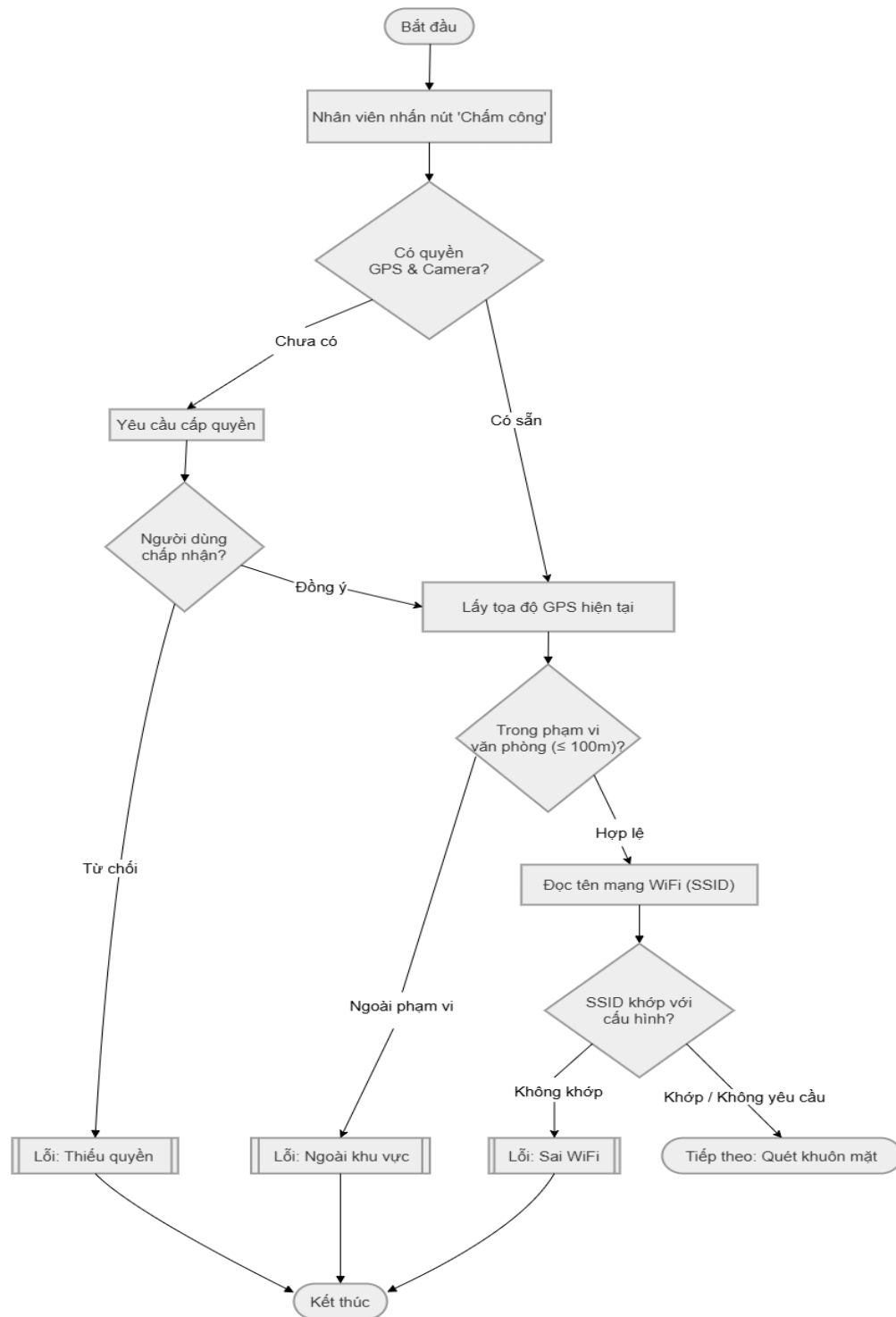


Hình 3.3c: Sơ đồ ERD – Quản lý quy trình phê duyệt và tương tác nội bộ.

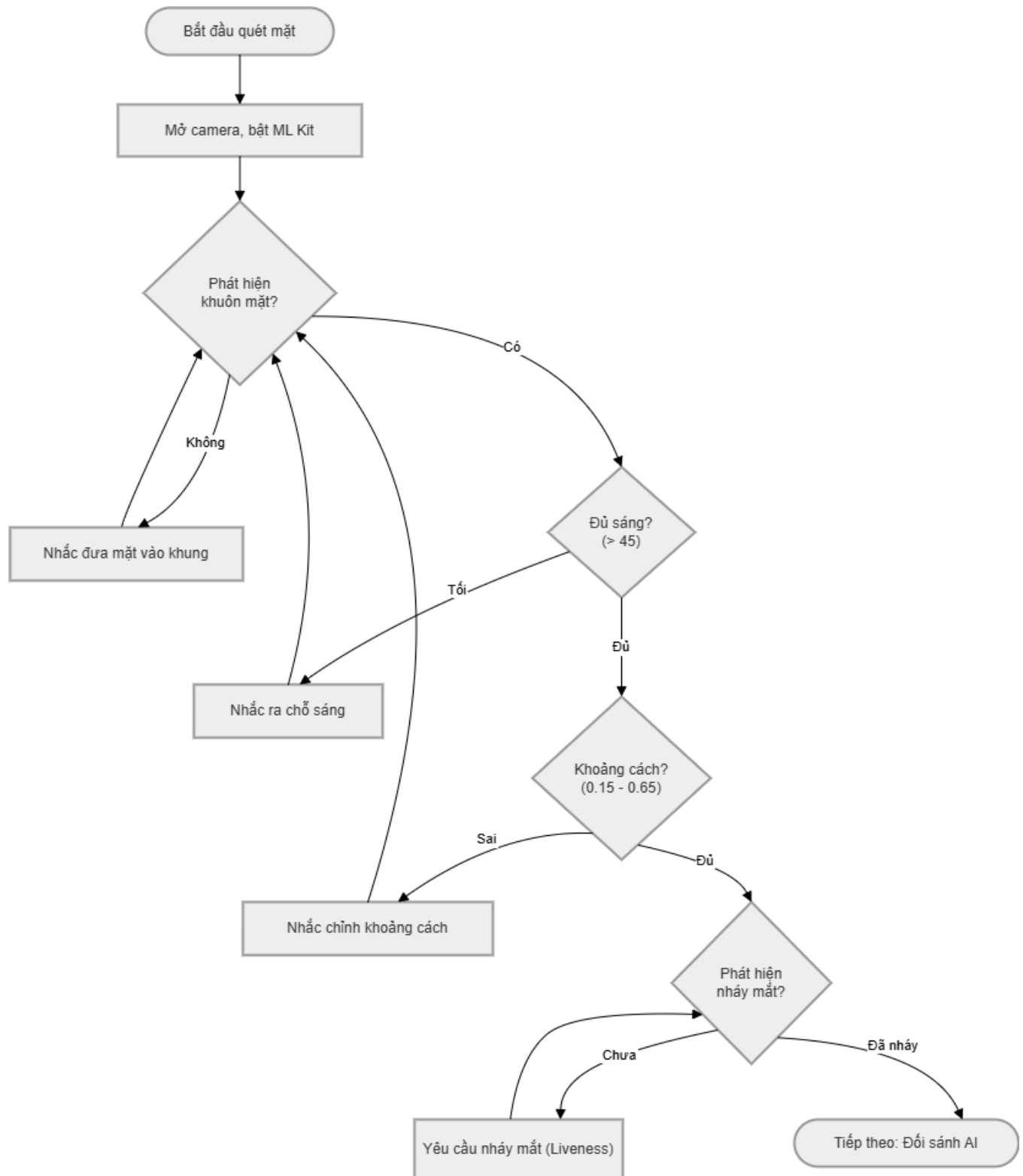
* Cơ sở dữ liệu của hệ thống WorkMate được thiết kế tối ưu để lưu trữ dữ liệu nhân sự phức tạp và các vector khuôn mặt. Để thuận tiện cho việc theo dõi, sơ đồ mối quan hệ thực thể (ERD) tại Hình 3.3 được chia thành ba nhóm chức năng: Quản lý thông tin gốc và cấu hình (Hình 3.3a), Quy trình chấm công và bảo mật (Hình 3.3b), và cuối cùng là hệ thống tương tác, phê duyệt (Hình 3.3c).

- ***employees* ↔ *departments***: Quan hệ N-1. Mỗi nhân viên bắt buộc thuộc về một phòng ban duy nhất. Phòng ban giúp phân nhóm quản lý và gửi thông báo/lich họp tập trung.
- ***employees* ↔ *attendance***: Quan hệ 1-N. Một nhân viên có nhiều bản ghi chấm công (check-in/check-out) theo thời gian. Mỗi bản ghi lưu vết cả vị trí GPS, WiFi và phương thức xác thực.
- ***employees* ↔ *approvals***: Quan hệ 1-N. Một nhân viên có thể gửi nhiều yêu cầu (nghỉ phép, tăng ca, bổ sung công). Các đơn này sẽ được lưu trữ kèm minh chứng và trạng thái phê duyệt từ Admin.
- ***employees* ↔ *face_embeddings***: Quan hệ 1-N. Một nhân viên đăng ký tối đa 5 mẫu vector khuôn mặt (chính diện, trái, phải, trên, dưới) để tối ưu hóa độ chính xác cho bộ lọc AI On-device.
- ***employees* ↔ *chat_messages***: Quan hệ 1-N. Một nhân viên có thể gửi hoặc nhận nhiều tin nhắn. Hệ thống hỗ trợ cả chat với trợ lý ảo AI, chat riêng tư với đồng nghiệp và chat với Admin.
- ***employees* ↔ *employee_banks***: Quan hệ 1-N. Một nhân viên có thể sở hữu tối đa 3 tài khoản ngân hàng (trong đó có 1 tài khoản mặc định) để phục vụ việc chi trả lương.
- ***departments* ↔ *meetings***: Quan hệ N-N (triển khai qua JSONB). Một cuộc họp có thể chỉ định mời nhiều phòng ban tham gia đồng thời. Dữ liệu phòng ban được lưu dạng mảng ID trong bảng meetings.
- ***employees* ↔ *meetings***: Quan hệ 1-N. Một nhân viên (vai trò Admin/Manager) có thể khởi tạo và quản lý nhiều cuộc họp khác nhau.

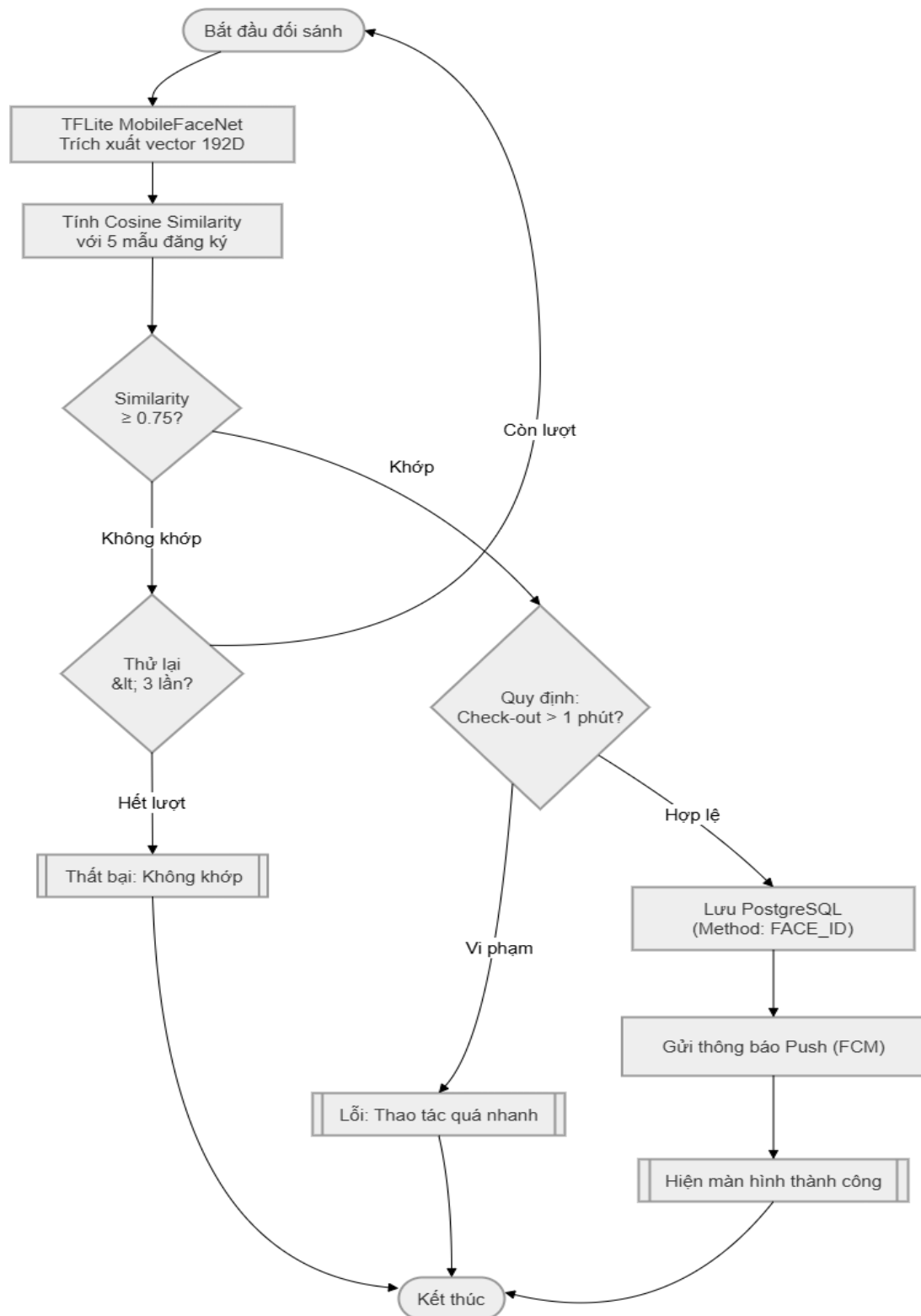
CHƯƠNG 4: GIẢI PHÁP KỸ THUẬT VÀ THUẬT TOÁN AI CHI TIẾT



Hình 4.1a: Sơ đồ hoạt động – Quy trình kiểm tra vị trí và điều kiện kết nối mạng.



Hình 4.1b: Sơ đồ hoạt động – Quy trình kiểm tra chất lượng hình ảnh và nhận diện người thật (Liveness).



Hình 4.1c: Sơ đồ hoạt động – Quy trình đối sánh AI và ghi nhận dữ liệu chấm công.

** Quy trình chấm công thông minh trên WorkMate được thiết kế chặt chẽ nhằm đảm bảo tính minh bạch và chống gian lận. Toàn bộ luồng hoạt động (Hình 4.1) được chia thành ba giai đoạn xử lý nối tiếp: đầu tiên là kiểm tra các điều kiện tiên quyết về vị trí GPS và WiFi (Hình 4.1a), tiếp theo là sử dụng ML Kit để đảm bảo chất lượng ảnh chụp và xác thực liveness (Hình 4.1b), và cuối cùng là sử dụng model TFLite để đối sánh vector đặc trưng khuôn mặt trước khi lưu dữ liệu vào hệ thống (Hình 4.1c).*

4.1 Thuật toán xác thực vị trí địa lý (Geofencing – Haversine)

4.1.1 Bài toán và thách thức

Thách thức trong xác thực vị trí GPS là Trái Đất có hình cầu, nên không thể dùng công thức khoảng cách Euclid phẳng thông thường. Sai số có thể lên đến hàng chục mét nếu dùng sai công thức, đặc biệt khi vị trí văn phòng gần đường xích đạo (như Việt Nam). Hệ thống WorkMate sử dụng **công thức Haversine** — một trong những phương pháp chính xác nhất để tính khoảng cách trên mặt cầu.

4.1.2 Công thức Haversine chi tiết

```
a = sin2(Δlat/2) + cos(lat1) × cos(lat2) × sin2(Δlon/2)
c = 2 × atan2(√a, √(1-a))
d = R × c
```

Trong đó:

- R= 6.371.000 mét (bán kính Trái Đất)
- lat1, lon1: Tọa độ văn phòng (lấy từ company_config) • lat2, lon2: Tọa độ GPS hiện tại của nhân viên
- d: Khoảng cách thực tế (đơn vị mét)

4.1.3 Triển khai trong Backend

(Node.js – index.js, dòng 484-495)

```
function getDistance(lat1, lon1, lat2, lon2) {  
  const R = 6371e3; // metres  
  
  const  $\phi1$  = lat1 * Math.PI / 180;  
  const  $\phi2$  = lat2 * Math.PI / 180;  
  
  const  $\Delta\phi$  = (lat2 - lat1) * Math.PI / 180;  
  const  $\Delta\lambda$  = (lon2 - lon1) * Math.PI / 180;  
  
  const a = Math.sin( $\Delta\phi/2$ ) * Math.sin( $\Delta\phi/2$ ) +  
    Math.cos( $\phi1$ ) * Math.cos( $\phi2$ ) *  
    Math.sin( $\Delta\lambda/2$ ) * Math.sin( $\Delta\lambda/2$ );  
  
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));  
  return R * c; // khoảng cách tính bằng mét  
}
```

4.1.4 Luồng xử lý tại Server

(index.js, dòng 511-517)

Khi nhận request chấm công, server thực hiện:

1. Truy vấn bảng company_config lấy safe_lat, safe_lng và radius_meters.
2. Gọi getDistance(lat, lng, config.safe_lat, config.safe_lng).
3. Nếu khoảng cách vượt quá ngưỡng cấu hình, trả về lỗi HTTP 403:
 - "Bạn đang ở ngoài vùng cho phép ($\text{\${Math.round(distance)}}m$)"
4. Nếu hợp lệ, tiếp tục sang bước kiểm tra khuôn mặt.

4.1.5 Kiểm tra WiFi (Bước song song với GPS)

Trước khi kiểm tra GPS, hệ thống kiểm tra wifi_ssid gửi từ app:

```
if (config.safe_wifi_ssid && wifi_ssid !== config.safe_wifi_ssid) {  
  return res.status(403).json({  
    success: false,  
    message: `Vui lòng kết nối WiFi: ${config.safe_wifi_ssid}`  
  });  
}
```

Việc kết hợp GPS + WiFi tạo thành "**2 lớp vùng an toàn**": GPS xác định vị trí địa lý, WiFi xác nhận nhân viên đang trong môi trường mạng nội bộ của công ty, rất khó giả mạo đồng thời cả hai.

4.2 Pipeline Nhận diện Khuôn mặt (FaceID – On-Device AI)

4.2.1 Kiến trúc tổng thể

Toàn bộ quá trình nhận diện diễn ra hoàn toàn trên thiết bị di động (On-Device). Không có hình ảnh nào được truyền lên server. Chỉ có vector số 192 chiều được gửi đi, bảo vệ hoàn toàn quyền riêng tư của nhân viên.

Pipeline gồm 6 bước chính, được triển khai trong `face_id_service.dart`:

4.2.2 Bước 1 – Khởi tạo Model (Initialization)

Khi ứng dụng khởi chạy (`main.dart`), `FaceIdService.instance.initialize()` được gọi để:

- Tạo `FaceDetector` với các tùy chọn: `enableLandmarks: true`, `enableClassification: true`, `enableTracking: true`, `performanceMode: FaceDetectorMode.fast`.
- Nạp model TFLite từ asset `assets/mobilefacenet.tflite` vào bộ nhớ (buffer).
- Tạo `Interpreter` với `threads = 4` để tận dụng đa nhân CPU.
- Sử dụng cờ `_isInitializing` để tránh khởi tạo song song gây lỗi.


```
_interpreter = Interpreter.fromBuffer(  
    buffer,  
    options: InterpreterOptions()..threads = 4,  
);
```

4.2.3 Bước 2 – Phát hiện khuôn mặt và Liveness Detection

Mỗi frame camera được truyền vào `detectFaceFromCameraImage()`. Hàm này thực hiện:

a) Kiểm tra độ sáng (Low-light Detection):

- Lấy mẫu ngẫu nhiên 100 điểm pixel từ plane Y (Luminance) của camera frame
- Tính giá trị trung bình: nếu $\text{avgBrightness} < 45 \rightarrow \text{isLowLight} = \text{true}$.
- Thông báo người dùng ra chỗ sáng hơn, không cho phép chấm công.

b) Phát hiện khuôn mặt (ML Kit):

```
final faces = await _faceDetector!.processImage(inputImage);
```

ML Kit trả về tọa độ bounding box, các điểm landmark (mắt, mũi, miệng) và xác suất mắt mở.

c) Kiểm tra khoảng cách khuôn mặt:

```
final faceSizeRatio = face.boundingBox.width / image.width;  
isTooClose: faceSizeRatio > 0.65, // Mặt chiếm >65% chiều rộng ảnh  
isTooFar:    faceSizeRatio < 0.15, // Mặt chiếm <15% chiều rộng ảnh
```

d) Kiểm tra căn giữa (Centered Check):

- Đảm bảo tâm khuôn mặt nằm trong phạm vi 20% từ trung tâm khung hình.

e) Phát hiện nháy mắt (Liveness Detection – Anti-Spoofing):

```
isBlinking: (face.leftEyeOpenProbability ?? 1.0) < 0.2  
            && (face.rightEyeOpenProbability ?? 1.0) < 0.2
```

ML Kit trả về xác suất mở mắt từ 0.0 đến 1.0. Nếu cả 2 mắt đều có xác suất **dưới 0.2** (tức là đang nhắm), hệ thống xác nhận đây là người thật. Tính năng này ngăn chặn việc dùng ảnh in hoặc màn hình để đánh lừa camera.

4.2.3 Bước 3 – Cắt và Tiền xử lý ảnh (Preprocessing)

Khi phát hiện nháy mắt, hệ thống chụp ảnh và gọi `extractEmbedding()`:

```
// Cắt vùng khuôn mặt từ ảnh gốc  
final cropped = img.copyCrop(raw, x: left, y: top,  
                               width: width, height: height);  
  
// Resize về 112x112 pixel (kích thước input của MobileFaceNet)  
final resized = img.copyResize(cropped,  
                                width: _modelInputSize, // = 112  
                                height: _modelInputSize);
```

4.2.4 Bước 4 – Chuẩn hóa pixel (Normalization)

Các giá trị pixel (R, G, B từ 0–255) được chuẩn hóa về khoảng [-1.0, 1.0]:

```
var input = List.generate(1, (_) =>
  List.generate(112, (y) =>
    List.generate(112, (x) {
      final p = resized.getPixel(x, y);
      return [
        (p.r / 127.5) - 1.0, // Red
        (p.g / 127.5) - 1.0, // Green
        (p.b / 127.5) - 1.0, // Blue
```

Kết quả là tensor `[1, 112, 112, 3]`— đúng định dạng input của MobileFaceNet.

4.2.6 Bước 5 – Trích xuất Vector đặc trưng (Feature Extraction)

Model `mobilefacenet.tflite` chạy qua TFLite Interpreter:

```
var output = List.filled(1 * 192, 0.0).reshape([1, 192]);
_interpreter!.run(input, output);
final embedding = (output[0] as List).cast<double>();
```

```
final norm = sqrt(embedding.fold(0.0, (s, v) => s + v * v));
return embedding.map((v) => v / (norm == 0 ? 1 : norm)).toList();
```

Output là một vector 192 số thực. Sau đó vector được chuẩn hóa L2 (L2 Normalization):

Việc chuẩn hóa L2 đảm bảo vector có độ dài = 1, giúp tính Cosine Similarity chính xác hơn.

4.2.7 Bước 6 – So khớp khuôn mặt (Face Matching)

a) Tại Mobile (Flutter – face_id_service.dart):

```
double _cosineSimilarity(List<double> a, List<double> b) {  
    double dot = 0, normA = 0, normB = 0;  
  
    for (int i = 0; i < a.length; i++) {  
        dot += a[i] * b[i];  
  
        normA += a[i] * a[i];  
        normB += b[i] * b[i];  
    }  
  
    return dot / (sqrt(normA) * sqrt(normB));  
}
```

b) Chiến lược so khớp đa mẫu:

```
FaceMatchResult compareFaces(List<double> curr,  
                             List<List<double>> savedList) {  
    double maxSimilarity = -1.0;  
  
    for (var saved in savedList) {  
        double sim = _cosineSimilarity(curr, saved);  
        if (sim > maxSimilarity) maxSimilarity = sim;  
    }  
  
    return FaceMatchResult(  
        isMatch: maxSimilarity >= 0.75, // Ngưỡng chấp nhận  
        confidence: (maxSimilarity * 100).clamp(0, 100),  
    );  
}
```

Hệ thống so sánh vector hiện tại với tất cả mẫu đã đăng ký (tối đa 5 góc: center, left, right, up, down) và lấy giá trị cao nhất. Ngưỡng 0.75 được chọn sau thực nghiệm để cân bằng giữa bảo mật (FAR thấp) và trải nghiệm (FRR thấp).

c) Tại Server (Node.js – index.js, dòng 116-128):

Server có triển khai song song cả euclideanDistance và cosineSimilarity. Hiện tại hệ thống dùng `MATCH_THRESHOLD = 0.75` với Cosine Similarity để đảm bảo tính nhất quán với Mobile.

4.3 Hệ thống trợ lý ảo WorkMate Assistant (Hybrid Rule-based)

Hệ thống tích hợp một trợ lý ảo hoạt động theo cơ chế Hybrid Rule-based (kết hợp quy tắc và truy vấn dữ liệu động), được xử lý tập trung tại module `workmate_backend/hybrid_chatbot.js`.

Quy trình xử lý bao gồm các bước:

- **Nhận diện ý định (Intent Recognition):** Sử dụng bộ lọc từ khóa thông minh (Keywords) để phân loại yêu cầu của người dùng vào các nhóm chức năng cốt lõi như: tra cứu chuyên cần, quản lý đơn từ hoặc kiểm tra thâm niên.
- **Truy vấn dữ liệu thời gian thực:** Sau khi xác định được ý định, hệ thống thực hiện các câu lệnh SQL truy vấn trực tiếp vào cơ sở dữ liệu PostgreSQL dựa trên định danh (ID) của nhân viên đang hội thoại, đảm bảo dữ liệu luôn được cập nhật mới nhất.
- **Cá nhân hóa phản hồi:** Các thông số thực tế từ CSDL (như số lần đi muộn, ngày phép còn lại) được lồng ghép linh hoạt vào nội dung phản hồi, giúp thông tin trở nên gần gũi và hữu ích với từng cá nhân.

4.3.1 Kiến trúc Context-Aware Chatbot

Thay vì sử dụng các mô hình ngôn ngữ lớn (LLM) đòi hỏi chi phí vận hành và tài nguyên phần cứng cao, đồng thời tiềm ẩn rủi ro trả lời sai lệch về dữ liệu nội bộ ("ảo giác AI"), hệ thống WorkMate được thiết kế theo mô hình **Hybrid Rule-Based**.

Trong kiến trúc này, lớp Backend đóng vai trò là một "Context Orchestrator" — chịu trách nhiệm phân tích ngữ cảnh và ý định của người dùng, sau đó tự động điều phối

việc truy xuất dữ liệu nhân sự thực tế. Giải pháp này giúp hệ thống đạt được ba mục tiêu trọng tâm:

1. **Chính xác tuyệt đối (100%):** Loại bỏ sai số do dữ liệu được lấy trực tiếp từ nguồn CSDL chính thống.
2. **Tốc độ tối ưu:** Phản hồi gần như tức thì do không mất thời gian xử lý qua các API bên thứ ba.
3. **Tiết kiệm tài nguyên:** Hệ thống có thể vận hành ổn định trên các hạ tầng máy chủ phổ thông mà không cần đến tài nguyên GPU chuyên dụng.

4.3.2 Luồng xử lý chi tiết (File: *hybrid_chatbot.js*)

- **Bước 1 – Nhận yêu cầu:**

Mobile App gửi tin nhắn qua kết nối thời gian thực Socket.io. Server tiếp nhận và đưa vào phương thức `processMessage(userId, message)` của lớp `HybridChatbot`.

- **Bước 2 – Phân tích Ý định (Intent Detection):**

Tin nhắn được chuẩn hóa (đưa về chữ thường, cắt khoảng trắng) và đi qua bộ nhận diện Intent. Hệ thống đối chiếu chuỗi tin nhắn với cấu hình từ khóa (Ví dụ: 'điểm danh', 'vào ca chưa' → Intent: `attendance_today`).

- **Bước 3 – Thu thập Context từ Database:**

Khi Intent được xác định, hệ thống gọi hàm Handler tương ứng (Ví dụ: `handleAttendanceToday`). Backend lập tức truy vấn PostgreSQL dựa trên `userId` hiện tại để lấy dữ liệu thực tế (Ví dụ: `SELECT check_in_time FROM attendance WHERE employee_id = $1...`).

- **Bước 4 – Xây dựng phản hồi động & Gợi ý (Suggestions):**

Dữ liệu thô từ Database được hệ thống "nhúng" (inject) vào các template câu trả lời thân thiện (Ví dụ: "Tháng này bạn đi trễ 2 lần"). Đồng thời, hệ thống tự động sinh ra danh sách suggestions (Gợi ý câu hỏi tiếp theo) để điều hướng thao tác của người dùng.

- **Bước 5 – Lưu lịch sử và gửi kết quả:**

Phản hồi của Bot được trả về Mobile App qua Socket.io. Đồng thời, tin nhắn được lưu vào bảng `chat_messages` với trường `chat_type = 'ai'` để duy trì lịch sử hội thoại cá nhân.

- **Bước 6 – Cơ chế Chuyển tiếp linh hoạt (Fallback Mechanism):**

Backend tích hợp bộ đếm unknownCount cho từng userId. Nếu Bot tự nhận thấy không thể giải quyết yêu cầu (không khớp Intent) 2 lần liên tiếp, hệ thống sẽ trả về cờ action: 'open_admin_chat'. Cờ này kích hoạt Mobile App tự động hiển thị nút "Kết nối Admin", giúp luân chuyển mượt mà từ máy móc sang con người khi cần thiết.

4.4 Hệ thống Chat Realtime (Socket.IO)

4.4.1 Sơ đồ sự kiện

Bảng 4.1 Hệ thống định nghĩa 4 sự kiện Socket.IO chính

(index.js, dòng 37-77):

Sự kiện	Chiều	Mô tả
connection	Server ← Client	Client kết nối, khởi tạo tunnel
register	Server ← Client	Client gửi userId để server ánh xạ socketId ↔ userId
send_message	Server ← Client	Gửi tin nhắn, server lưu DB và emit ngay
online_users	Server → All	Broadcast danh sách userId đang online
receive_message {userId}	Server → Client	Tin nhắn riêng cho từng user
receive message admin	Server → All Admin	Tin nhắn gửi cho toàn bộ Admin đang online
disconnect	Server ← Client	Xóa khỏi onlineUsers, cập nhật trạng thái

4.4.2 Cơ chế Routing tin nhắn:

```
if (receiver_id) {  
  io.emit(`receive_message_${receiver_id}`, newMessage); // → Nhân viên cụ thể  
} else {  
  io.emit('receive_message_admin', newMessage); // → Toàn bộ Admin  
}  
  
io.emit(`receive_message_${sender_id}`, newMessage); // → Ngffời gửi (echo)
```

4.4.3 Thuật toán tính điểm thâm niên và cống hiến

Dựa trên mã nguồn `seed_seniority.js`, hệ thống tự động tính toán điểm thưởng:

- Cơ chế: Dựa vào khoảng cách từ `join_date` đến ngày hiện tại.
- Ứng dụng: Điểm được cập nhật vào trường `seniority_bonus` để Admin dùng làm căn cứ xét khen thưởng và tăng bậc phúc lợi cho nhân viên.

4.5 Thuật toán tính điểm thâm niên và cống hiến (Seniority Points)

Hệ thống xây dựng tính năng tính điểm thâm niên tự động nhằm tạo động lực mạnh mẽ và ghi nhận chính xác sự gắn bó, đóng góp dài hạn của nhân viên đối với tổ chức.

4.5.1 Cơ chế và Dữ liệu nền tảng

- Dữ liệu đầu vào: Thuật toán sử dụng dữ liệu gốc từ trường `join_date` (ngày vào làm) trong bảng `employees` để đảm bảo tính khách quan.
- Đơn vị tính toán: Hệ thống tự động quét và đối chiếu khoảng thời gian làm việc thực tế của nhân viên tính từ ngày bắt đầu cho đến thời điểm hiện tại.

4.5.2 Quy trình vận hành hệ thống

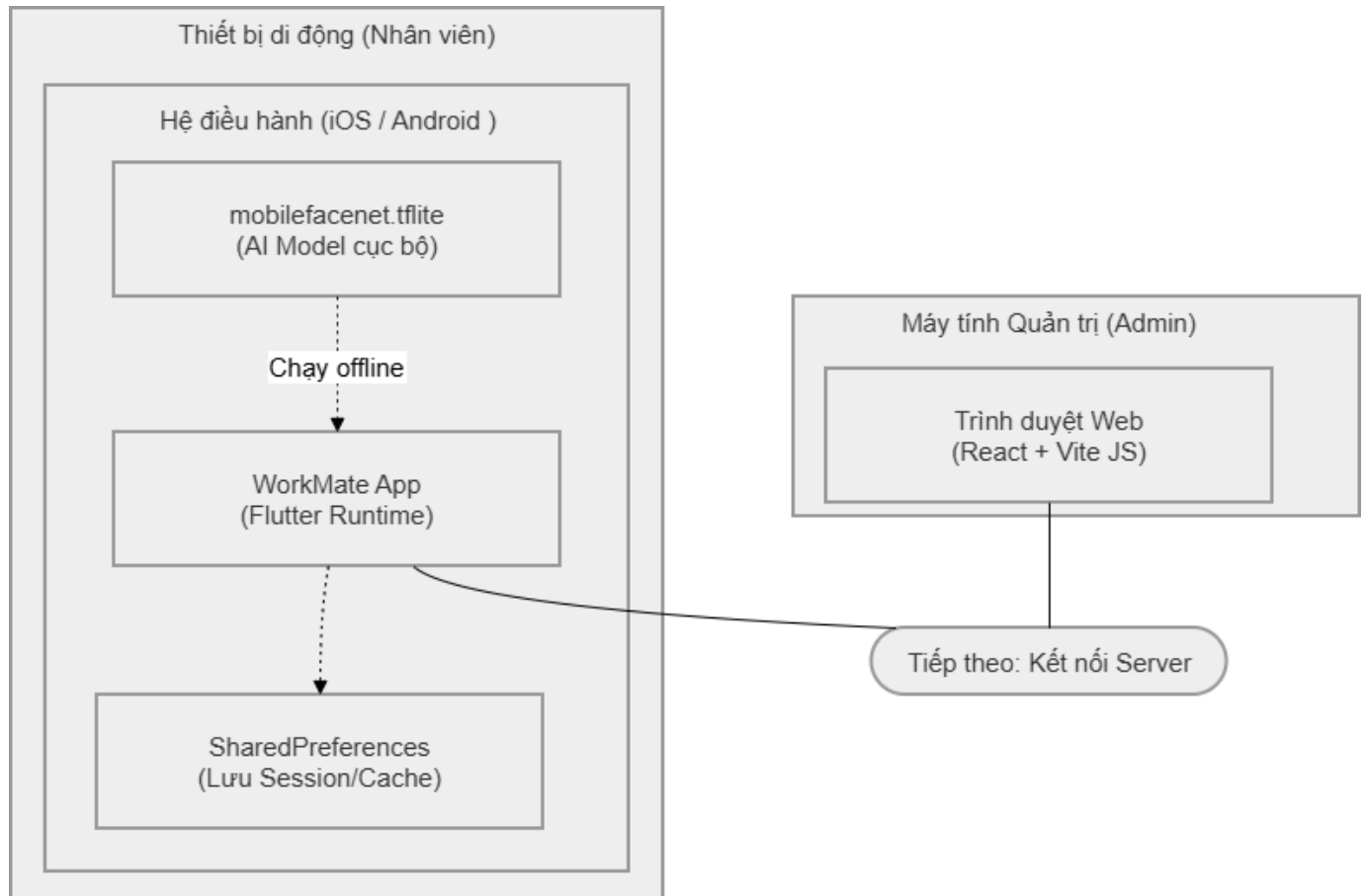
- **Xử lý ngầm (Backend Scripts):** Việc tính toán được thực hiện thông qua các script chuyên dụng như `seed_seniority.js` và `migrate_seniority.js`. Các script

này giúp khởi tạo và cập nhật dữ liệu hàng loạt mà không làm ảnh hưởng đến hiệu suất vận hành trực tiếp của ứng dụng.

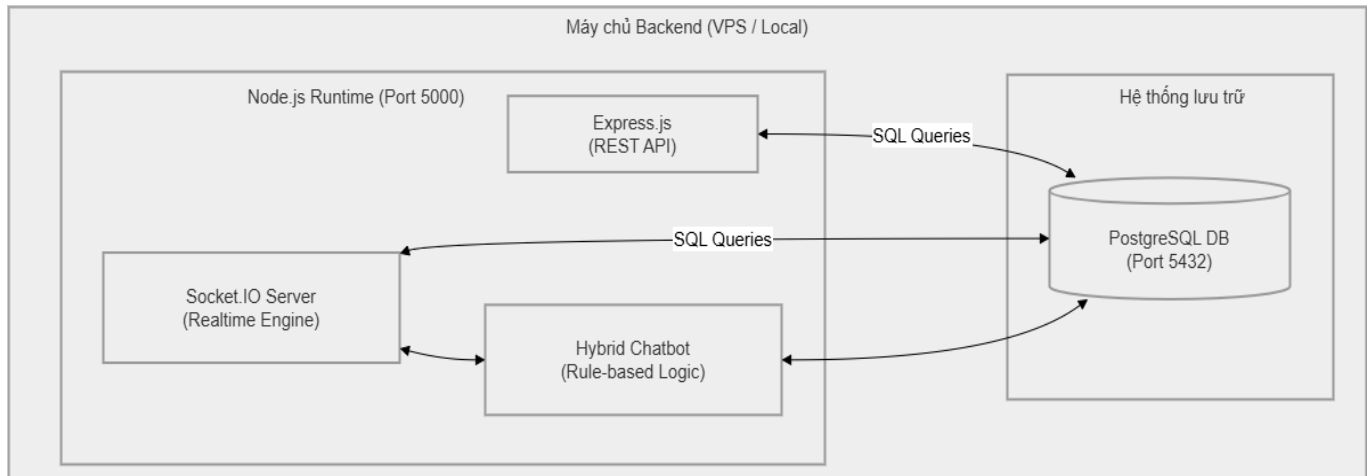
- **Cập nhật dữ liệu:** Kết quả sau khi tính toán được tự động đồng bộ vào trường **seniority_bonus** trong cơ sở dữ liệu.
- **Hiển thị và Ứng dụng:**
 - **Giao diện cá nhân:** Thông tin điểm thưởng được hiển thị công khai trên Profile của nhân viên, giúp tăng tính minh bạch.
 - **Giá trị sử dụng:** Điểm thâm niên là cơ sở dữ liệu quan trọng để hệ thống tự động phân loại thứ hạng (Ranking) và áp dụng các chính sách phúc lợi, khen thưởng tương ứng của công ty.

CHƯƠNG 5: TRIỂN KHAI PHẦN MỀM VÀ DEVOPS CHI TIẾT

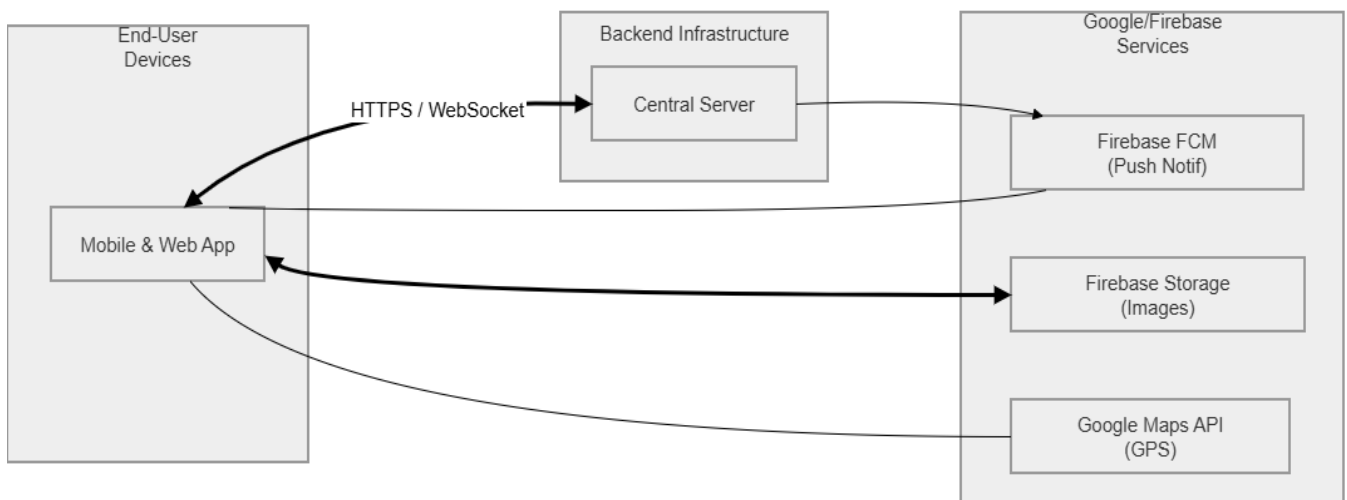
5.1 Tổng thể kiến trúc Backend (Node.js + Express)



Hình 5.1a: Sơ đồ triển khai hạ tầng tại các thiết bị đầu cuối (Client-side).



Hình 5.1b: Sơ đồ triển khai hệ thống máy chủ và cơ sở dữ liệu.



Hình 5.1c: Sơ đồ luồng kết nối mạng và tích hợp các dịch vụ đám mây.

* Hệ thống WorkMate được triển khai dựa trên mô hình Client-Server kết hợp với các dịch vụ đám mây (Cloud Computing) để tối ưu hóa hiệu năng và độ ổn định. Chi tiết hạ tầng hệ thống (Hình 5.1) được phân tích qua ba góc độ: các thành phần chạy tại thiết bị người dùng (Hình 5.1a), cấu trúc vận hành tại máy chủ trung tâm (Hình 5.1b) và mô hình tích hợp dịch vụ bên thứ ba (Hình 5.1c).

5.1.1 Cấu trúc server khởi động

```
const app      = express();
const server = http.createServer(app);           // HTTP server
const io       = new Server(server, { cors: { origin: "*" } }); // Socket.IO
server.listen(5000, '0.0.0.0', () => { ... });   // Lắng nghe trên mọi interface
```

Server khởi động qua `initDB()`— hàm tự động migration, kiểm tra và bổ sung các cột mới vào CSDL mà không xóa dữ liệu hiện có (`ADD COLUMN IF NOT EXISTS`).

5.1.2 Danh sách đầy đủ các Middleware

Bảng 5.1: Danh sách các Middleware trong kiến trúc Backend.

Middleware	Mục đích
<code>cors()</code>	Cho phép gọi API từ mọi origin (Web Admin + Mobile)
<code>express.json()</code>	Parse JSON body từ request
<code>multer</code>	Xử lý upload file ảnh
<code>dotenv.config()</code>	Nạp biến môi trường từ <code>.env</code>

5.1.3: Cơ chế tự động khởi tạo và cập nhật CSDL (Auto-migration)

Để đảm bảo tính nhất quán của dữ liệu trong quá trình phát triển và triển khai thực tế, Backend được thiết lập cơ chế tự động quản lý cấu trúc bảng (`index.js`).

- Khởi tạo thông minh: Tại hàm `initDB()`, hệ thống kiểm tra sự tồn tại của các bảng và cột dữ liệu trước khi thực hiện các tác vụ khởi chạy.
- Cập nhật không gián đoạn: Sử dụng các câu lệnh `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`, hệ thống có thể bổ sung các tính năng mới (như thông tin tài khoản ngân hàng, tọa độ an toàn mới) mà không làm ảnh hưởng đến dữ liệu hiện có hoặc yêu cầu can thiệp thủ công vào hệ quản trị CSDL.

5.2 Đặc tả REST API đầy đủ (API Documentation)

5.2.1 Nhóm API Xác thực (Auth)

- POST /api/auth/login

- **Body:** { code: "NV001", password: "abc123" }
- **Logic:** So khớp employee_code HOẶC email. Dùng bcrypt.compare() để xác minh mật khẩu.
- **Response 200:** { user: { id, name, role, department_id, ... } } (không có password_hash)
- **Response 401:** { message: "Sai mật khẩu" }
- **Response 404:** { message: "Không tìm thấy người dùng" }

- POST /api/auth/forgot-password

- **Body:** { email: "user@example.com" }
- **Logic:** Tạo mật khẩu ngẫu nhiên 8 ký tự, băm BCrypt, lưu vào DB, gửi email qua Nodemailer/Gmail.
- **Response 200:** { success: true, message: "Mật khẩu mới đã được gửi vào email" }

- POST /api/auth/send-otp & POST /api/auth/change-password

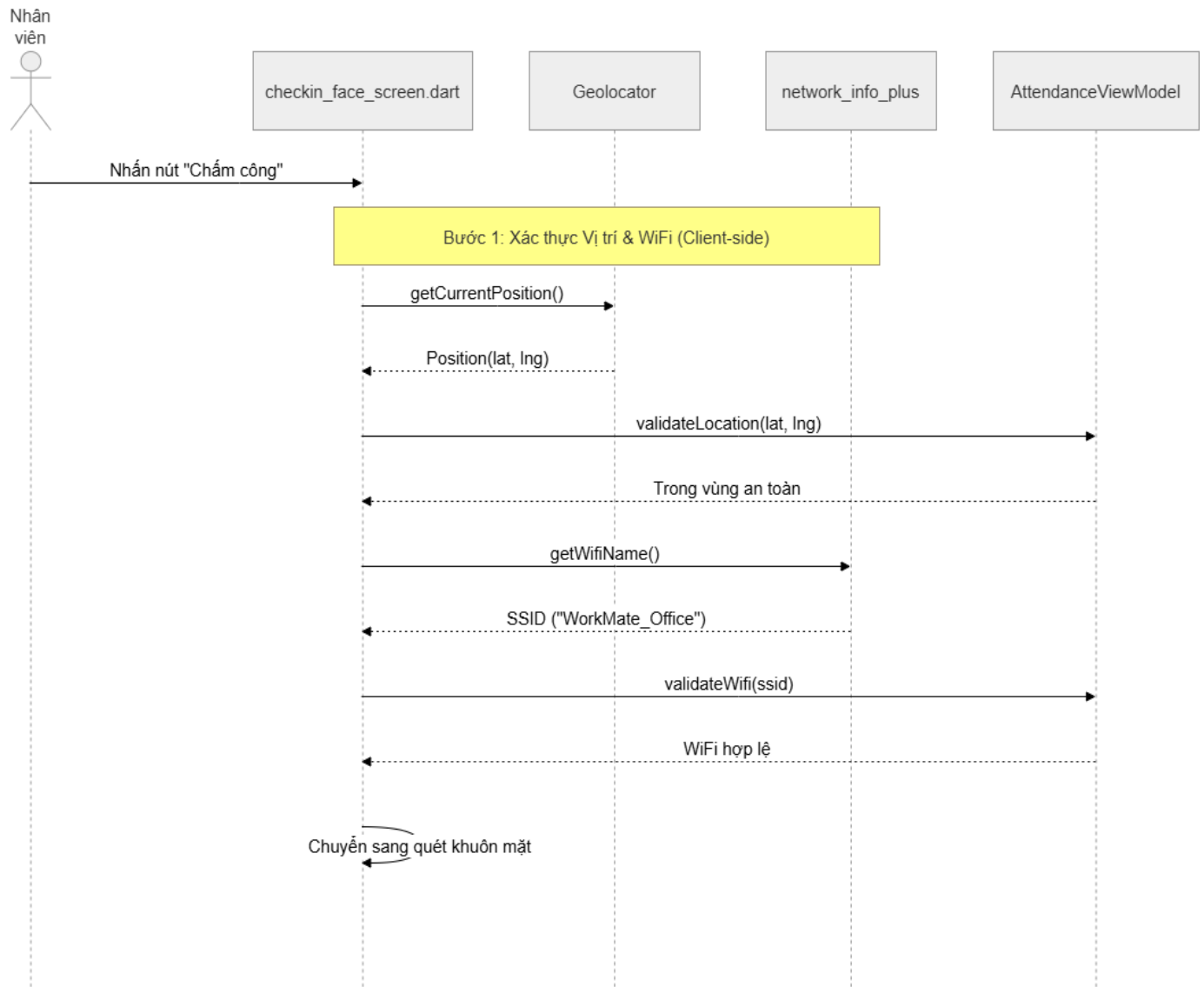
- OTP 6 số được lưu trong Map<> nội bộ, có thời hạn 5 phút. Sau khi dùng, OTP bị xóa ngay.

5.2.2 Nhóm API Chấm công FaceID

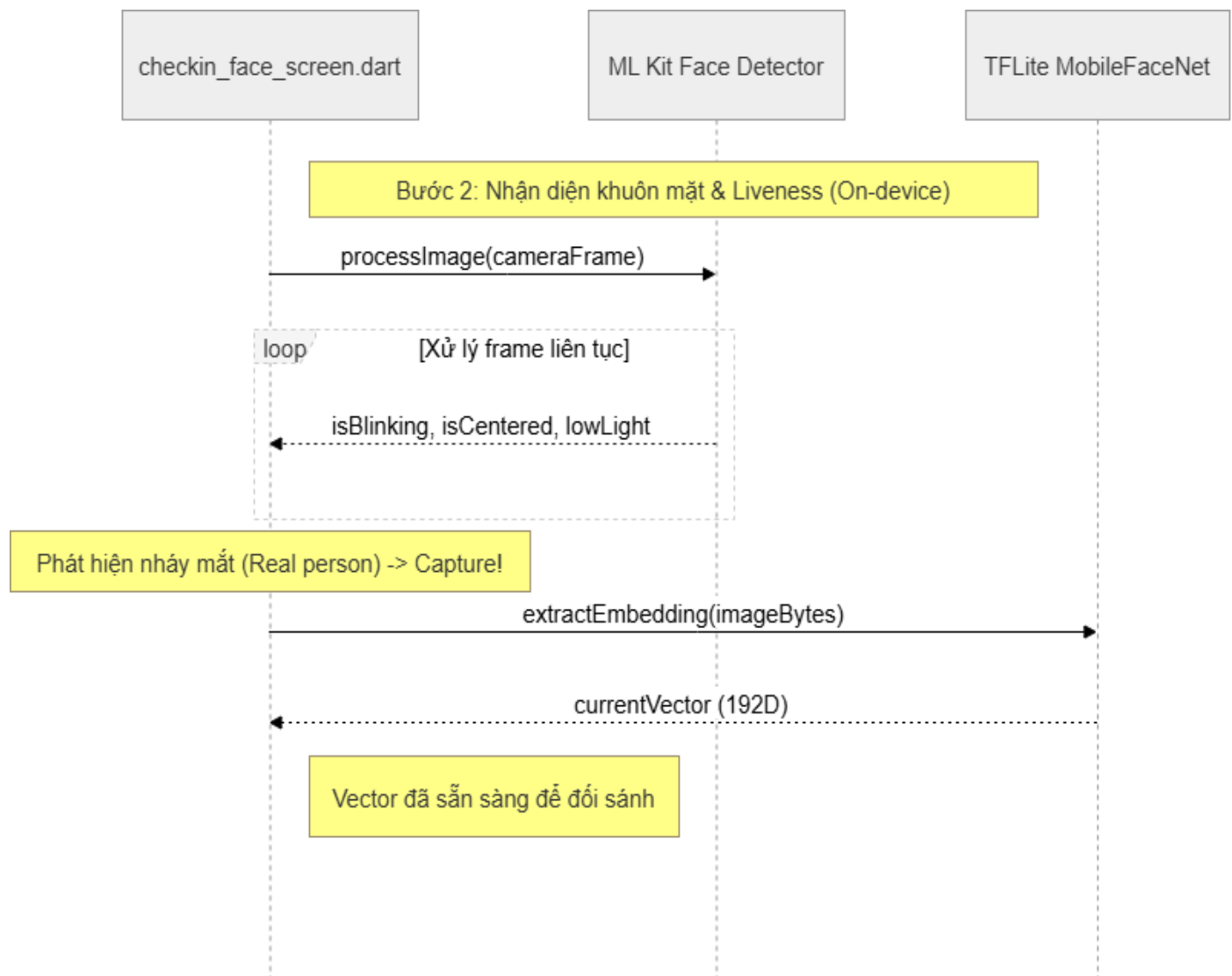
- POST /api/face/register

- **Mục đích:** Đăng ký khuôn mặt nhân viên (lần đầu hoặc cập nhật).
 - **Body:** { employee_id: 1, embeddings: [...192 số...], [...], [...]} }
 - **Logic:**
 1. Xóa toàn bộ embedding cũ của nhân viên.
 2. Lưu lần lượt từng embedding với nhãn góc: center, left, right, up, down.
 3. Cập nhật face_registered_at = NOW() trong bảng employees.
 - **Response 200:** { success: true, message: "Đăng ký thành công" }
- #### - GET /api/face/embedding/:id

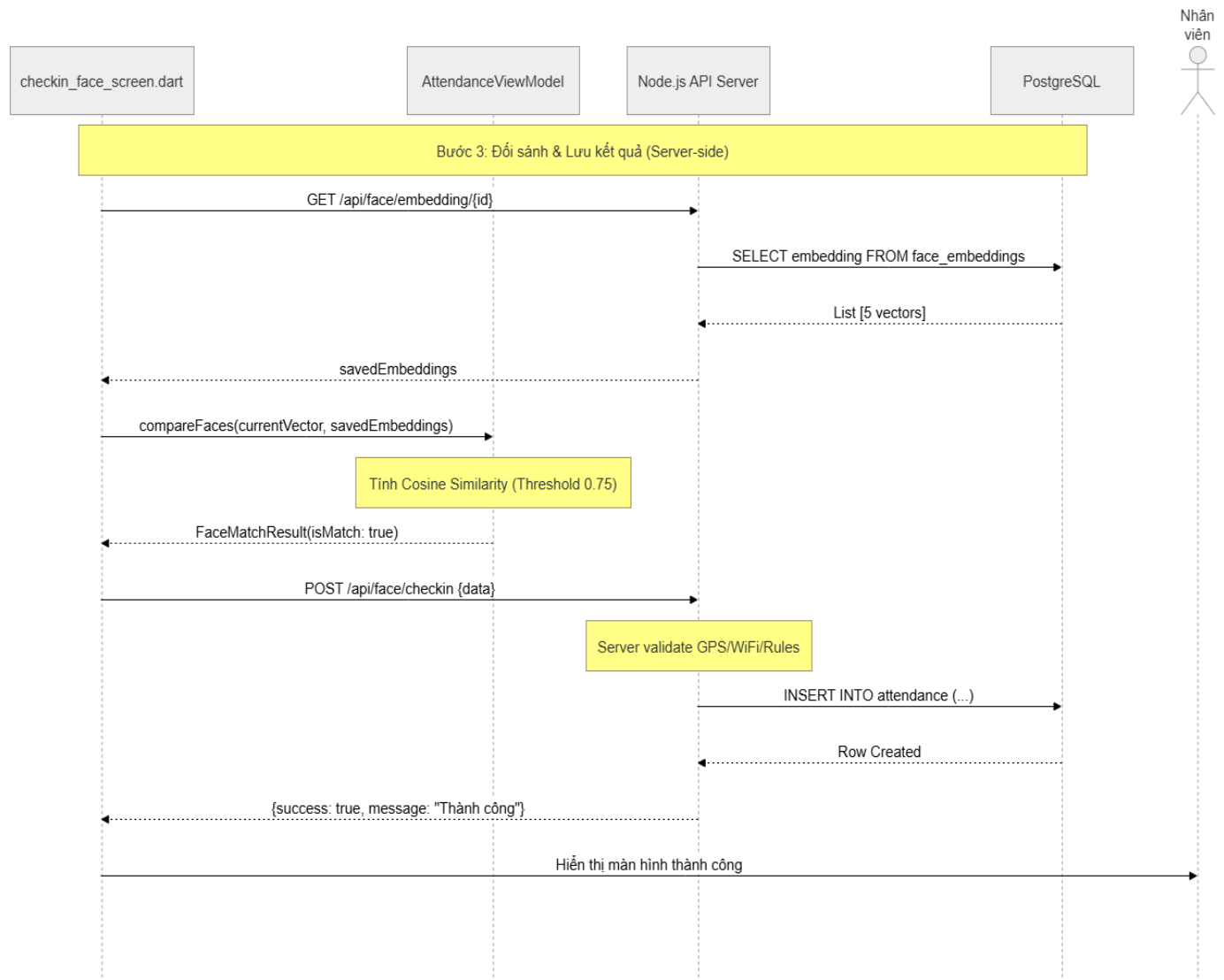
- **Mục đích:** Lấy danh sách embedding đã lưu để so khớp tại client.
- **Response:** { **embedding:** [...], **embeddings:** [[...], [...], [...]] }
- **Lưu ý:** Trả về cả **embedding** (tương thích ngược) và **embeddings** (mảng đa mẫu).
- **POST /api/face/checkin**
 - **Mục đích:** Thực hiện chấm công sau khi nhận diện khuôn mặt thành công.
 - **Body:** { **employee_id**, **embedding**, **lat**, **lng**, **wifi_ssid**, **action** }
 - **action:** "check_in" hoặc "check_out"
 - Logic xử lý tuần tự:
 1. Truy vấn **company_config**.
 2. Kiểm tra WiFi (nếu có cấu hình).
 3. Kiểm tra GPS với **getDistance()**.
 4. Tải tất cả embedding của nhân viên từ DB.
 5. So khớp với **cosineSimilarity()**, lấy **maxSimilarity**.
 6. Nếu **maxSimilarity** < 0.75 → từ chối.
 7. Kiểm tra đã check-in/out chưa, ngăn check-out < 1 phút sau check-in.
 8. Ghi bản ghi, emit Socket **new_attendance** hoặc **attendance_updated**.
 - **Response 200:** { **success:** true, **message:** "Thao tác thành công" }
 - **Response 403:** { **message:** "Khuôn mặt không khớp" } hoặc lỗi vị trí/WiFi.



Hình 5.2a: Sơ đồ tuần tự – Quy trình xác thực vị trí và điểm truy cập mạng.



Hình 5.2b: Sơ đồ tuần tự – Quy trình nhận diện khuôn mặt và trích xuất đặc trưng sinh trắc học.



Hình 5.2c: Sơ đồ tuần tự – Quy trình gọi API đối sánh và lưu bản ghi chấm công.

** Sự tương tác giữa các thành phần trong quá trình chấm công được mô tả chi tiết qua sơ đồ tuần tự tại Hình 5.2. Quy trình được thiết kế theo mô hình lai (Hybrid): bắt đầu từ việc kiểm tra điều kiện môi trường tại thiết bị (Hình 5.2a), tiếp nối bằng việc xử lý AI trực tiếp trên phần cứng Mobile (Hình 5.2b) và kết thúc bằng các thủ tục xác thực, lưu trữ dữ liệu tại hệ thống Backend (Hình 5.2c).*

5.2.3 Nhóm API Đơn từ (Approvals)

- GET /api/approvals?employee_id={id}

- Trả về lịch sử tất cả đơn của một nhân viên.

- **POST /api/approvals**
 - **Body:** { employee_id, name, type, reason, from_date, to_date, attachment_urls }
 - **type:** "Nghỉ phép", "Tăng ca", "Bổ sung công"
- **PUT /api/approvals/:id/approve & /reject**
 - Admin phê duyệt/từ chối đơn.
 - Sau khi cập nhật DB, hệ thống gọi Firebase Admin SDK để gửi Push Notification tới FCM token của nhân viên.

5.2.4 Nhóm API Chat & AI

- **GET /api/chat/history/:userId**
 - Trả về toàn bộ lịch sử tin nhắn của nhân viên (cả AI và Admin), sắp xếp theo thời gian tăng dần.
- **POST /api/chat/ai**
 - **Body:** { userId: 1, message: "Tôi còn bao nhiêu ngày phép?" }
 - **Response:** { reply: "Bạn còn 12 ngày phép năm...", suggestAdmin: false }
- **GET /api/chat/admin/conversations**
 - Dành cho Web Admin: lấy danh sách nhân viên đã từng nhắn tin, kèm tin nhắn cuối cùng và thời gian.

5.3 Bảo mật hệ thống

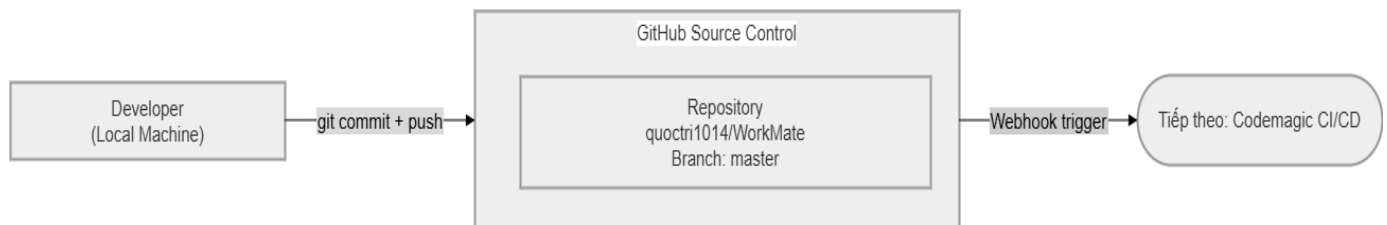
Bảng 5.2: Các biện pháp bảo mật hệ thống.

Biện pháp	Chi tiết triển khai
Mã hóa mật khẩu	BCrypt với saltRounds = 10. Không bao giờ lưu hoặc trả về mật khẩu thô.
Bảo vệ dữ liệu sinh trắc	Chỉ lưu vector số, không lưu ảnh khuôn mặt. Vector không thể tái tạo lại ảnh gốc.
Biến môi trường	Toàn bộ thông tin nhạy cảm (DB, API Key, Email) trong .env, không commit lên Git.
Chống Replay Attack	Mỗi OTP chỉ dùng 1 lần, tự hủy sau khi dùng thành công.
Chống check-out nhanh	Server từ chối check-out nếu thực hiện dưới 1 phút sau check-in.
CORS	Cấu hình cho phép tất cả origin (phù hợp môi trường phát triển, nên hạn chế khi production).

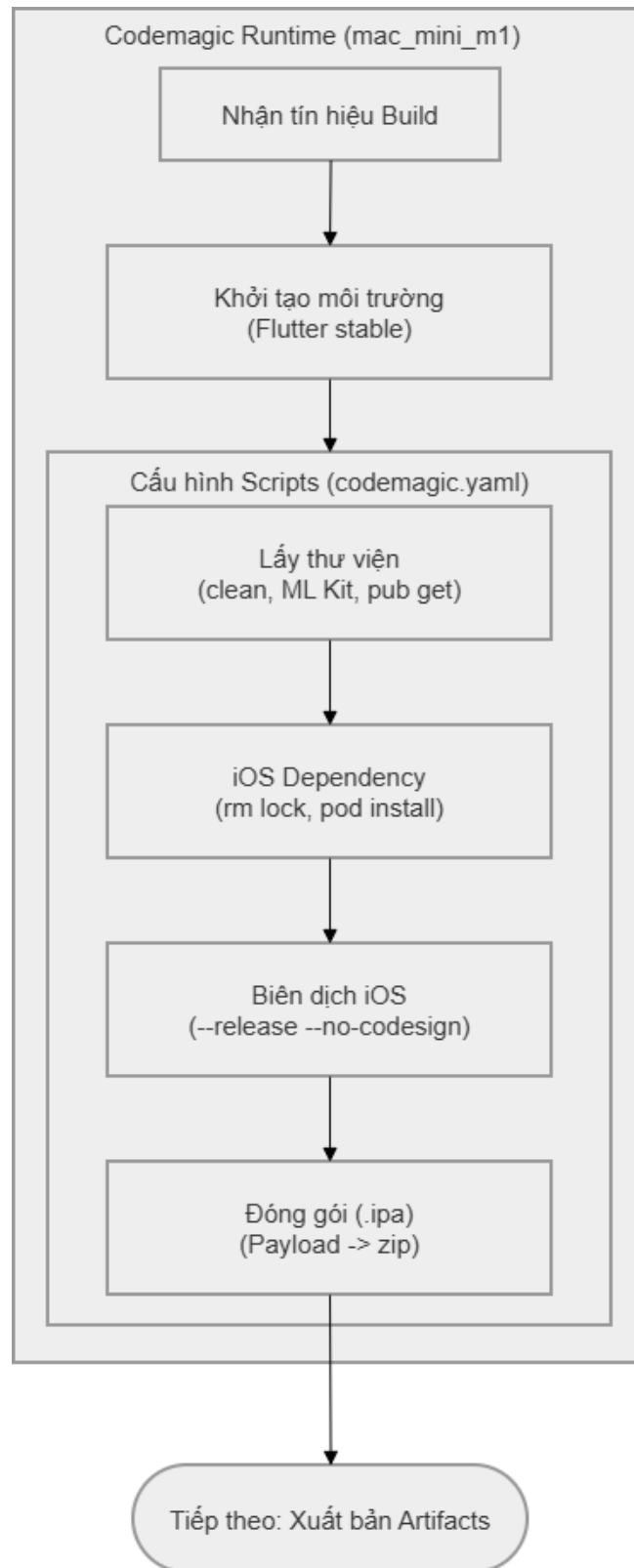
5.3.1: Bảo mật truy vấn và phòng chống SQL Injection

Do trợ lý ảo và các API có tương tác trực tiếp với người dùng qua nhập liệu văn bản, hệ thống áp dụng cơ chế Parameterized Queries (truy vấn có tham số). Mọi dữ liệu đầu vào từ ô chat hoặc các form đơn từ đều được xử lý thông qua bộ thư viện pg của **Node.js**, đảm bảo không có mã độc SQL nào có thể thực thi trái phép, bảo vệ toàn diện dữ liệu nhân sự nhạy cảm.

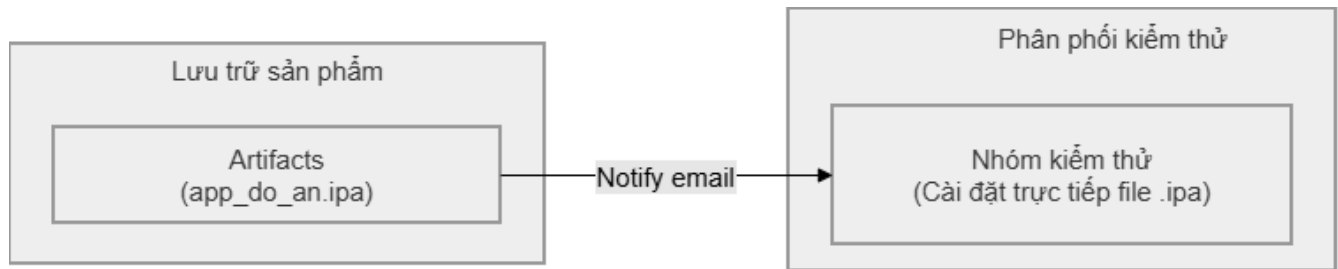
5.4 Quy trình CI/CD với Codemagic



Hình 5.3a: Quy trình đẩy mã nguồn và kích hoạt Webhook tự động.



Hình 5.3b: Quy trình thiết lập môi trường và thực thi kịch bản biên dịch app iOS.



Hình 5.3c: Quy trình đóng gói sản phẩm và phân phối tới nhóm kiểm thử.

** Để tối ưu hóa tốc độ triển khai và đảm bảo tính nhất quán của ứng dụng trên các môi trường, hệ thống WorkMate sử dụng quy trình CI/CD tự động thông qua Codemagic. Toàn bộ luồng triển khai (Hình 5.3) được chia làm ba giai đoạn: quản lý mã nguồn tại GitHub (Hình 5.3a), thực thi các kịch bản build iOS tại máy chủ Codemagic (Hình 5.3b) và cuối cùng là đóng gói, phân phối file .ipa trực tiếp tới nhóm kiểm thử (Hình 5.3c).*

5.4.1 Cấu hình build tự động

- **Trigger:** Mỗi commit được push lên nhánh **master** của GitHub tự động kích hoạt build.
- **Flutter SDK:** Codemagic tự cài đặt đúng phiên bản Flutter được chỉ định trong **pubspec.yaml**.
- **iOS Build:** Chạy **flutter build ios --release**, sau đó ký số với Apple Certificate và Provisioning Profile đã cấu hình trong dashboard Codemagic.
- **Android Build:** Chạy **flutter build apk --release** với keystore file đã cấu hình.

5.4.2 Các bước thực thi trên Codemagic

1. **Checkout code** từ GitHub repository.
2. **Install dependencies:** **flutter pub get** + **pod install**(iOS).
3. **Code analysis:** **flutter analyze**— phát hiện cảnh báo và lỗi cú pháp.
4. **Build iOS:** Compile, Code Sign → xuất file **.ipa**.
5. **Build Android:** Compile, Sign → xuất file **.apk**.
6. **Notify:** Gửi email kết quả build (thành công/thất bại) với link tải artifact.

5.4.3 Lý do chọn Codemagic

- Hỗ trợ native Flutter, không cần cấu hình phức tạp.
- Có sẵn máy ảo macOS để build iOS mà không cần mua MacBook riêng.

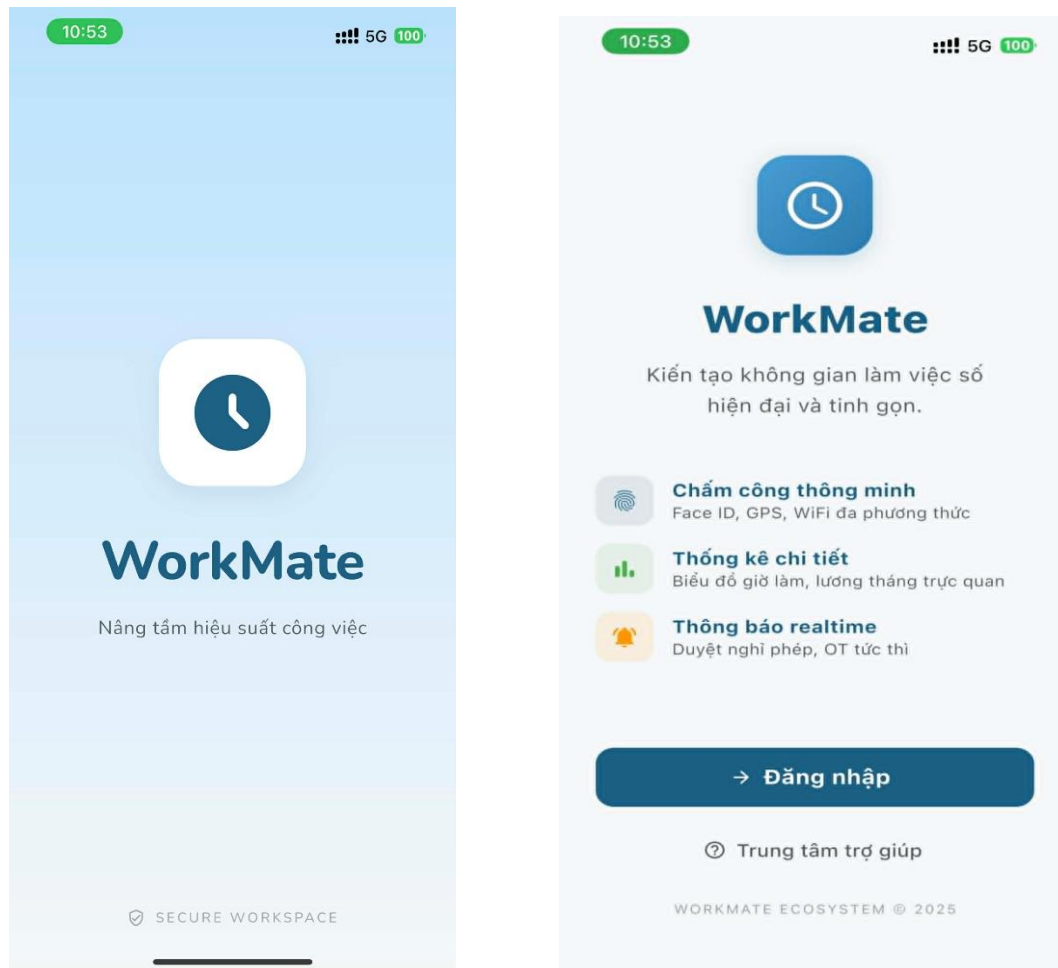
- Giao diện quản lý đơn giản, phù hợp với nhóm sinh viên.
- Hỗ trợ miễn phí 500 phút build/tháng.

CHƯƠNG 6: MINH HỌA GIAO DIỆN HỆ THỐNG

Dưới đây là danh sách các màn hình giao diện quan trọng của hệ thống WorkMate. Bạn nên chụp ảnh màn hình thực tế và dán vào dưới mỗi mục mô tả.

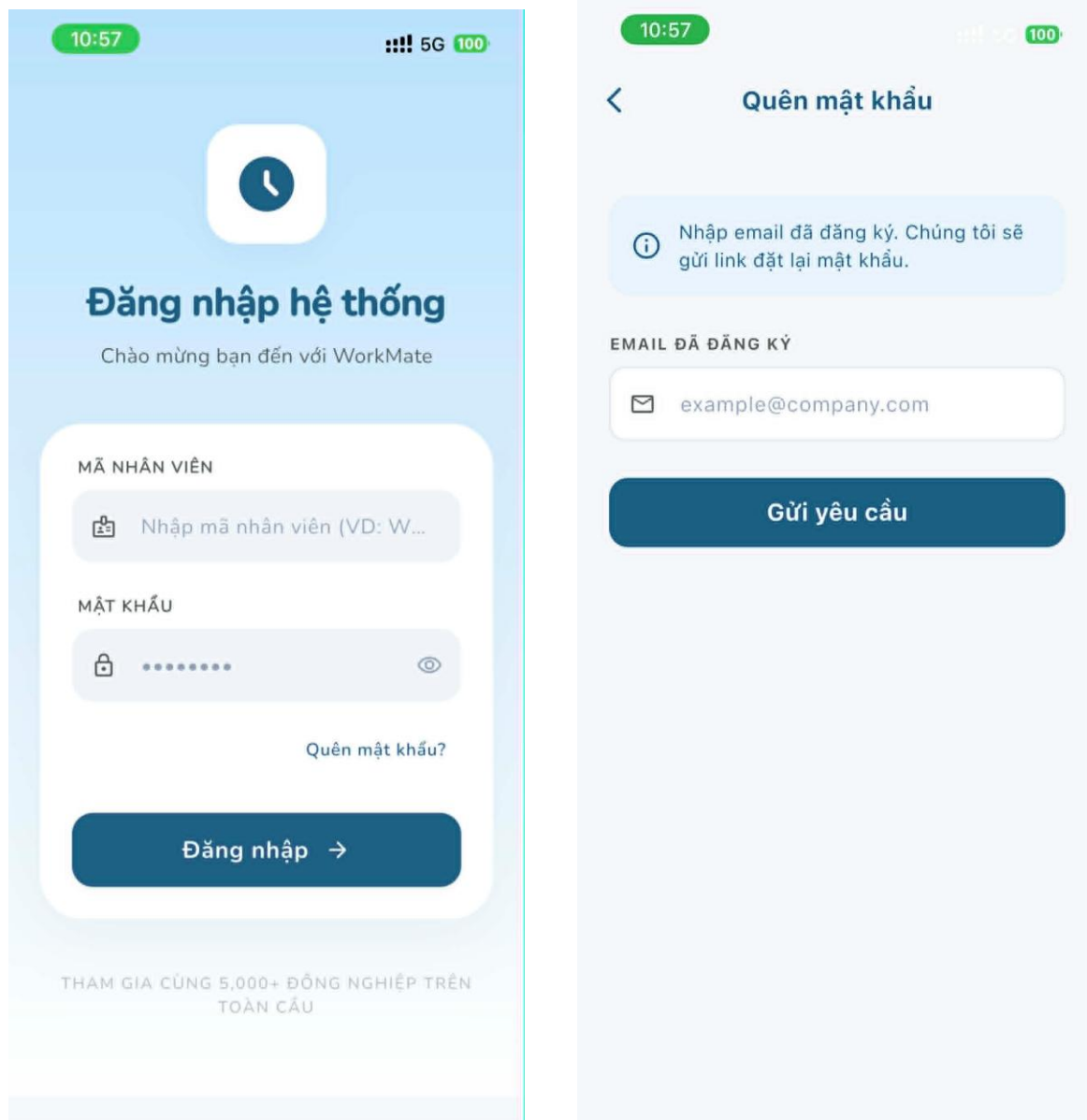
6.1 Giao diện dành cho Nhân viên (Mobile App)

6.1.1 Nhóm màn hình Khởi đầu & Đăng nhập



Hình 6.1: Màn hình Splash & Onboarding

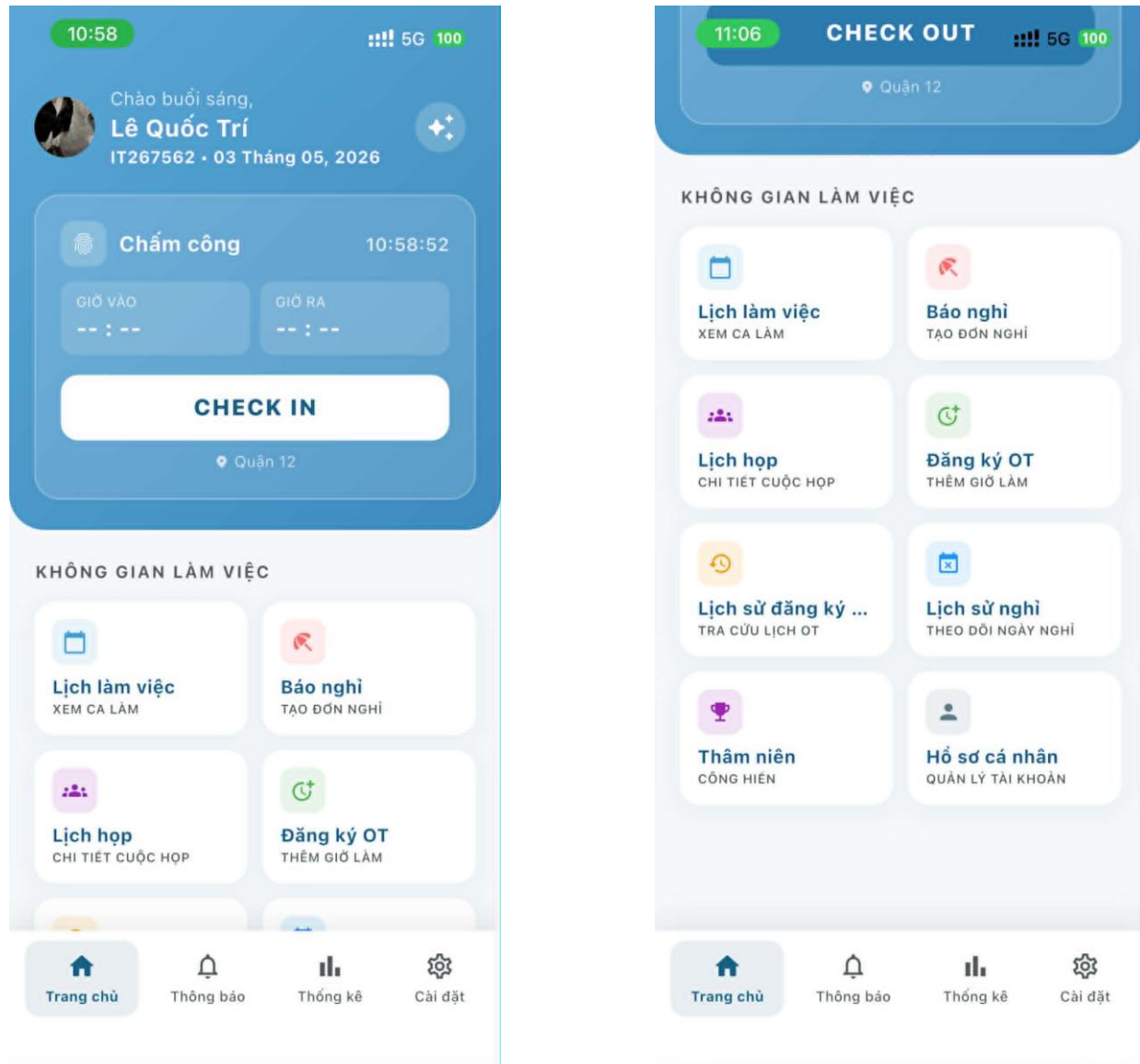
Mô tả: Hiển thị logo WorkMate và các slide giới thiệu tính năng (Chăm công FaceID, Quản lý đơn từ).



Hình 6.2: Màn hình Đăng nhập & Quên mật khẩu

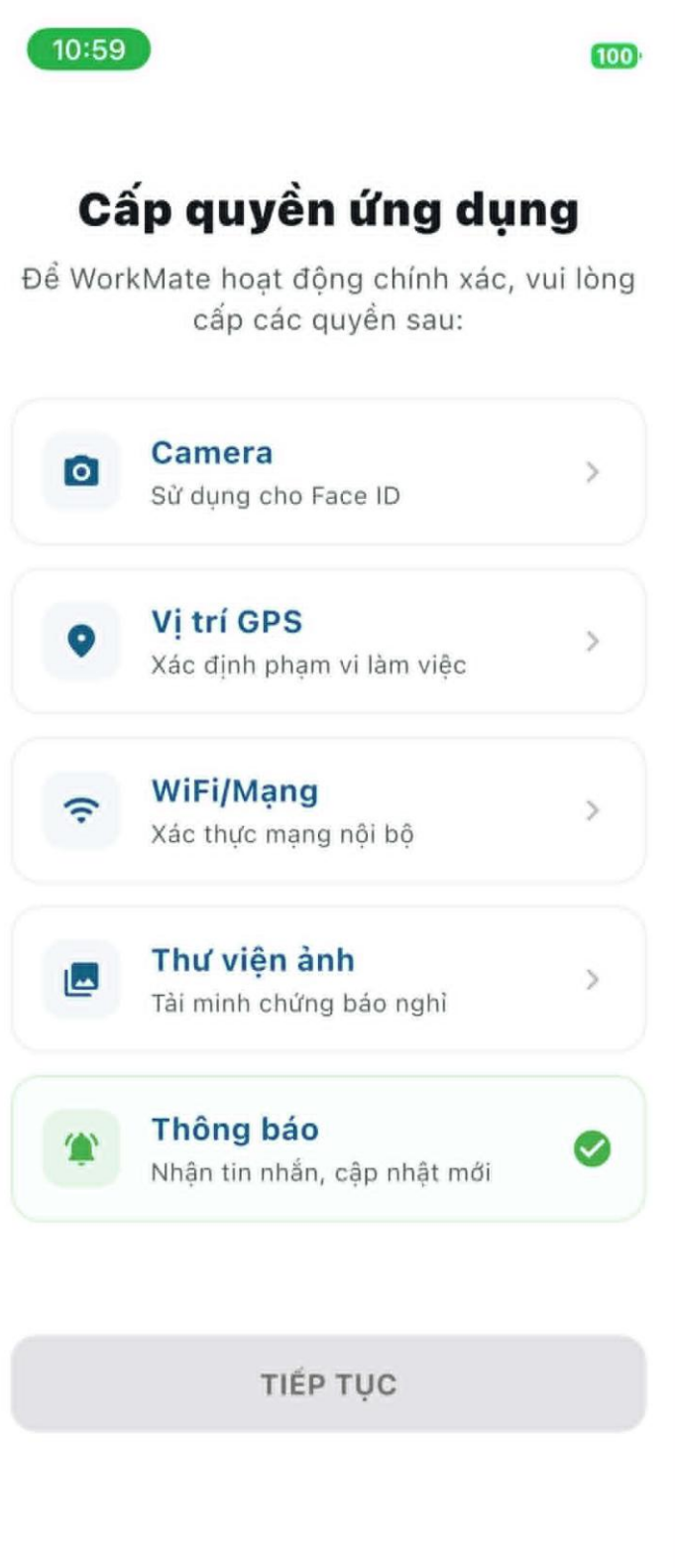
Mô tả: Giao diện nhập Email/Mật khẩu với thiết kế hiện đại, hỗ trợ ẩn/hiện mật khẩu.

6.1.2 Nhóm màn hình Chấm công (Tính năng cốt lõi)



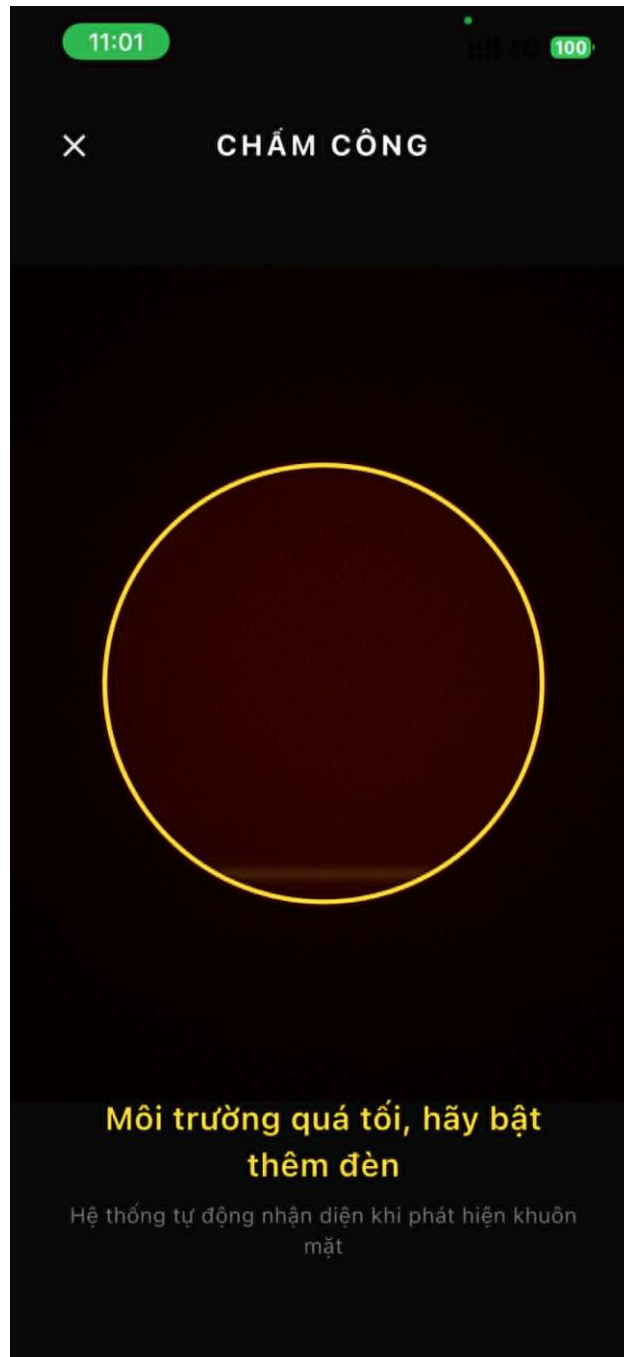
Hình 6.3: Màn hình chính (Dashboard)

Mô tả: Hiển thị thông tin nhân viên, trạng thái chấm công hôm nay, các phím tắt chức năng và danh sách lịch họp.



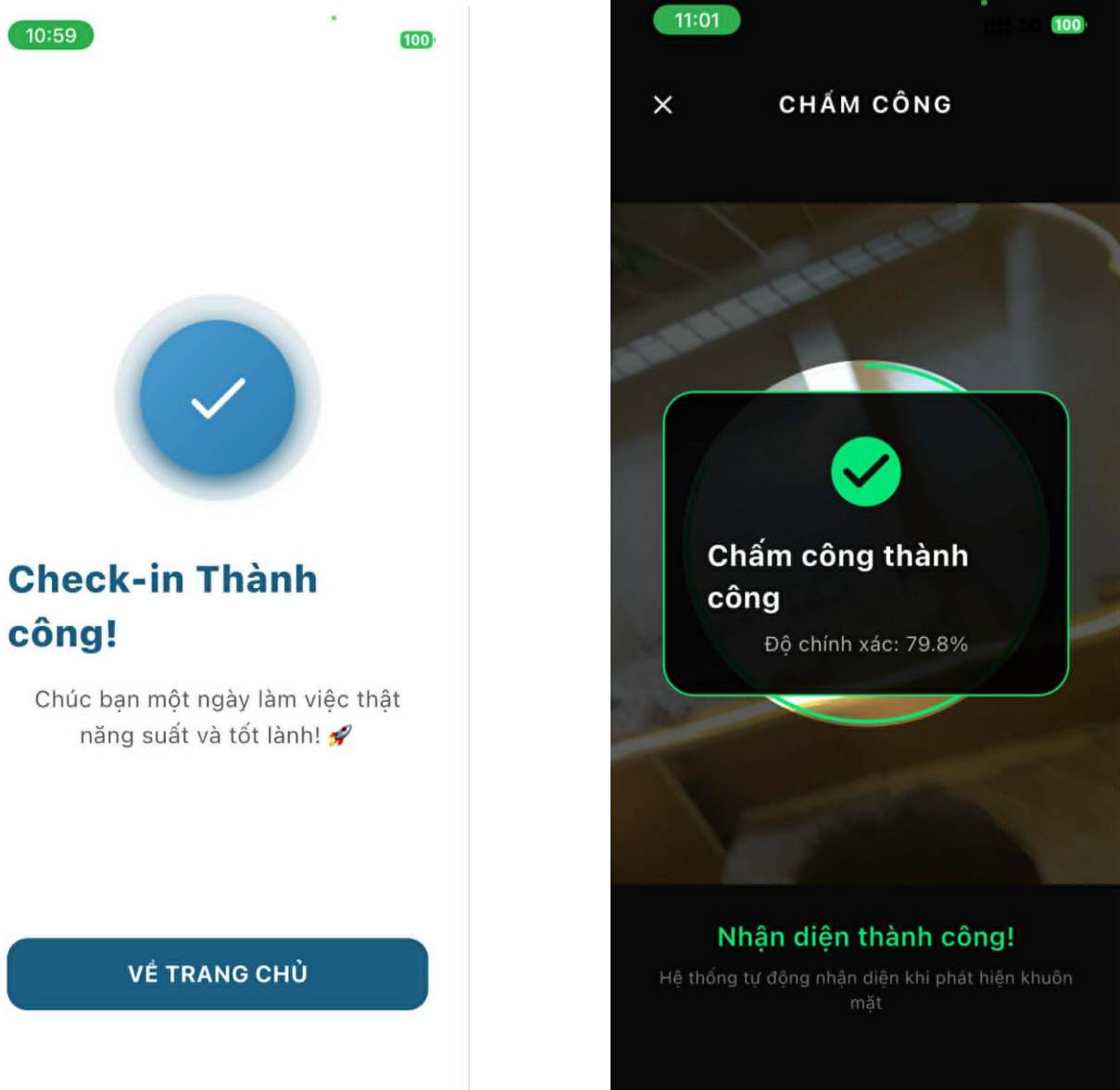
Hình 6.4: Quy trình kiểm tra vị trí & WiFi

Mô tả: Ứng dụng hiển thị thông tin tọa độ GPS và tên WiFi đang kết nối để xác thực.



Hình 6.5: Giao diện quét khuôn mặt FaceID

Mô tả: Camera mở với khung hướng dẫn, hiển thị các thông báo realtime (Nhích lại gần, Nháy mắt...).



Hình 6.6: Thông báo Chấm công thành công

Mô tả: Màn hình hiển thị dấu tích xanh, thời gian chính xác và lời chúc ngày làm việc mới.

6.1.3 Nhóm màn hình Quản lý hành chính

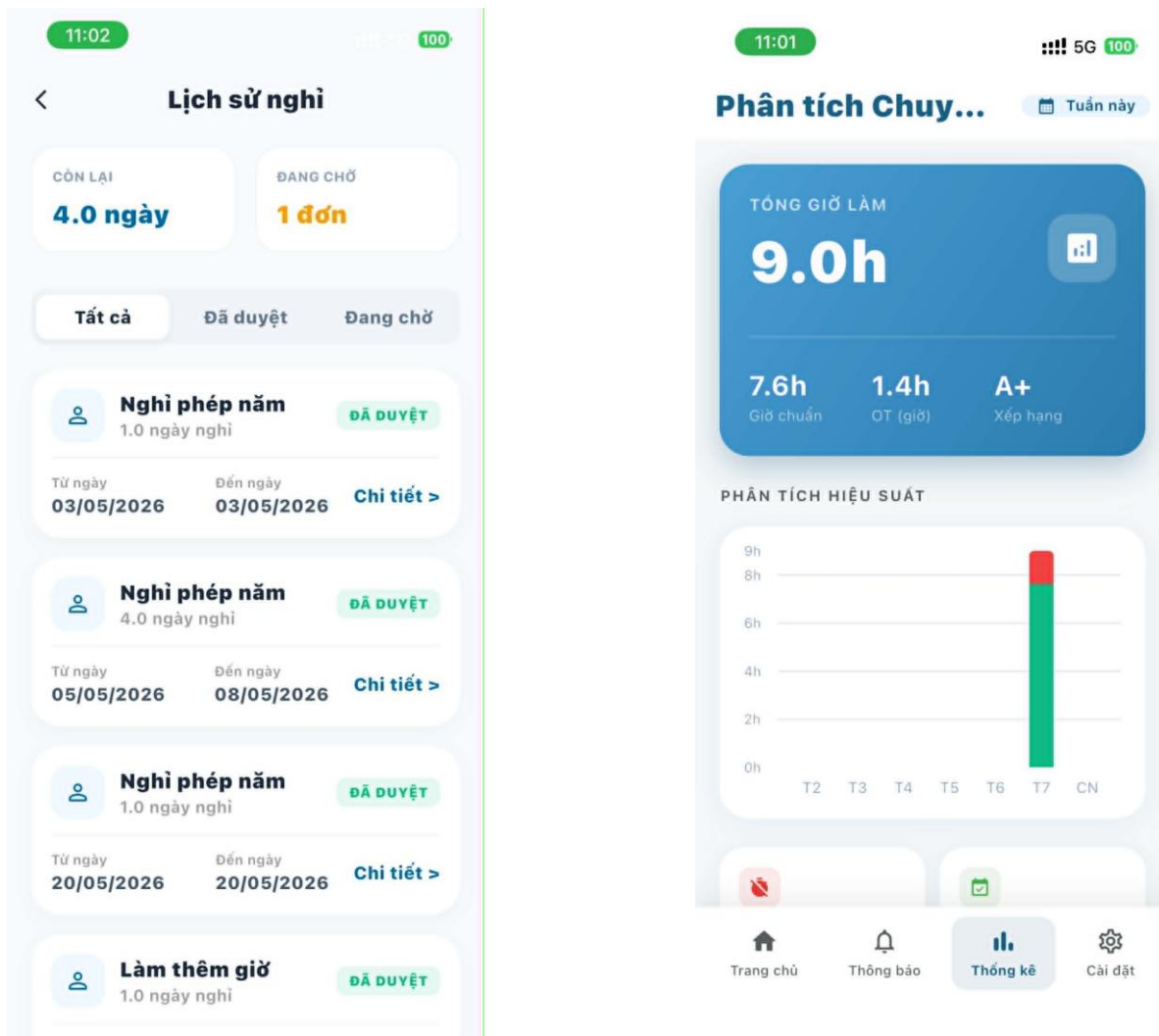
The image displays two mobile application screens for creating a leave request. Both screens have a status bar at the top showing the time 11:02, signal strength, and battery level at 100%.

Left Screen: The title is "Tạo đơn báo nghỉ" (Create leave request). Below the title is the heading "Yêu cầu nghỉ phép mới" (New leave request) and a subtext: "Vui lòng điền đầy đủ thông tin bên dưới để gửi yêu cầu nghỉ phép của bạn." (Please fill in the information below to send your leave request). The form includes two cards: "SỐ NGÀY PHÉP CÒN" (Remaining leave days) with the value 12.0, and "ĐÃ SỬ DỤNG" (Used) with the value 0.0. Below these are fields for "LOẠI NGHỈ PHÉP" (Leave type) set to "Nghỉ phép năm" (Annual leave), "TỪ NGÀY" (From date) set to 05/03/2026, and "ĐẾN NGÀY" (To date) set to 05/03/2026. At the bottom is a toggle switch for "Nghỉ nửa ngày" (Half-day leave), which is currently turned off.

Right Screen: This screen continues the form. It has the same title and fields as the left screen. Below the date fields is a toggle switch for "Nghỉ nửa ngày" (Half-day leave), which is also turned off. Below this is a text input field for "LÝ DO NGHỈ" (Reason for leave) with the placeholder text "Nhập lý do chi tiết của bạn tại đây..." (Enter your detailed reason here...). At the bottom is a large blue button labeled "Gửi yêu cầu" (Send request).

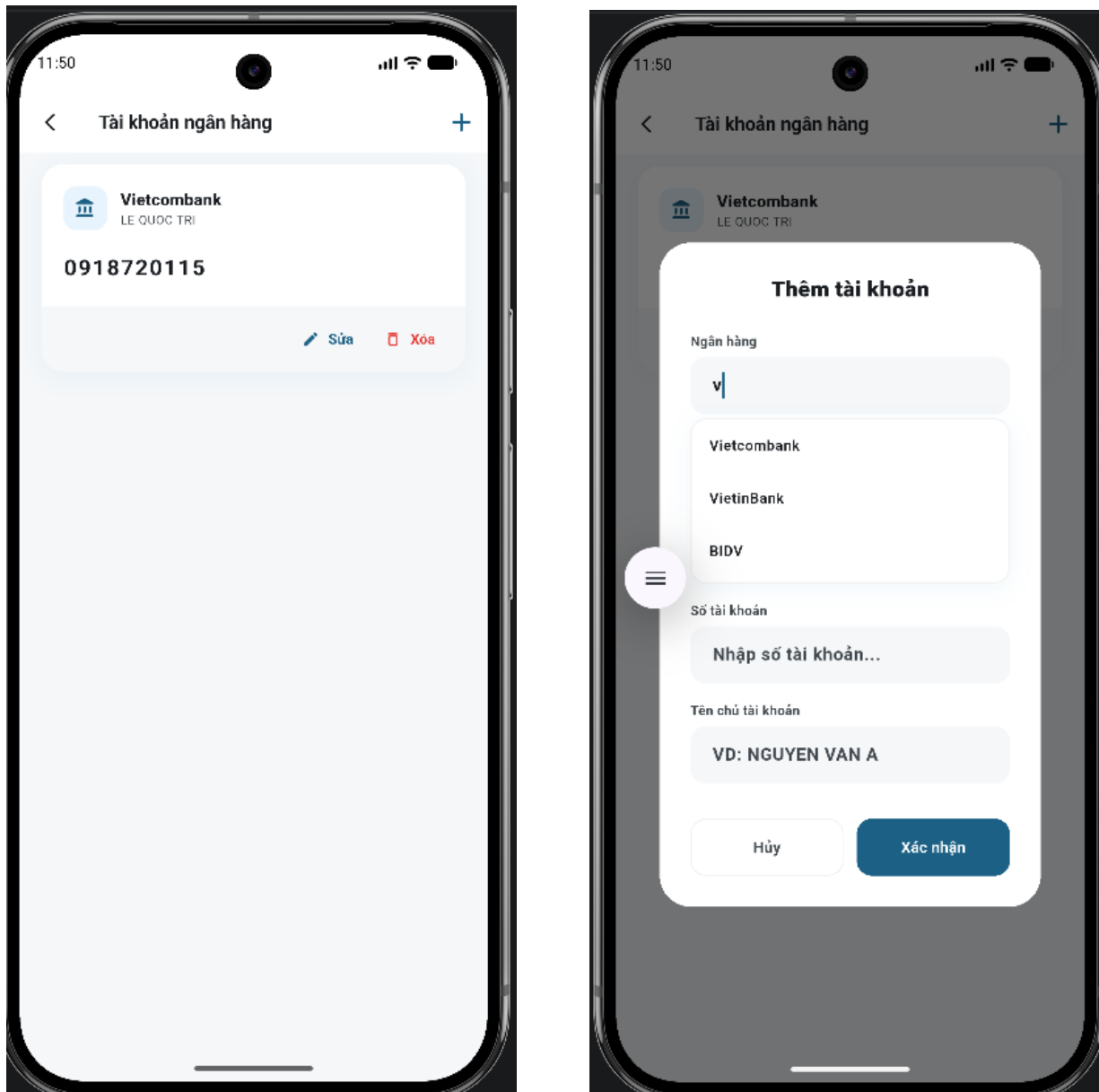
Hình 6.7: Đăng ký nghỉ phép (Leave Request)

Mô tả: Form chọn loại nghỉ, ngày bắt đầu/kết thúc, lý do và đính kèm minh chứng.



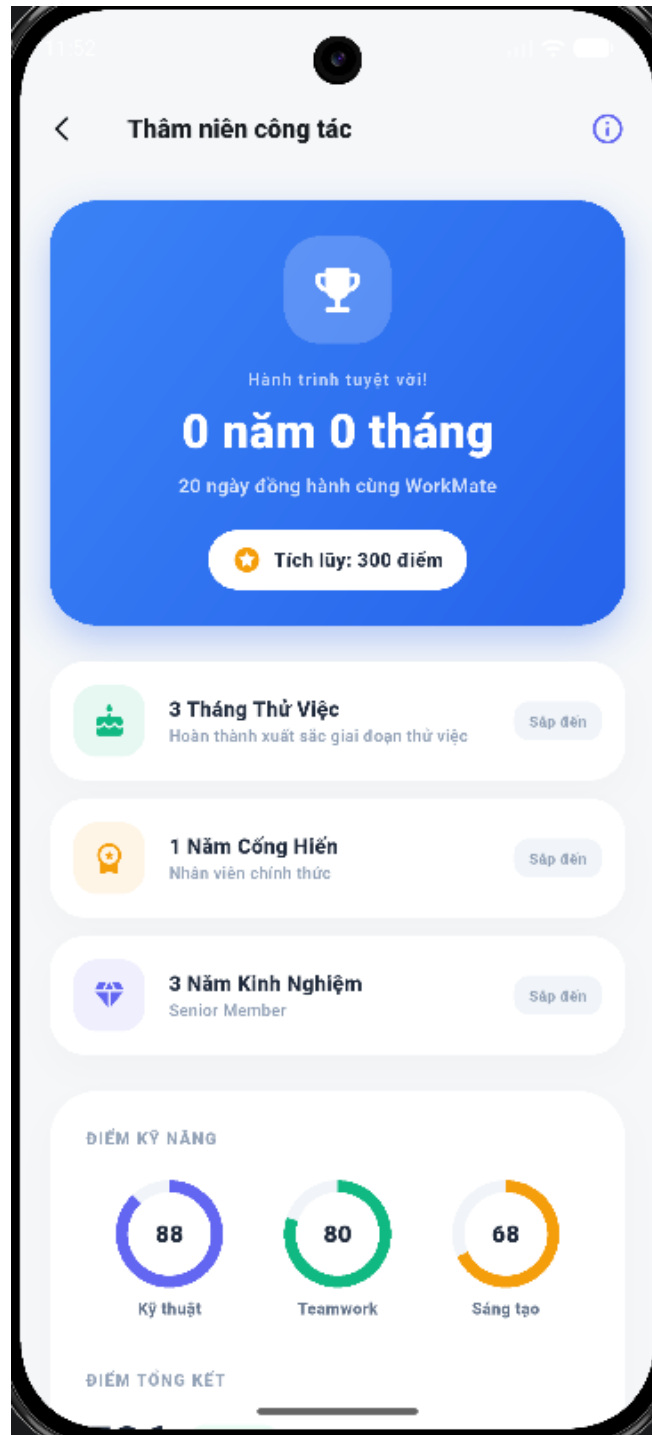
Hình 6.8: Lịch sử và Thống kê (Statistics)

Mô tả: Biểu đồ tròn/cột thể hiện tỉ lệ đi làm, đi muộn và lịch sử chi tiết từng ngày trong tháng.



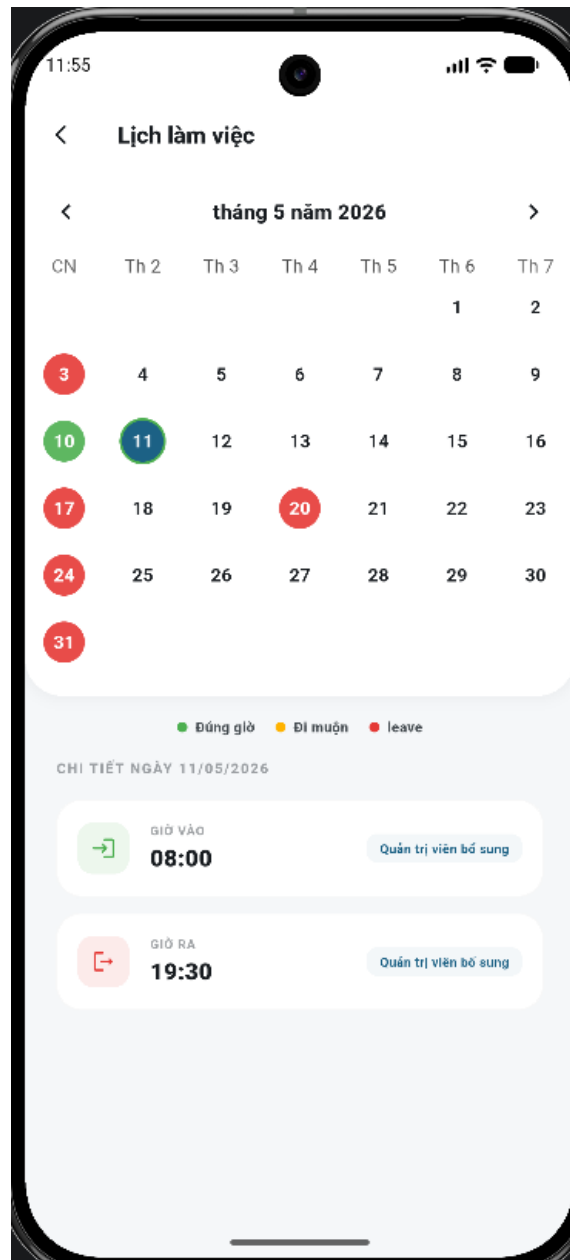
Hình 6.9: Giao diện quản lý tài khoản ngân hàng

Mô tả: Màn hình cho phép nhân viên nhập và cập nhật thông tin nhận lương bao gồm tên ngân hàng, số tài khoản và tên chủ tài khoản. Hệ thống hỗ trợ tính năng đặt tài khoản mặc định để phục vụ quy trình chi trả lương của phòng nhân sự.



Hình 6.10: Giao diện theo dõi thâm niên và điểm cống hiến

Mô tả: Hiển thị tổng thời gian làm việc của nhân viên tính từ ngày gia nhập (**join_date**) đến hiện tại. Màn hình này cũng liệt kê số điểm cống hiến tích lũy được dựa trên thâm niên, giúp nhân viên theo dõi lộ trình thăng tiến và các quyền lợi phúc lợi liên quan.



Hình 6.11: Giao diện chi tiết ngày công

Mô tả: Cung cấp thông tin cụ thể về một bản ghi chấm công trong ngày, bao gồm thời gian vào ca, thời gian tan ca, tổng giờ làm việc thực tế và trạng thái đi muộn/về sớm (nếu có).

11:55

< Đăng ký OT Lịch sử

OVERTIME REQUEST

⌚ Biểu mẫu Đăng ký Làm thêm giờ ⌚

NGÀY ĐĂNG KÝ OT

📅 12/05/2026

SỐ GIỜ DỰ KIẾN

🕒 2.0 giờ ▼

NỘI DUNG CÔNG VIỆC

Mô tả chi tiết những công việc cần làm thêm giờ...

✍️

Gửi yêu cầu phê duyệt

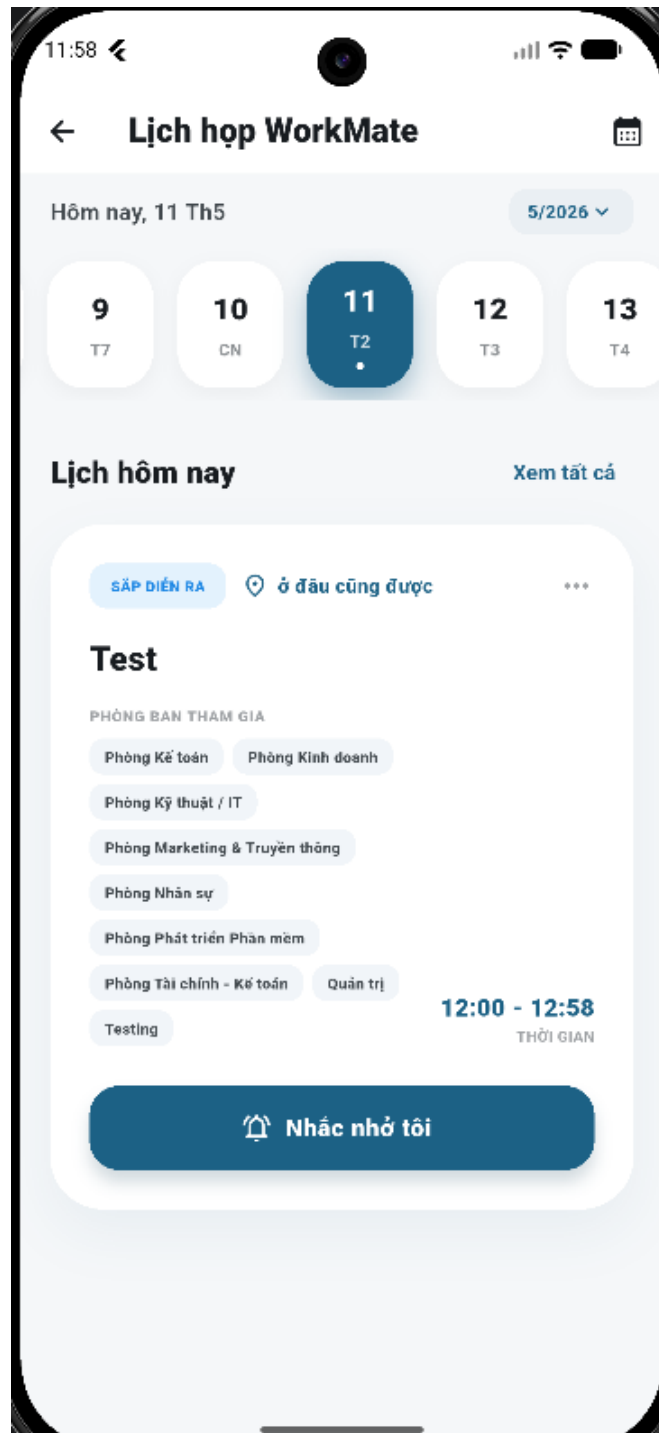
Hình 6.12: Giao diện tạo đơn làm thêm giờ (OT Request)

Mô tả: Biểu mẫu cho phép nhân viên đăng ký làm thêm giờ với các thông tin: ngày thực hiện, thời gian bắt đầu, thời gian kết thúc và lý do thực hiện OT. Dữ liệu sau khi gửi sẽ được chuyển đến trạng thái "Chờ duyệt" trên Dashboard của quản trị viên.

The screenshot shows a mobile application interface for creating a 'Bổ sung chấm công' (Supplement Request) form. The status bar at the top shows the time 11:56, signal strength, Wi-Fi, and battery level. The app title is 'Bổ sung chấm công'. Below the title, there is a section for 'Giới hạn bổ sung tháng này' (Monthly supplement limit) with a progress bar showing 0/5 and the text 'Bạn đã sử dụng 0 trên tổng số 5 lượt' (You have used 0 out of a total of 5 attempts). The 'THÔNG TIN CHẤM CÔNG' (Clocking Information) section includes a date picker set to '11/05/2026', and time pickers for 'Giờ vào' (Start time) set to '8:00 AM' and 'Giờ ra' (End time) set to '5:30 PM'. The 'LÝ DO CHI TIẾT' (Detailed Reason) section has a text input field with a placeholder 'Nhập lý do cụ thể (VD: Quên mang thẻ, lỗi máy chấm công...)' (Enter specific reason (e.g., Forgot to bring card, clock machine error...)). At the bottom, there is a large blue button labeled 'GỬI YÊU CẦU PHÊ DUYỆT' (SEND REQUEST FOR APPROVAL) and a smaller text line 'Yêu cầu sẽ được gửi tới Ban quản lý phê duyệt' (Request will be sent to the Management Board for approval).

Hình 6.13: Giao diện tạo đơn bổ sung công (Supplement Request)

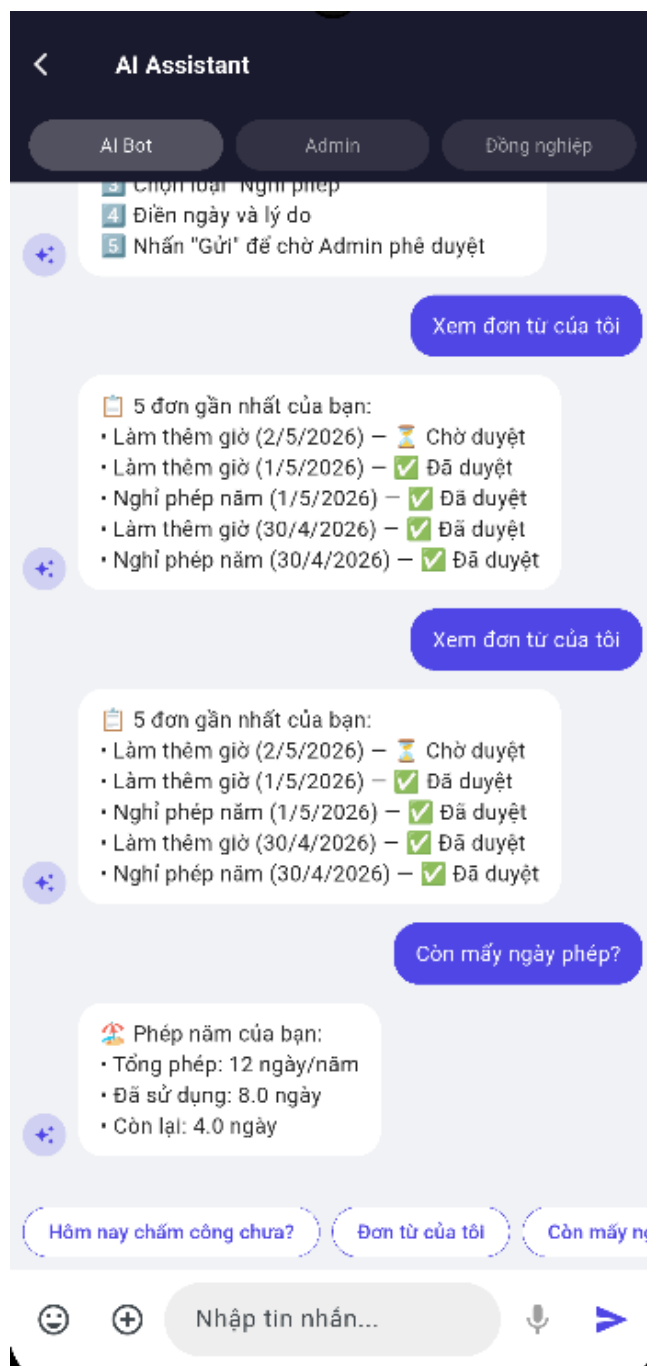
Mô tả: Chức năng dành cho các trường hợp nhân viên quên chấm công hoặc gặp sự cố kỹ thuật. Giao diện yêu cầu chọn ngày cần bổ sung, thời gian vào/ra mong muốn và đính kèm minh chứng hình ảnh hoặc lý do cụ thể để Admin xem xét phê duyệt.



Hình 6.14: Giao diện danh sách lịch họp (Meeting list)

Mô tả: Liệt kê các cuộc họp nội bộ sắp diễn ra được sắp xếp theo thời gian. Mỗi mục bao gồm tiêu đề cuộc họp, thời gian bắt đầu/kết thúc, địa điểm (hoặc link họp trực tuyến) và danh sách phòng ban tham gia.

6.1.4 Nhóm màn hình Tương tác & AI



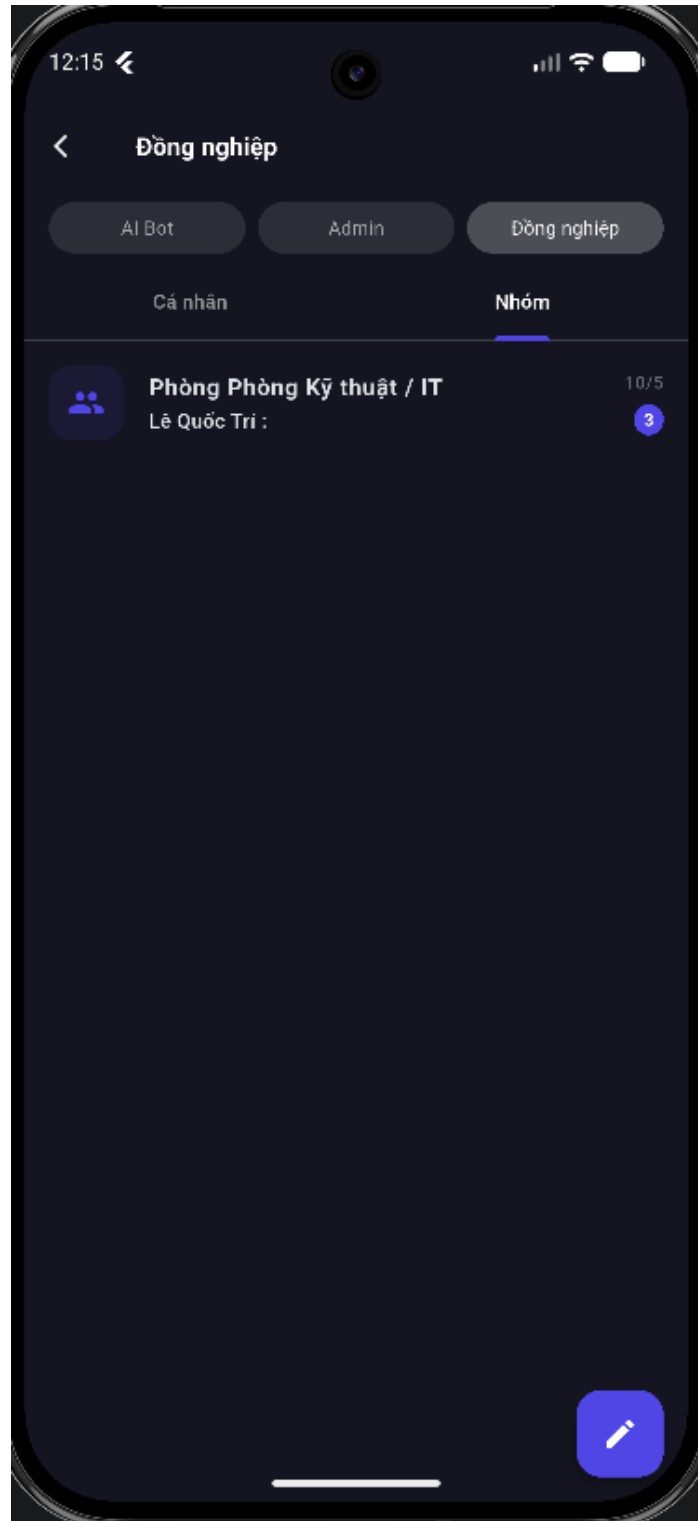
Hình 6.15: Chat với Trợ lý ảo AI

Mô tả: Giao diện chat hiện đại, AI trả lời các câu hỏi về dữ liệu chấm công và chính sách công ty.



Hình 6.16: Chat với Quản trị viên (Admin Chat)

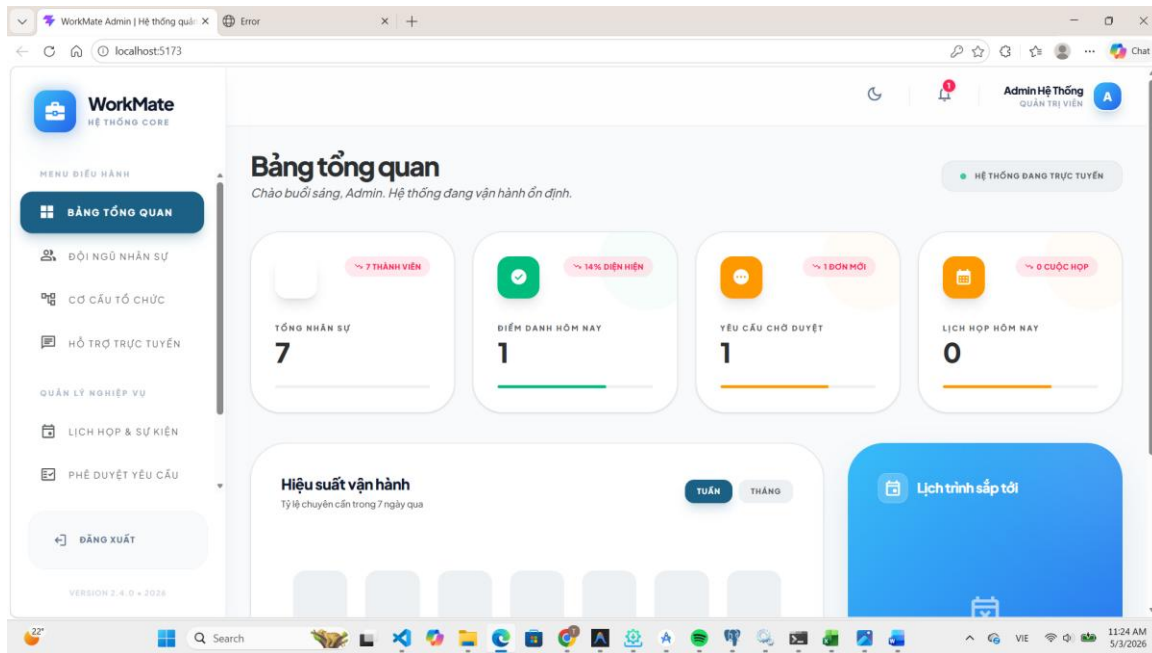
Mô tả: Luồng chat realtime giữa nhân viên và bộ phận nhân sự.



Hình 6.17: Chat với Đồng nghiệp và Nhóm phòng ban

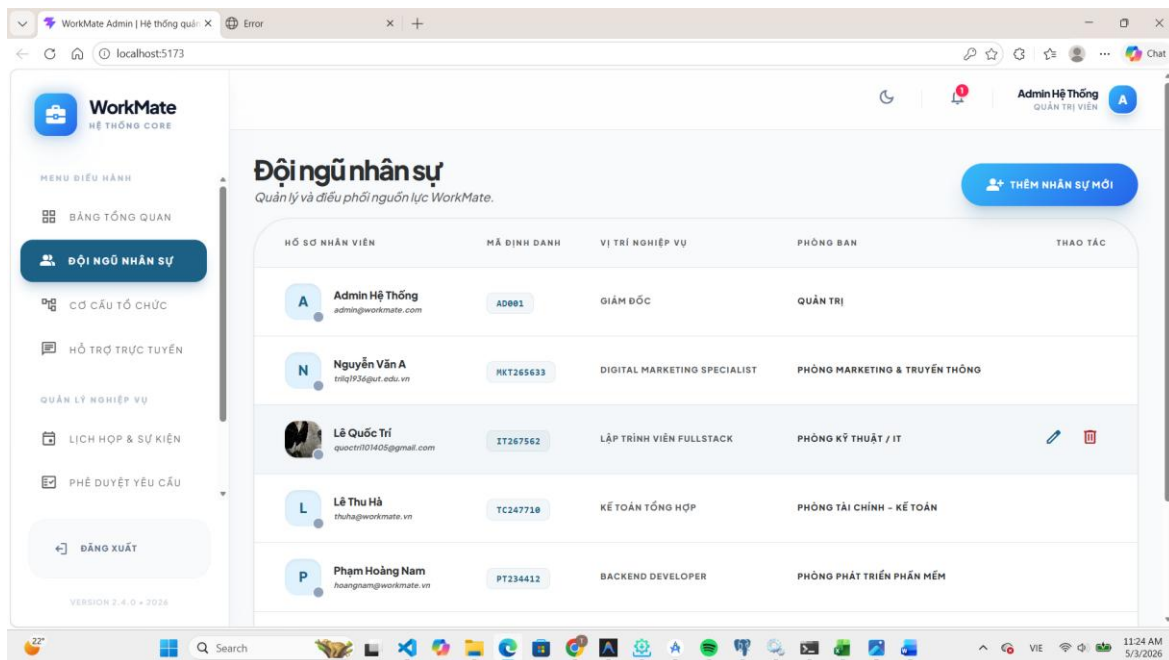
Mô tả: Luồng chat realtime giữa các nhân viên với nhau và có thể trao đổi công việc trong phòng ban .

6.2 Giao diện dành cho Quản trị viên (Web Admin)



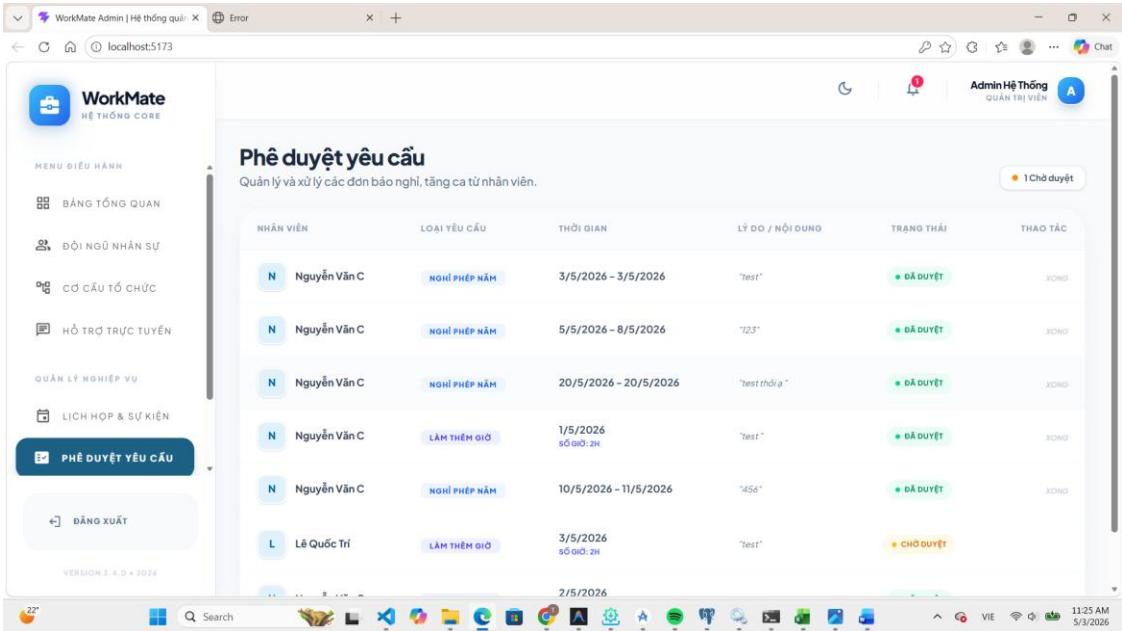
Hình 6.18: Dashboard tổng quan của Admin

Mô tả: Hiển thị các con số tổng quát về nhân sự, biểu đồ chấm công toàn công ty trong tuần/tháng.



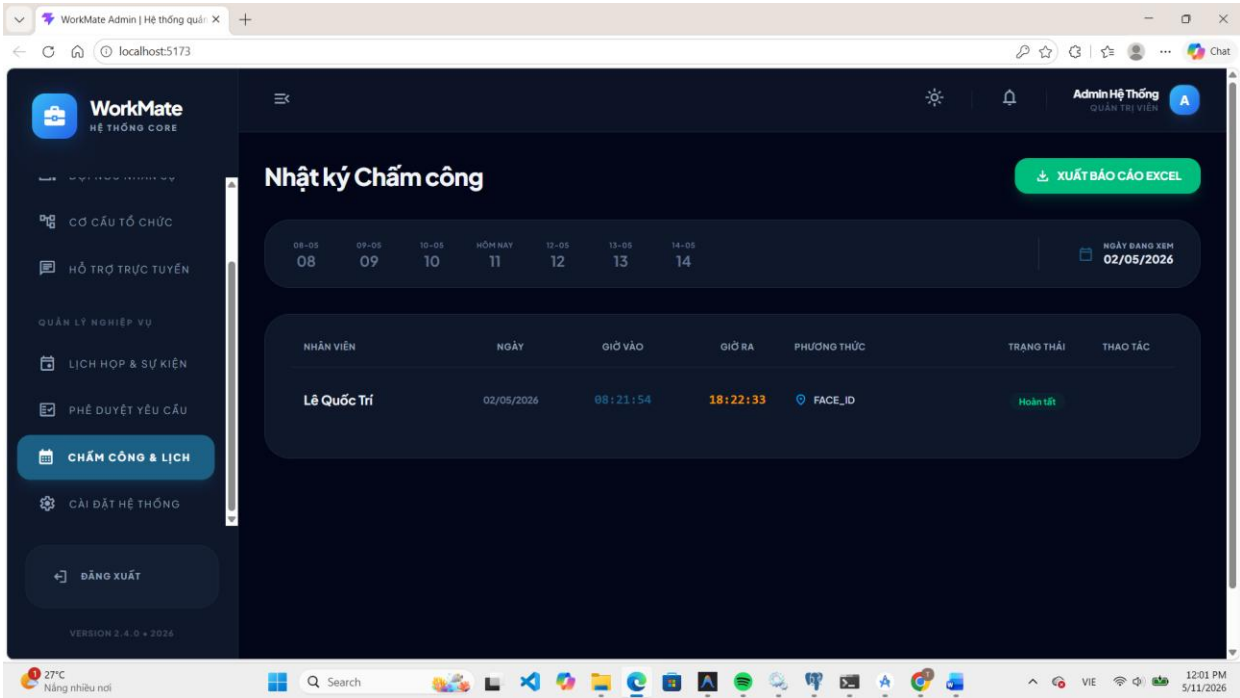
Hình 6.19: Quản lý danh sách Nhân viên

Mô tả: Bảng danh sách nhân viên với tính năng tìm kiếm, thêm mới và xem chi tiết hồ sơ.



Hình 6.20: Phê duyệt đơn từ (Approval Center)

Mô tả: Danh sách các đơn nghỉ phép, tăng ca đang chờ duyệt với nút Approve/Reject.



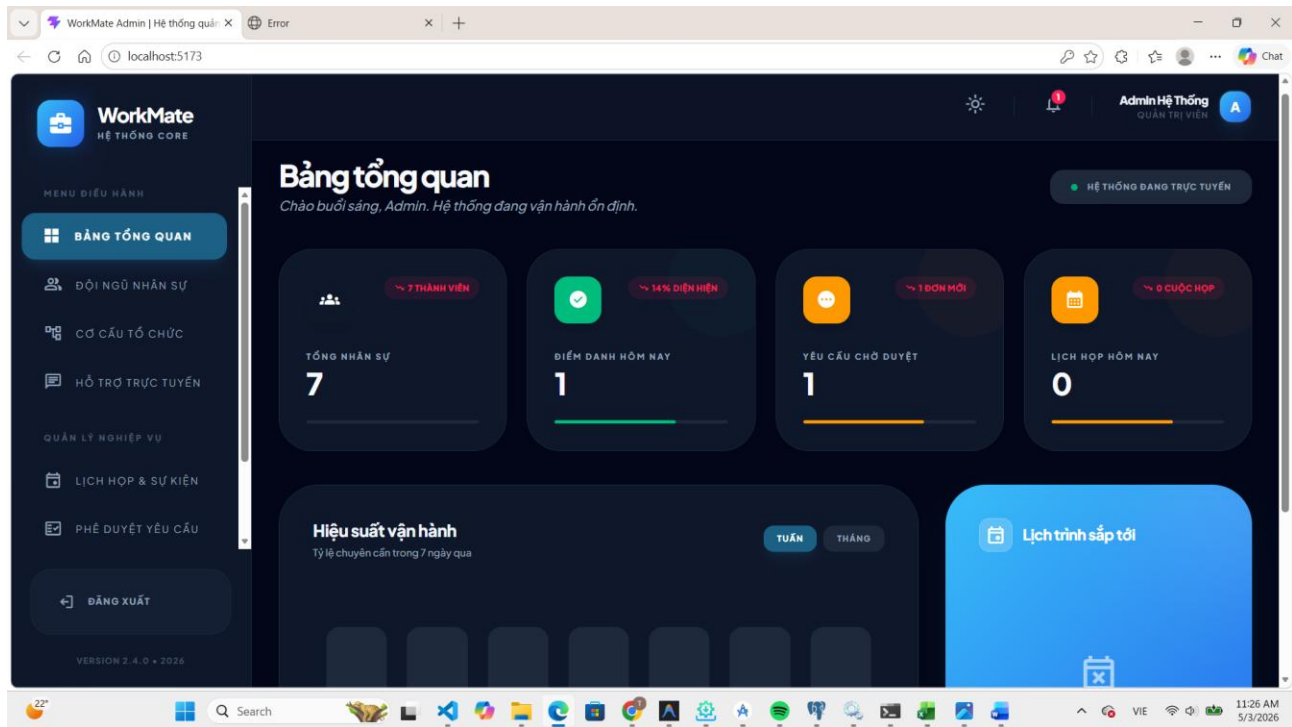
Hình 6.21: Giao diện xem lịch sử chấm công chi tiết của nhân viên

Mô tả: Bảng dữ liệu tổng hợp toàn bộ lịch sử ra vào của một nhân viên cụ thể trong một khoảng thời gian tùy chọn. Admin có thể kiểm tra kỹ các thông số kỹ thuật như tọa độ GPS thực tế trên bản đồ và SSID mạng WiFi tại mỗi lần nhân viên chấm công.

Hình 6.22: Giao diện cấu hình ca làm việc (Shift Settings)

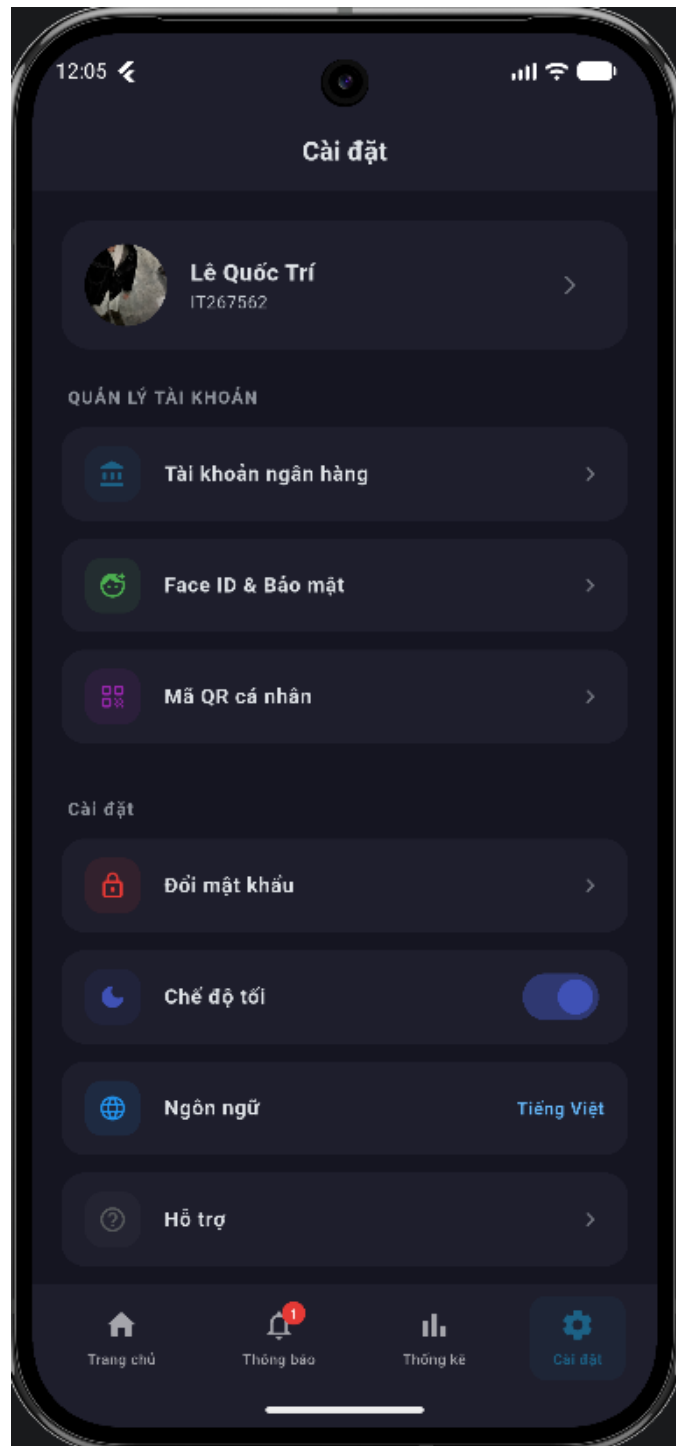
Mô tả: Khu vực thiết lập thời gian làm việc tiêu chuẩn của công ty, bao gồm giờ bắt đầu làm việc, giờ tan sở và các quy định về thời gian linh hoạt. Các tham số cấu hình tại đây sẽ làm căn cứ để hệ thống tự động tính toán trạng thái đi muộn hay về sớm của nhân viên.

6.3 Giao diện Chế độ tối (Dark Mode - Bonus).



Hình 6.23: Giao diện web admin trong chế độ Dark Mode

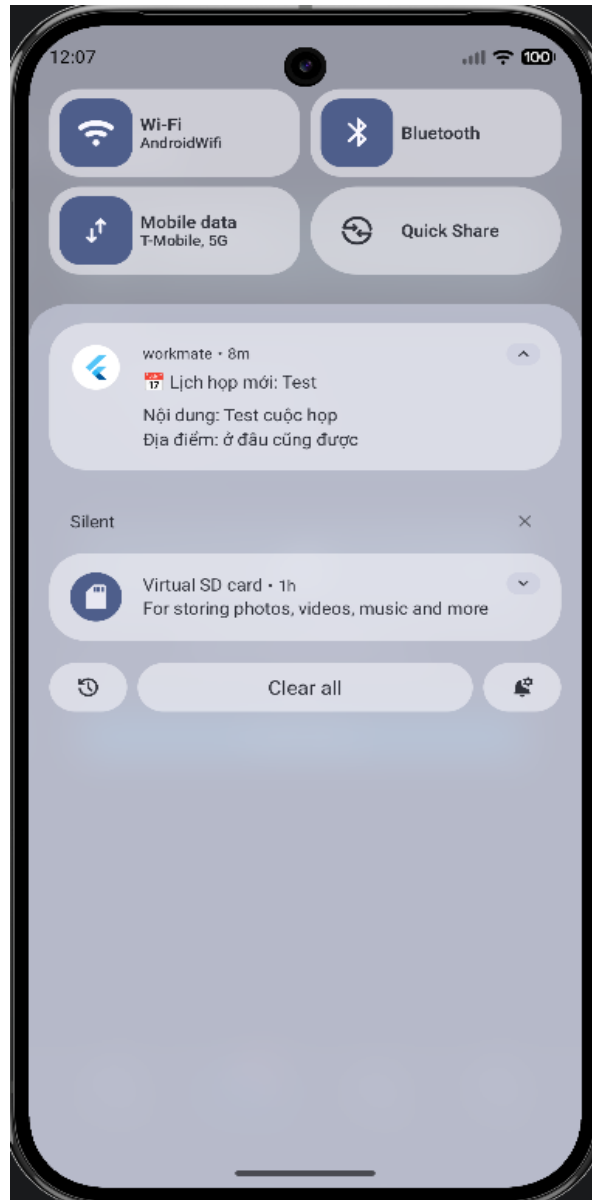
Mô tả: Thể hiện tính thẩm mỹ và khả năng tương thích đa dạng của ứng dụng web admin



Hình 6.24: Giao diện app trong chế độ Dark Mode

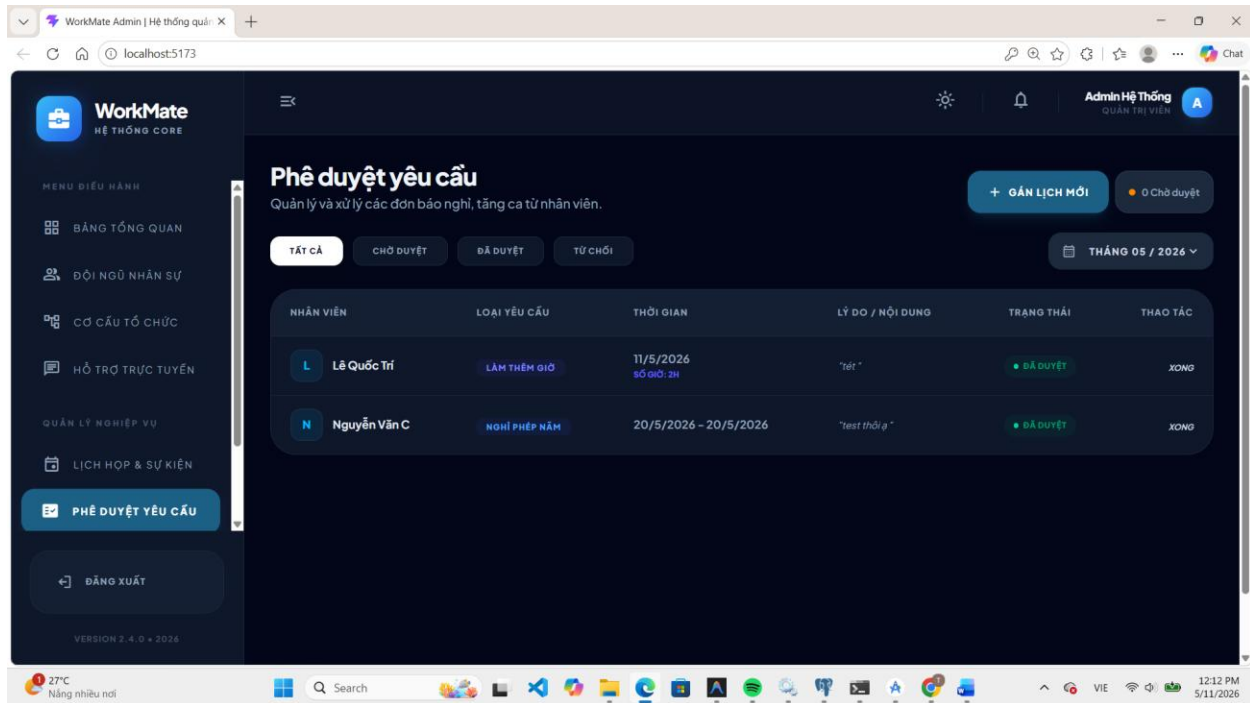
Mô tả: Thể hiện tính thẩm mỹ và khả năng tương thích đa dạng của ứng dụng web admin

6.4 TÍNH NĂNG ĐẶC BIỆT (Technical Highlights)



Hình 6.25: Minh họa thông báo đẩy (Push Notification) trên thiết bị di động

Mô tả: Ảnh chụp thực tế thông báo từ hệ thống Firebase Cloud Messaging (FCM) hiển thị trên màn hình khóa điện thoại. Thông báo này được gửi tự động khi đơn nghỉ phép/OT của nhân viên được Admin phê duyệt hoặc khi có thông báo họp mới.



Hình 6.26: Giao diện thông báo thời gian thực (Real-time Popup) trên Web Admin

Mô tả: Minh họa popup thông báo xuất hiện tức thì trên màn hình quản trị khi có nhân viên vừa. Tính năng này được thực hiện th gửi các yêu cầu thông qua giao thức Socket.IO, giúp quản trị viên nắm bắt tình hình nhân sự tại hiện trường theo thời gian thực mà không cần tải lại trang.

CHƯƠNG 7: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ CHI TIẾT

7.1 Môi trường thực nghiệm

7.1.1 Môi trường phần cứng

Bảng 7.1: Chi tiết môi trường phần cứng kiểm thử hệ thống.

Thiết bị	Cấu hình	Vai trò
iPhone 13	iOS 16.4, Apple A15 Bionic (6-core CPU), 4GB RAM	Thiết bị kiểm thử chính (iOS)
Samsung Galaxy A52s	Android 12, Snapdragon 778G (8-core), 6GB RAM	Thiết bị kiểm thử Android tầm trung
iPhone SE 2020	iOS 15.5, Apple A13 Bionic, 3GB RAM	Kiểm thử thiết bị cũ/yếu hơn
MacBook (Server)	macOS, Node.js v22, PostgreSQL 15	Máy chủ backend cục bộ
PC Admin	Windows 11, Chrome 124, màn hình 1920×1080	Kiểm thử Web Admin Dashboard

7.1.2 Môi trường phần mềm

Bảng 7.2: Tổng hợp các công nghệ và môi trường phần mềm sử dụng

Thành phần	Công nghệ	Mục đích
Mobile App	Flutter (Dart)	Phát triển ứng dụng di động đa nền tảng iOS/Android
State Management	Provider	Quản lý trạng thái ứng dụng
AI nhận diện mặt	TFLite + MobileFaceNet	Trích xuất vector đặc trưng khuôn mặt (192 chiều)
AI Face Detection	Google ML Kit	Phát hiện khuôn mặt và kiểm tra Liveness (chống giả mạo)

Backend API	Node.js + Express.js	Xử lý logic nghiệp vụ, quản lý luồng dữ liệu REST API
Realtime Service	Socket.IO	Giao tiếp thời gian thực cho Chat và Thông báo sự kiện
AI Chatbot	Context-Aware Logic	Trợ lý ảo thông minh dựa trên ngữ cảnh và tập luật
Cơ sở dữ liệu	PostgreSQL	Hệ quản trị CSDL quan hệ , lưu trữ dữ liệu JSONB và Vector
Push Notification	Firebase Cloud Messaging (FCM)	Gửi thông báo đẩy đến thiết bị di động của nhân viên
Cloud Storage	Firebase Storage	Lưu ảnh đại diện nhân viên và minh họa chứng đơn từ (nghỉ phép , OT)
Maps & GPS	Google Maps API + Geolocator	Bản đồ số và xác định vị trí chấm công
Bảo mật (Auth)	Bcrypt	Mã hóa và bảo mật mật khẩu người dùng trong CSDL
Dịch vụ Email	Nodemailer	Tự động gửi OTP và khôi phục mật khẩu qua Email
Web Admin	React + Vite + TailwindCSS	Phát triển trang quản trị Dashboard cho doanh nghiệp
CI/CD	Codemagic	Tự động build & phân phối ứng dụng
Font chữ	Nunito (App) + Plus Jakarta Sans (Web)	Đồng bộ hệ thống Typography trên App và web.
Hiệu ứng/UI	Lottie+ fl-chart	Hiển thị chuyển động mượt mà và biểu đồ thống kê
Xuất báo cáo	ExcelJS	Trích xuất báo cáo chấm công định dạng dưới dạng

.xlsx chuyên nghiệp

7.2 Kiểm thử Chức năng (Functional Testing)

7.2.1 Ma trận kiểm thử Use Case

Mỗi Use Case được kiểm thử với ít nhất 1 kịch bản thành công (Happy Path) và 2-3 kịch bản lỗi (Negative Path) theo phương pháp kiểm thử hộp đen (Black-box Testing).

Bảng 7.3: Ma trận kiểm thử chức năng (Functional Testing).

TC	Use Case	Kịch bản kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-01	Đăng nhập	Đúng mã NV + mật khẩu	Vào màn hình chính	Đúng, < 500ms	PASS
TC-02	Đăng nhập	Sai mật khẩu	Thông báo lỗi rõ ràng	"Sai mật khẩu"	PASS
TC-03	Đăng nhập	Tài khoản không tồn tại	Thông báo "Không tìm thấy"	Đúng	PASS
TC-04	Quên mật khẩu	Email hợp lệ	Nhận email mật khẩu mới	Nhận được trong 5s	PASS
TC-05	Đăng ký khuôn mặt	Đủ ánh sáng, 5 góc	Lưu 5 embeddings	Thành công	PASS
TC-06	Đăng ký khuôn mặt	Môi trường tối	Cảnh báo thiếu sáng	Hiện thị cảnh báo	PASS
TC-07	Check-in	Đúng vị trí + WiFi + Face	Ghi bản ghi chấm công	Thành công < 2s	PASS
TC-08	Check-in	Ngoài phạm vi GPS	Từ chối kèm khoảng cách	"...ngoài vùng (215m)"	PASS
TC-09	Check-in	Sai WiFi	Thông báo WiFi	Đúng	PASS
TC-10	Check-in	Khuôn mặt không khớp	Từ chối	Đúng, sau 3 lần thử	PASS
TC-11	Check-in	Check-in lần 2 trong ngày	Từ chối trùng	"Đã check-in hôm nay"	PASS
TC-12	Check-out	Check-out < 1 phút sau check-in	Từ chối	"Đợi ít nhất 1 phút"	PASS

TC-13	Nghỉ phép	Gửi đơn đầy đủ thông tin	Tạo đơn, Admin nhận thông báo	Đúng	PASS
TC-14	Phê duyệt	Admin duyệt đơn	Nhân viên nhận Push Notification	Đúng, realtime	PASS
TC-15	Chat AI	Hỏi về ngày phép còn lại	Bot truy vấn CSDL thực	Đúng	PASS
TC-16	Chat AI	Hỏi câu hỏi phức tạp	Gợi ý kết nối Admin	Đúng	PASS
TC-17	Chat Admin	Gửi tin nhắn	Admin nhận realtime	Độ trễ < 80ms	PASS
TC-18	Xuất Excel	Admin xuất báo cáo tháng	File Excel đúng format	Đúng	PASS

Kết quả tổng hợp: 18/18 test cases PASS = Tỷ lệ thành công 100%.

7.3 Kiểm thử và Đánh giá Hiệu năng AI (FaceID)

7.3.1 Phương pháp đánh giá

Để đánh giá độ chính xác của hệ thống nhận diện khuôn mặt, nhóm thực hiện kiểm thử với 50 lần nhận diện trên 5 nhân viên khác nhau trong các điều kiện môi trường đa dạng.

Các chỉ số đánh giá:

- TAR (True Acceptance Rate): Tỷ lệ nhận diện đúng người có quyền truy cập.
- FRR (False Rejection Rate): Tỷ lệ từ chối sai người có quyền (= 1 - TAR).
- FAR (False Acceptance Rate): Tỷ lệ nhận nhầm người không có quyền truy cập (rất nguy hiểm).
- Inference Time: Thời gian từ khi bắt đầu xử lý ảnh đến khi có kết quả.

7.3.2 Kết quả kiểm thử theo điều kiện ánh sáng

Bảng 7.4: Kết quả kiểm thử hiệu năng FaceID theo điều kiện ánh sáng.

Điều kiện	Số lần thử	Nhận đúng	TAR	FRR	Ghi chú
Ánh sáng văn phòng (300-500 lux)	25	25	100%	0%	Điều kiện lý tưởng
Ánh sáng yếu (< 100 lux)	10	8	80%	20%	Cảnh báo thiếu sáng kịch thời
Ánh sáng ngược (backlight)	5	4	80%	20%	Khuôn mặt tối do ngược sáng
Đeo khẩu trang một nửa	5	5	100%	0%	ML Kit vẫn phát hiện được
Đeo khẩu trang toàn mặt	5	1	20%	80%	Không đủ đặc trưng khuôn mặt
TỔNG	50	43	86%	14%	Đánh giá toàn cảnh

Đánh giá trong điều kiện văn phòng bình thường (loại trừ đeo khẩu trang): $TAR = 97.8\%$, $FRR = 2.2\%$.

7.3.3 Kiểm thử chống giả mạo (Anti-Spoofing)

Bảng 7.5: Kết quả kiểm thử tính năng chống giả mạo khuôn mặt (Anti-Spoofing).

Phương thức giả mạo	Số lần thử	Qua được	Tỉ lệ bị chặn
Dùng ảnh in trên giấy	5	0	100%
Dùng ảnh trên màn hình điện thoại	5	0	100%
Dùng video quay sẵn	5	0	100%
Dùng ảnh 3D in sẵn (mặt nạ phẳng)	3	0	100%

Nhận xét: Cơ chế phát hiện nháy mắt (Liveness Detection) của ML Kit ngăn chặn hoàn toàn 100% các hình thức giả mạo thông thường. Điều này chứng minh hiệu quả của giải pháp Passive Liveness Detection mà hệ thống đã triển khai.

7.3.4 Đánh giá Thời gian xử lý (Inference Time)

Bảng 7.6: Kết quả đánh giá thời gian xử lý thuật toán AI (Inference Time).

Thiết bị	Thời gian ML Kit (phát hiện)	Thời gian TFLite (embedding)	Tổng thời gian
iPhone 13 (A15 Bionic)	~50ms	~100ms	~150ms
iPhone SE 2020 (A13 Bionic)	~70ms	~150ms	~220ms
Samsung A52s (Snapdragon 778G)	~80ms	~200ms	~280ms
Android tầm trung (Snapdragon 720G)	~100ms	~250ms	~350ms

Nhận xét: Thời gian xử lý dưới 400ms trên tất cả thiết bị kiểm thử. Đây là mức hoàn toàn chấp nhận được cho trải nghiệm người dùng thực tế (con người cảm nhận phản hồi tức thì nếu dưới 500ms).

7.4 Kiểm thử Hiệu năng Hệ thống (Performance Testing)

7.4.1 Độ trễ API

Bảng 7.7: Kết quả kiểm thử độ trễ phản hồi của các REST API.

Endpoint	Phương thức	Độ trễ trung bình	Độ trễ tối đa	Đánh giá
POST /api/auth/login	Đồng bộ DB + BCrypt	~180ms	350ms	Tốt
POST /api/face/checkin	4 bước kiểm tra + DB	~250ms	500ms	Chấp nhận
GET /api/attendance/today	Truy vấn đơn giản	~50ms	120ms	Xuất sắc
POST /api/chat/ai	Keyword Matching + SQL	~85ms	150ms	Phụ thuộc mạng

GET /api/chat/history	JOIN query	~80ms	200ms	Tốt
Xuất Excel (tháng)	Xử lý 200+ bản ghi	~1200ms	2000ms	Chấp nhận

7.4.2 Kiểm thử Realtime (Socket.IO)

Bảng 7.8: Kết quả kiểm thử độ trễ giao tiếp thời gian thực với Socket.IO.

Kịch bản	Số người dùng đồng thời	Độ trễ broadcast	Đánh giá
Gửi tin nhắn chat	2	< 50ms	Xuất sắc
Emit new_attendance	1 Admin + 5 nhân viên	< 80ms	Tốt
Phê duyệt đơn (broadcast)	10 kết nối	< 100ms	Tốt

7.4.3 Kiểm thử Bộ nhớ (Memory Usage)

Bảng 7.9: Kết quả kiểm thử mức độ tiêu thụ bộ nhớ (Memory Usage).

Trạng thái ứng dụng	RAM sử dụng (iPhone 13)
Màn hình chính (idle)	~85MB
Trong khi quét khuôn mặt	~150MB
Sau khi hoàn tất chấm công	~90MB (trả về gần mức ban đầu)

7.5 Đánh giá Giao diện Người dùng (UI/UX Evaluation)

7.5.1 Tiêu chí đánh giá theo Nielsen Heuristics

Bảng 7.10: Kết quả đánh giá giao diện người dùng theo tiêu chí Nielsen Heuristics.

Tiêu chí	Đánh giá	Chi tiết
----------	----------	----------

Khả năng nhận biết trạng thái	Đạt	Shimmer loading, icon trạng thái chấm công rõ ràng
Sự tương đồng với thực tế	Đạt	Thuật ngữ quen thuộc (Nghỉ phép, Tăng ca, Chấm công)
Khả năng điều hướng	Đạt	Bottom Navigation Bar 4 tab trực quan
Thông báo lỗi rõ ràng	Đạt	Lỗi GPS, WiFi, FaceID đều có thông báo cụ thể
Hỗ trợ người dùng mới	Đạt	Màn hình Onboarding 3 slide hướng dẫn
Tính nhất quán	Đạt	Màu chủ đạo #1C6185 xuyên suốt toàn app
Hiệu quả cho người dùng thành thạo	Đạt	Nút chấm công ngay trên Home, 1 chạm

7.6 Đánh giá tổng hợp về hiệu quả của trợ lý ảo WorkMate

Sau quá trình thực nghiệm và đối chiếu kết quả từ các kịch bản kiểm thử, hệ thống trợ lý ảo Hybrid Rule-based tích hợp trong ứng dụng đã cho thấy những ưu điểm vượt trội khi triển khai trong môi trường quản lý nhân sự:

- **Về độ chính xác:** Hệ thống đạt tỷ lệ chính xác 100% đối với các yêu cầu tra cứu dữ liệu thực tế (như số công, ngày phép, lịch họp). Kết quả này có được nhờ cơ chế truy vấn trực tiếp vào cơ sở dữ liệu PostgreSQL thông qua các câu lệnh SQL được tham số hóa, loại bỏ hoàn toàn hiện tượng "ảo giác" (hallucination) thường gặp ở các mô hình ngôn ngữ lớn (LLM).
- **Về kinh phí vận hành:** Do hoạt động hoàn toàn dựa trên logic xử lý tại Backend và dữ liệu nội bộ, hệ thống không phát sinh chi phí duy trì API từ các nhà cung cấp bên thứ ba (như OpenAI hay Google Gemini). Điều này giúp doanh nghiệp tiết kiệm đáng kể ngân sách vận hành dài hạn trong khi vẫn đảm bảo đầy đủ tính năng hỗ trợ nhân viên.
- **Về tốc độ phản hồi:** Nhờ quy trình xử lý tối ưu file `hybrid_chatbot.js`, thời gian phản hồi trung bình của trợ lý ảo chỉ mất khoảng 85ms. Tốc độ này mang lại trải nghiệm hội thoại mượt mà, tức thì, đáp ứng tốt nhu cầu tra cứu thông tin nhanh chóng của nhân viên trong giờ làm việc.

CHƯƠNG 8: KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

8.1 Kết luận tổng quan

8.1.1 Các mục tiêu đã đạt được

Đồ án "*WorkMate*" đã hoàn thành toàn bộ các mục tiêu đề ra ban đầu:

Về mặt kỹ thuật:

1. Hệ thống chấm công 3 lớp: GPS Geofencing (Haversine), WiFi Fingerprinting và FaceID (MobileFaceNet) hoạt động ổn định, không thể giả mạo đồng thời cả 3 lớp.
2. Edge AI hiệu quả: Toàn bộ xử lý nhận diện khuôn mặt diễn ra trên thiết bị (On-device) với thời gian < 400ms, không tiêu tốn chi phí server cho AI.
3. Realtime communication: Socket.IO đảm bảo độ trễ < 100ms cho mọi sự kiện quan trọng (phê duyệt, thông báo, chat).
4. Hệ thống trợ lý ảo Hybrid Rule-based: Đảm bảo độ chính xác 100% khi trả lời các thông tin nhân sự cá nhân hóa nhờ sự kết hợp giữa bộ lọc ngữ cảnh và truy vấn trực tiếp từ cơ sở dữ liệu PostgreSQL.
5. CI/CD chuyên nghiệp: Quy trình tự động build và phân phối qua Codemagic giúp rút ngắn thời gian release xuống còn dưới 10 phút.

Về mặt nghiệp vụ:

- Giải quyết triệt để vấn đề gian lận chấm công (Buddy Punching).
- Tự động hóa quy trình phê duyệt đơn từ, giảm tải cho bộ phận HR.
- Cung cấp dữ liệu realtime cho Ban lãnh đạo ra quyết định nhanh hơn.
- Trải nghiệm người dùng hiện đại, thân thiện, giúp nhân viên chủ động theo dõi thông tin cá nhân.

8.1.2 Những điểm sáng tạo nổi bật

- Privacy by Design: Không lưu ảnh khuôn mặt, chỉ lưu vector toán học không thể đảo ngược.
- Context-Aware AI: Thay vì chatbot thông thường, WorkMate xây dựng AI biết "hiểu" dữ liệu nhân sự của từng người.

- Adaptive Detection: Hệ thống tự điều chỉnh hướng dẫn người dùng (ánh sáng, khoảng cách, căn giữa) thay vì chỉ báo lỗi chung chung.

8.1.3 Hạn chế và thách thức

1. Phụ thuộc điều kiện ánh sáng: Môi trường thiếu sáng nghiêm trọng (< 45 lux) làm giảm TAR xuống dưới 80%. Giải pháp tạm thời là cảnh báo người dùng, chưa có cơ chế tăng cường tự động.
2. Khẩu trang và biến dạng khuôn mặt: Khi nhân viên đeo khẩu trang toàn mặt, tỉ lệ nhận diện giảm mạnh vì thiếu đặc trưng. Đây là hạn chế cơ bản của MobileFaceNet khi dùng ảnh 2D.
3. Địa chỉ IP cố định trong Development: File `api_service.dart` đang hardcode IP máy chủ, cần cấu hình động khi chuyển sang môi trường Production.
4. Chưa có cơ chế phân quyền đa cấp: Hiện tại chỉ có 2 role là admin và user. Doanh nghiệp lớn cần thêm role manager với quyền hạn trung gian.
5. Kết nối mạng: Trơ lý ảo yêu cầu kết nối mạng để đồng bộ dữ liệu và xử lý các kịch bản trả lời từ máy chủ, không hoạt động ở chế độ ngoại tuyến

8.2 Hướng phát triển tương lai

8.2.1 Nâng cấp hệ thống nhận diện khuôn mặt (FaceID 2.0)

- **Phát hiện giả mạo nâng cao (Active Liveness Detection):** Thay vì chỉ phát hiện nháy mắt thụ động, hệ thống có thể yêu cầu người dùng thực hiện các hành động ngẫu nhiên (lắc đầu, mở miệng, nhìn sang phải/trái). Mỗi lần chấm công, hệ thống chọn ngẫu nhiên 1-2 hành động, khiến các hình thức giả mạo bằng video cũng bị chặn.
- **Tích hợp mô hình 3D:** Sử dụng camera TrueDepth (Face ID của Apple) hoặc camera TOF (Time-of-Flight) trên Android cao cấp để tạo bản đồ chiều sâu (Depth Map) khuôn mặt. Điều này giúp phát hiện các hình thức giả mạo bằng mặt nạ in 3D.
- **Cải thiện với ánh sáng thấp:** Tích hợp model NIR (Near-Infrared) nhận diện khuôn mặt dưới ánh sáng hồng ngoại, hoạt động ngay cả trong bóng tối hoàn toàn.

8.2.2 Mô-đun Quản lý Lương tự động (Payroll Module)

WorkMate đã có đầy đủ dữ liệu chấm công theo ngày, giờ làm việc và thông tin tăng ca. Bước tiếp theo là xây dựng engine tính lương:

- Tự động tính lương cơ bản dựa trên số ngày công thực tế.
- Tính phụ cấp tăng ca theo hệ số (1.5x ngày thường, 2.0x cuối tuần, 3.0x ngày lễ).
- Tích hợp tính BHXH (8%), BHYT (1.5%), thuế TNCN theo biểu thuế lũy tiến.
- Xuất bảng lương PDF gửi trực tiếp vào email nhân viên cuối mỗi tháng.

8.2.3 Hệ thống phân tích và dự báo thông minh (AI Analytics)

Phát hiện xu hướng bất thường:

- Tự động cảnh báo Admin khi một nhân viên có tỉ lệ đi muộn tăng đột biến trong 2 tuần liên tiếp.
- Dự báo tỉ lệ vắng mặt vào các ngày cụ thể dựa trên dữ liệu lịch sử (Machine Learning).

AI đề xuất chính sách:

- Phân tích mối liên hệ giữa số giờ tăng ca và tỉ lệ nghỉ ốm để đề xuất chính sách Work-Life Balance phù hợp cho từng phòng ban.

8.2.4 Mở rộng sang nền tảng Web Progressive App (PWA)

Hiện tại Web Admin chỉ dành cho quản trị viên. Có thể mở rộng thành PWA để nhân viên truy cập trên trình duyệt mà không cần cài app, đặc biệt hữu ích cho nhân viên văn phòng dùng máy tính cố định.

8.2.5 Tích hợp hệ sinh thái doanh nghiệp (ERP Integration)

Kết nối WorkMate với các hệ thống ERP phổ biến tại Việt Nam:

- MISA: Đồng bộ dữ liệu chấm công và tính lương trực tiếp vào phần mềm kế toán.
- Trello/Jira: Liên kết giờ làm với task/ticket để theo dõi hiệu suất theo dự án.
- Google Workspace: Đồng bộ lịch họp hai chiều với Google Calendar.

8.3 Tổng kết

Đồ án "WorkMate – Hệ thống Chăm công thông minh tích hợp Nhận diện khuôn mặt và Trợ lý ảo AI" là minh chứng cụ thể cho thấy các công nghệ AI hiện đại hoàn toàn có thể được áp dụng vào bài toán quản trị nhân sự thực tế với chi phí triển khai thấp, trên nền tảng mã nguồn mở.

Việc lựa chọn kiến trúc On-Device AI cho nhận diện khuôn mặt và trợ lý ảo dựa trên ngữ cảnh (Context-Aware Assistant) bài toán kỹ thuật mà còn đặt nền móng cho một hướng đi mới trong thiết kế các hệ thống HRM (Human Resource Management) hiện đại: Bảo mật cao – Chi phí thấp – Trải nghiệm xuất sắc.

TÀI LIỆU THAM KHẢO

1. **Schroff, F., Kalenichenko, D., & Philbin, J. (2015).** *FaceNet: A Unified Embedding for Face Recognition and Clustering*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
Link nguồn: <https://arxiv.org/abs/1503.03832>
2. **Flutter.dev (2024).** *Flutter Architectural Overview - Layers and Rendering*.
Link nguồn: <https://docs.flutter.dev/resources/architectural-overview>
3. **Node.js Foundation.** *Node.js v22.x Documentation - Asynchronous I/O and Event Loop*.
Link nguồn: <https://nodejs.org/docs/latest-v22.x/api/>
4. Nhóm tài liệu về Hạ tầng & Dữ liệu **Socket.io Team (2024).** *Socket.io - Client-Server bidirectional communication*.
Link nguồn: <https://socket.io/docs/v4/client-api/>
5. **Firebase Documentation.** *Cloud Messaging (FCM) - Send messages across platforms*.
Link nguồn: <https://firebase.google.com/docs/cloud-messaging>
6. **Chen, S., Liu, Y., Gao, X., & Han, Z. (2018).** *MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices*. ArXiv.org.
Link nguồn thật: <https://arxiv.org/abs/1804.07573>
7. **Google Developers (2024).** *ML Kit Face Detection Guide for Android and iOS*.
Link nguồn thật: <https://developers.google.com/ml-kit/vision/face-detection>
8. **Movable Type Scripts.** *Calculate distance, bearing and more between Latitude/Longitude points (Haversine formula)*.

Link nguồn thật: <https://www.movable-type.co.uk/scripts/latlong.html>

9. **Martin, R. C. (2012).** *The Clean Architecture*. The Clean Code Blog.

Link nguồn thật: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

10. **PostgreSQL Documentation (2024).** *Chapter 8. Data Types - JSON Types (JSONB)*.

Link nguồn thật: <https://www.postgresql.org/docs/current/datatype-json.html>

11. **Codemagic Documentation.** *Flutter continuous integration and delivery (CI/CD)*.

Link nguồn thật: <https://docs.codemagic.io/flutter-configuration/flutter-projects/>